

accumulatedMutations_MEF

Qun Li

19 Aug 2025 at 10:18:05

Contents

1	Env	2
2	Libraries	2
3	Def function	4
4	Load Data	7
5	Observed Accumulated Mutation Num	7
6	Imputed Accumulated Mutation Num	9
7	Accumulated Mutation Num normalized by cell number	13
8	MPGgenotypecheck	15
9	Calculating mutations of each cell and cells of each mutation	17
10	calMulti	18
11	scMutScore Model Validation	26
12	compareClonesScore	37
13	compareClonesScore figures	42
14	save figures	55
15	check randomly clone scMutScore	56
16	tracing allClones	60
17	randomlySelectMut	63

18 lineageTracingExample	67
19 lineageTracingExample with scMutScore	69
20 ModelFigure	75
21 sessionInfo	79

1 Env

```
rm(list = ls())
setwd("/Users/liqun/Desktop/scMutrace-tutorial/ReproducibilityAnalysis/Figure5/")
```

2 Libraries

```
library(FSA)
## ## FSA v0.10.0. See citation('FSA') if used in publication.
## ## Run fishR() for related website and fishR('IFAR') for related book.
library(ineq)
library(vegan)
## Loading required package: permute
library(dplyr)
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
library(tidyr)
library(ggpubr)
## Loading required package: ggplot2
## Registered S3 methods overwritten by 'car':
##   method      from
##   hist.boot    FSA
##   confint.boot FSA
library(Seurat)
## Loading required package: SeuratObject
## Loading required package: sp
## 'SeuratObject' was built under R 4.5.0 but the current version is
## 4.5.1; it is recommended that you reinstall 'SeuratObject' as the ABI
## for R may have changed
##
## Attaching package: 'SeuratObject'
## The following objects are masked from 'package:base':
##
```

```

##      intersect, t
library(igraph)
##
## Attaching package: 'igraph'
## The following object is masked from 'package:Seurat':
##
##      components
## The following object is masked from 'package:tidyr':
##
##      crossing
## The following objects are masked from 'package:dplyr':
##
##      as_data_frame, groups, union
## The following object is masked from 'package:vegan':
##
##      diversity
## The following object is masked from 'package:permute':
##
##      permute
## The following objects are masked from 'package:stats':
##
##      decompose, spectrum
## The following object is masked from 'package:base':
##
##      union
library(ggraph)
##
## Attaching package: 'ggraph'
## The following object is masked from 'package:sp':
##
##      geometry
library(webshot)
library(ggplot2)
library(ggpmisc)
## Loading required package: ggpp
## Registered S3 methods overwritten by 'ggpp':
##      method                  from
##      heightDetails.titleGrob ggplot2
##      widthDetails.titleGrob  ggplot2
##
## Attaching package: 'ggpp'
## The following objects are masked from 'package:ggpubr':
##
##      as_npc, as_npcx, as_npcy
## The following object is masked from 'package:ggplot2':
##
##      annotate
library(stringr)
library(corrplot)
## corrplot 0.95 loaded
library(pheatmap)
library(reshape2)
##

```

```

## Attaching package: 'reshape2'
## The following object is masked from 'package:tidyr':
##
##      smiths
library(tidygraph)
##
## Attaching package: 'tidygraph'
## The following object is masked from 'package:igraph':
##
##      groups
## The following object is masked from 'package:stats':
##
##      filter
library(networkD3)
##
## Attaching package: 'networkD3'
## The following object is masked from 'package:Seurat':
##
##      JS
## The following object is masked from 'package:SeuratObject':
##
##      JS
library(tidyverse)
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats 1.0.0      v readr 2.1.5
## v lubridate 1.9.4    v tibble 3.3.0
## v purrr 1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x lubridate::%--%()      masks igraph::%--%()
## x ggpp::annotate()      masks ggplot2::annotate()
## x tibble::as_data_frame() masks igraph::as_data_frame(), dplyr::as_data_frame()
## x purrr::compose()      masks igraph::compose()
## x igraph::crossing()     masks tidyr::crossing()
## x tidygraph::filter()   masks dplyr::filter(), stats::filter()
## x dplyr::lag()           masks stats::lag()
## x purrr::simplify()      masks igraph::simplify()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
library(patchwork)

```

3 Def function

```

retriveCellsByLocation <- function(CSM, locList){
  CSM$CellTag <- rownames(CSM)
  CSM_tmp <- CSM[, c("CellTag",locList)]
  data_MutPerCell_CSM_tmp <- data.frame(
    CellTag = rownames(CSM_tmp),
    MutNum = rowSums(CSM_tmp == "Mut"),
    WTNun = rowSums(CSM_tmp == "WT")
  )
  cells_Total <- data_MutPerCell_CSM_tmp[rowSums(data_MutPerCell_CSM_tmp[,c("MutNum", "WTNum")]) > 0, ]
  cells_WT <- data_MutPerCell_CSM_tmp[data_MutPerCell_CSM_tmp[, "WTNum"] > 0,]$CellTag
}

```

```

cells_Mut <- data_MutPerCell_CSM_tmp[data_MutPerCell_CSM_tmp[, "MutNum"] > 0,]$CellTag
cells <- list(Total = cells_Total, Mut = cells_Mut, WT = cells_WT)
return(cells)
}

retriveLocsByCells <- function(CSM, cellList){
  CSM$CellTag <- rownames(CSM)
  CSM_tmp <- CSM[CSM$CellTag %in% cellList,]
  data_CellPerMut_CSM_tmp <- data.frame(
    Location = colnames(CSM_tmp),
    CellMutNum = colSums(CSM_tmp == "Mut"),
    CellWTNum = colSums(CSM_tmp == "WT")
  )
  pos_Total <- data_CellPerMut_CSM_tmp[rowSums(data_CellPerMut_CSM_tmp[, c("CellMutNum", "CellWTNum")]) > 0,]$Location
  pos_Mut <- data_CellPerMut_CSM_tmp[data_CellPerMut_CSM_tmp[, "CellMutNum"] > 0,]$Location
  pos_WT <- data_CellPerMut_CSM_tmp[data_CellPerMut_CSM_tmp[, "CellWTNum"] > 0,]$Location
  pos <- list(Total = pos_Total, Mut = pos_Mut, WT = pos_WT)
  return(pos)
}

cal_MultiSpe <- function(matrixData) {
  res <- data.frame(
    Shannon = vegan::diversity(matrixData, index = "shannon"),
    Simpson = vegan::diversity(matrixData, index = "simpson"),
    Gini = apply(matrixData, 1, function(x) ineq(x + 0.00001, type = "Gini")),
    sumMut = rowSums(matrixData),
    Tau = NA
  )
  # Tau = sum(1 - (x / max(x))) / (n - 1)
  res$Tau <- apply(matrixData, 1, function(x) {
    if (all(x == 0)) return(NA)
    x <- x / max(x)
    sum(1 - x) / (length(x) - 1)
  })
  rownames(res) <- rownames(matrixData)
  return(res)
}

cal_scMutScore <- function(data_cell_mut, data_mut_time){
  # data_cell_mut [cell * muts]
  # data_mut_time [muts * tissues/celltype/days]
  # data_cell_time_matrix
  replacement_map <- c("NoCall" = 0, "WT" = 0, "Mut" = 1)

  data_cell_mut_num <- as.data.frame(lapply(data_cell_mut, function(x) {
    ifelse(x %in% names(replacement_map), replacement_map[x], x)
  })))

  rownames(data_cell_mut_num) <- rownames(data_cell_mut)
  print(dim(data_cell_mut_num))
}

```

```

print(dim(data_mut_time))

data_cell_time_matrix <- as.matrix(data_cell_mut_num) %*% as.matrix(data_mut_time)
rownames(data_cell_time_matrix) <- rownames(data_cell_mut)
colnames(data_cell_time_matrix) <- colnames(data_mut_time)

mut_s_total <- unique(rownames(data_mut_time))

# get different tissues/time/celltype names
col_names_data <- colnames(data_cell_time_matrix)

col_length <- length(col_names_data)

col_coefficient_fix <- setNames(
  rep(1/col_length, col_length),
  col_names_data
)

data_cell_time_matrix <- data_cell_time_matrix[,col_names_data]

# Gini
res_index <- cal_MultiSpe(data_cell_time_matrix)

res_diversity <- as.data.frame((data_cell_time_matrix > 1) * rep(col_coefficient_fix[colnames(data_cell_time_matrix)],
res_diversity$Diversity <- rowSums(res_diversity)

res_diversity$Diversity[is.na(res_diversity$Diversity)] <- 0.00001

res_index$Gini[is.na(res_index$Gini)] <- 0.00001
res_index$Tau[is.na(res_index$Tau)] <- 0.00001

res_index$GiniTau <- res_index$Gini * res_index$Tau

res_index$scMutScore <- 2 / (
  (1 / (res_index$GiniTau + 0.00001)) +
  (1 / (res_diversity$Diversity + 0.00001))
)

res_index$Diversity <- res_diversity$Diversity

res_index$scMutScore <- scales::rescale(res_index$scMutScore, to = c(0, 1))

res <- list(diversity = res_diversity, score = res_index, matrixD = data_cell_time_matrix)
return(res)
}

propagate_mutations <- function(data, time_points) {
  for (i in 1:nrow(data)) {
    mut_time <- which(data[i, time_points] == "Mut")
    if (length(mut_time) > 0) {
      first_mut <- min(mut_time)
      data[i, time_points[first_mut:length(time_points)]] <- "Mut"
    }
  }
}

```

```

}
  return(data)
}

```

4 Load Data

```

MEF_CSM <- readRDS("../Data/MEF/MEF_CSM.rds")
MEF_CSM_hf1 <- MEF_CSM$hf1
MEF_CSM_hf2 <- MEF_CSM$hf2

MEF_SSM <- readRDS("../Data/MEF/MEF_SSM.rds")
MEF_SSM_hf1 <- MEF_SSM$hf1
MEF_SSM_hf2 <- MEF_SSM$hf2

MEF_Meta <- readRDS("../Data/MEF/MEF_Meta.rds")
MEF_Meta_hf1 <- MEF_Meta$hf1
MEF_Meta_hf2 <- MEF_Meta$hf2

# retrieved from MPG database, see https://github.com/QunATCG/scMutrace-tutorial/tree/main/AdvancedFunc
MPG_data <- read.csv("../Data/MEF/MPG_combined.csv")

```

5 Observed Accumulated Mutation Num

```

Reps_sequencing <- c("Rep1_all", "Rep2_all")
Days_sequencing <- c("d6", "d9", "d12", "d15", "d21", "d28")

col_hf1_names <- c("hf1.d0", "hf1.d3", "hf1.d6", "hf1.d9", "hf1.d12", "hf1.d15", "hf1.d21", "hf1.d28")
col_hf2_names <- c("hf2.d0", "hf2.d3", "hf2.d6", "hf2.d9", "hf2.d12", "hf2.d15", "hf2.d21", "hf2.d28")

data_accMutNum_res <- as.data.frame(matrix(nrow = length(Days_sequencing), ncol = length(Reps_sequencing),
colnames(data_accMutNum_res) <- Reps_sequencing
rownames(data_accMutNum_res) <- Days_sequencing

for (i in rownames(data_accMutNum_res)){
  data_accMutNum_res[i, "Rep1_all"] <- length(MEF_SSM_hf1[MEF_SSM_hf1[, paste0("hf1.", i)] == "Mut",]$mutNum)
  data_accMutNum_res[i, "Rep2_all"] <- length(MEF_SSM_hf2[MEF_SSM_hf2[, paste0("hf2.", i)] == "Mut",]$mutNum)
}

data_accMutNum_res$Days <- rownames(data_accMutNum_res)
data_accMutNum_res_plot <- melt(data_accMutNum_res)
## Using Days as id variables
colnames(data_accMutNum_res_plot) <- c("Days", "Group", "MutNum")

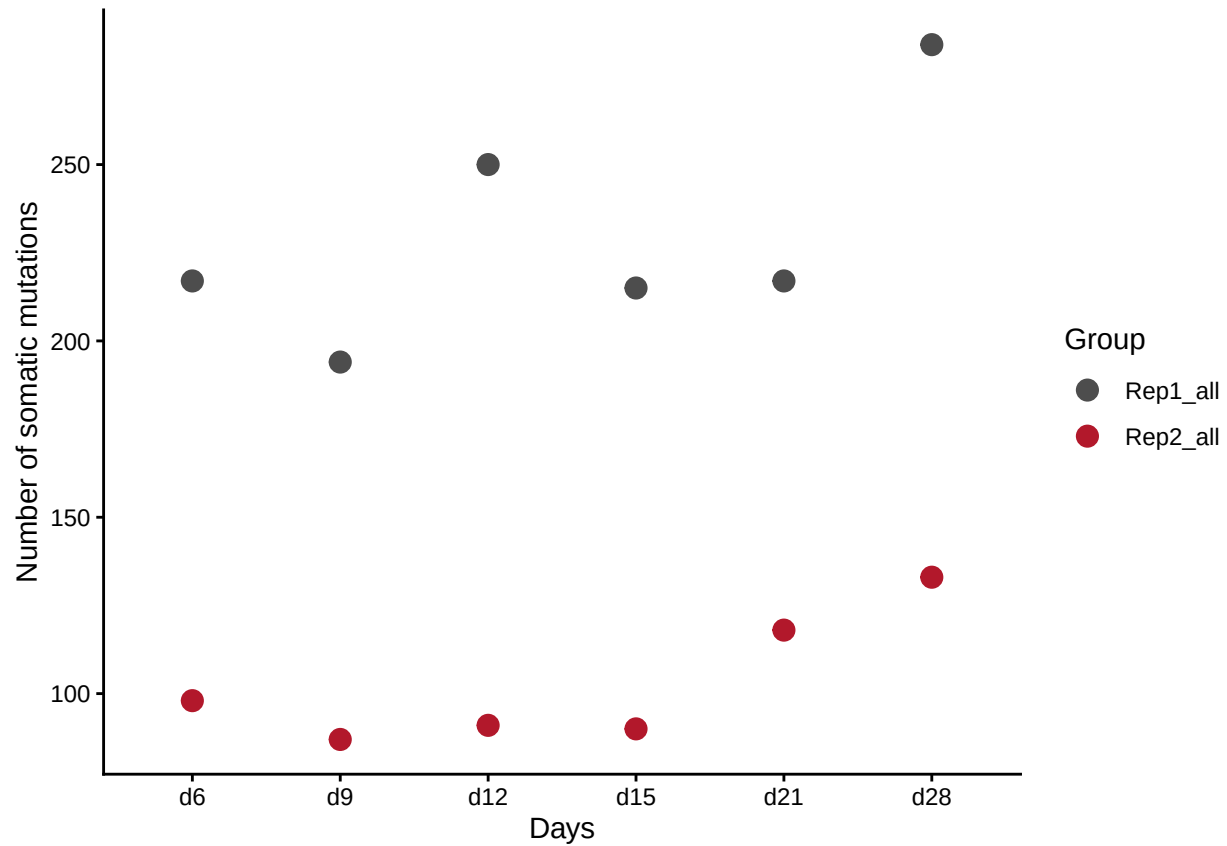
data_accMutNum_res_plot$Days <- factor(data_accMutNum_res_plot$Days, levels = c("d6", "d9", "d12", "d15", "d21", "d28"))

p_reps_somaticNum <- ggplot(data_accMutNum_res_plot, aes(x = Days, y = MutNum, colour = Group, fill = Group)) +
  geom_point(shape = 21, size = 3, stroke = 0.8) +
  ylab("Number of somatic mutations") +

```

```
scale_colour_manual(values = c("#4d4d4d", "#b2182b")) +
scale_fill_manual(values = c("#4d4d4d", "#b2182b")) +
theme_classic()
```

p_reps_somaticNum



```
replacement_map <- c("NoCall" = 0, "WT" = 0, "Mut" = 1)
```

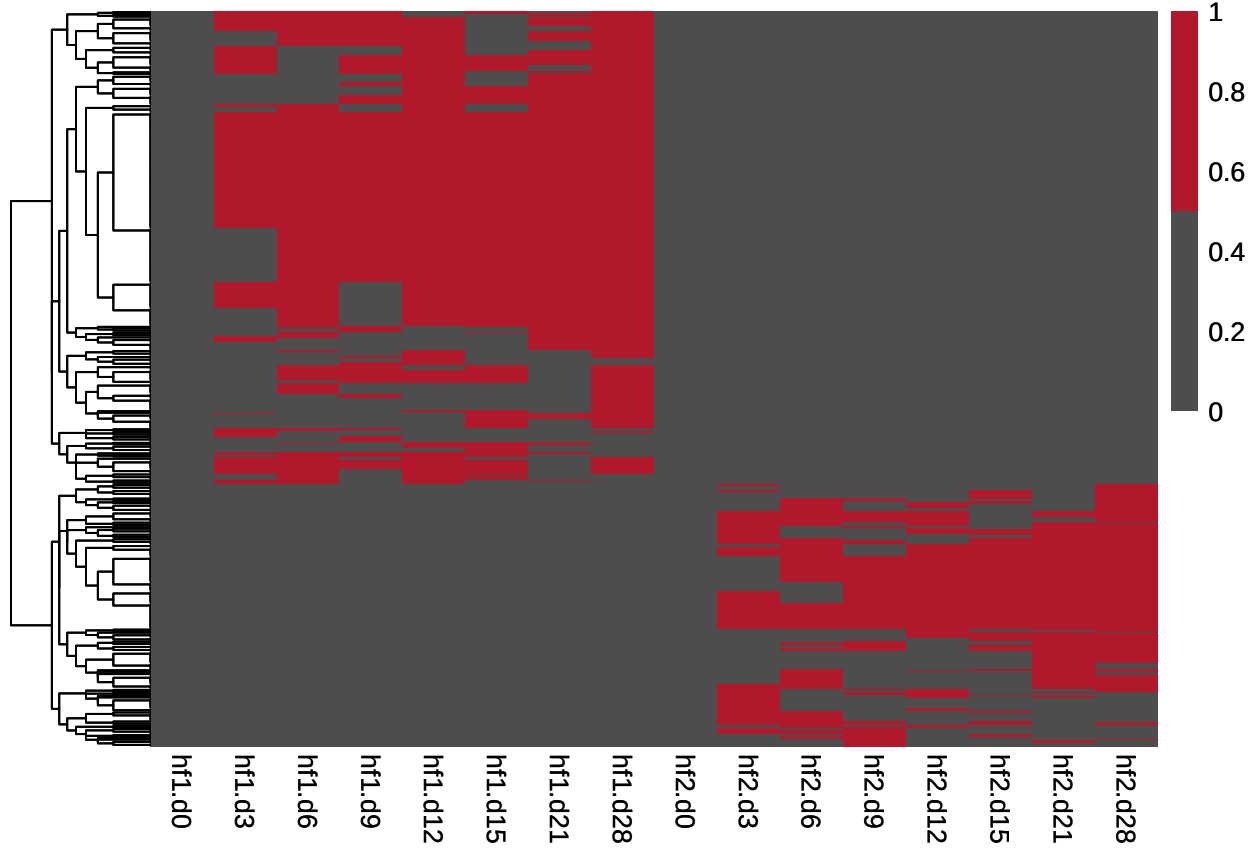
```
MEF_SSM_hf1_hf2 <- merge(MEF_SSM_hf1[,c("mutPos", col_hf1_names)], MEF_SSM_hf2[,c("mutPos", col_hf2_names)],
  by = "mutPos", all = TRUE)
MEF_SSM_hf1_hf2[is.na(MEF_SSM_hf1_hf2)] <- "NoCall"
```

```
MEF_SSM_hf1_hf2_num <- as.data.frame(lapply(MEF_SSM_hf1_hf2, function(x) {
  ifelse(x %in% names(replacement_map), replacement_map[x], x)
})))
```

```
rownames(MEF_SSM_hf1_hf2_num) <- MEF_SSM_hf1_hf2$mutPos
```

```
# propagate mutations
```

```
p_accSomaticMut_heatmap_observed <- pheatmap(MEF_SSM_hf1_hf2_num[,c(2:17)], show_rownames = F, color = c("black", "red"))
p_accSomaticMut_heatmap_observed
```

6 Imputed Accumulated Mutation Num

```

color_settings <- c(d0 = "#FFDC91FF", d3 = "#7876B1FF", d6 = "#4575b4",
                    d9 = "#91b9db", d12 = "#e0f3f8", d15 = "#fee090",
                    d21 = "#fc8d59", d28 = "#BC3C29FF")

color_settings_2 <- c(Day_0 = "#FFDC91FF", Day_3 = "#7876B1FF", Day_6 = "#4575b4",
                     Day_9 = "#91b9db", Day_12 = "#e0f3f8", Day_15 = "#fee090",
                     Day_21 = "#fc8d59", Day_28 = "#BC3C29FF")

Reps_sequencing <- c("Rep1_all", "Rep2_all")
Days_sequencing <- c("d0", "d3", "d6", "d9", "d12", "d15", "d21", "d28")

col_hf1_names <- c("hf1.d0", "hf1.d3", "hf1.d6", "hf1.d9", "hf1.d12", "hf1.d15", "hf1.d21", "hf1.d28")
col_hf2_names <- c("hf2.d0", "hf2.d3", "hf2.d6", "hf2.d9", "hf2.d12", "hf2.d15", "hf2.d21", "hf2.d28")

MEF_SSM_hf1_impute <- propagate_mutations(MEF_SSM_hf1, col_hf1_names)
MEF_SSM_hf2_impute <- propagate_mutations(MEF_SSM_hf2, col_hf2_names)

data_accMutNum_res <- as.data.frame(matrix(nrow = length(Days_sequencing), ncol = length(Reps_sequencing),
colnames(data_accMutNum_res) <- Reps_sequencing
rownames(data_accMutNum_res) <- Days_sequencing

```

```

for (i in rownames(data_accMutNum_res)){
  data_accMutNum_res[i, "Rep1_all"] <- length(MEF_SSM_hf1_impute[MEF_SSM_hf1_impute[, paste0("hf1.", i)],])
  data_accMutNum_res[i, "Rep2_all"] <- length(MEF_SSM_hf2_impute[MEF_SSM_hf2_impute[, paste0("hf2.", i)],])
}

data_accMutNum_res$Days <- rownames(data_accMutNum_res)
data_accMutNum_res_plot <- melt(data_accMutNum_res)
## Using Days as id variables
colnames(data_accMutNum_res_plot) <- c("Days", "Group", "MutNum")

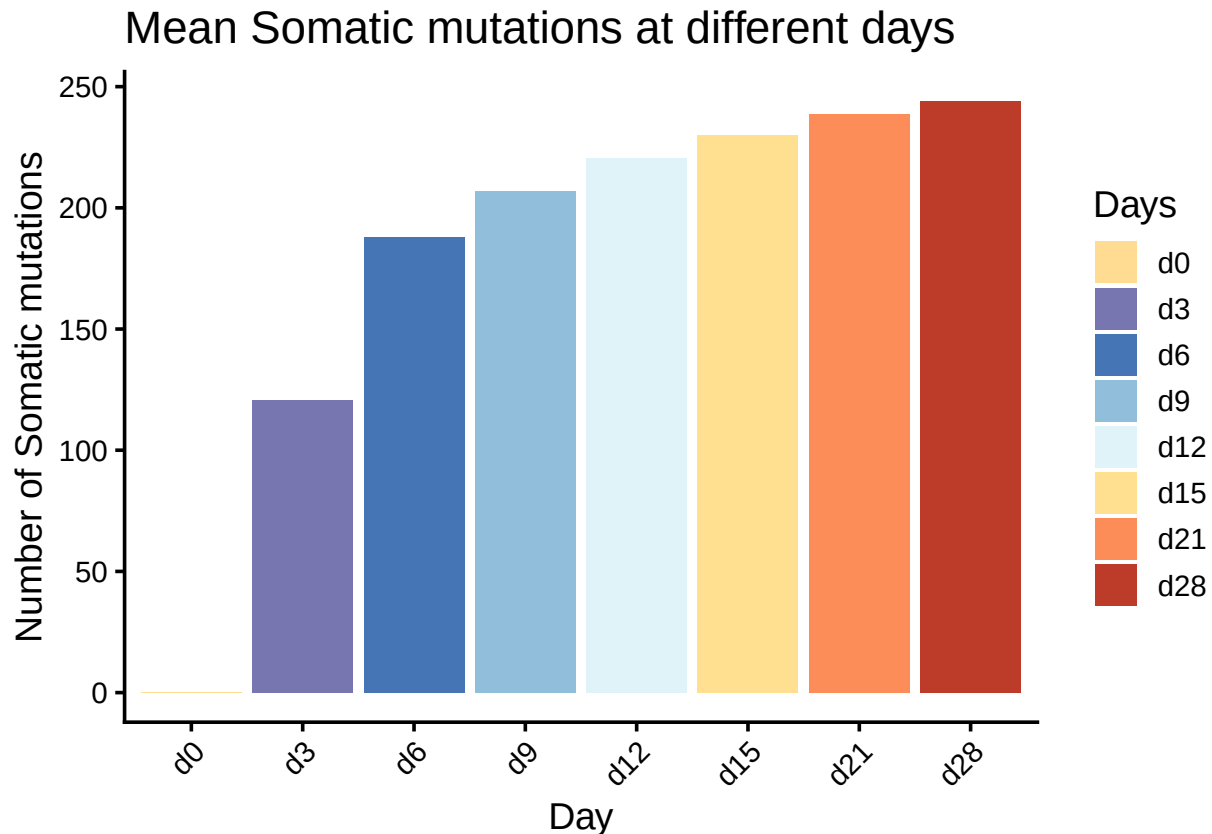
data_accMutNum_res_mean_plot <- aggregate(MutNum ~ Days, data = data_accMutNum_res_plot, FUN = mean)

data_accMutNum_res_mean_plot$Days <- factor(data_accMutNum_res_mean_plot$Days, levels = c("d0", "d3", "d6", "d9", "d12", "d15", "d21", "d28"))

p_reps_somaticNum <- ggplot(data_accMutNum_res_mean_plot, aes(x = Days, y = MutNum, fill = Days)) +
  geom_col(position = "dodge") +
  theme_minimal(base_size = 14) +
  labs(title = "Mean Somatic mutations at different days",
       x = "Day",
       y = "Number of Somatic mutations") +
  scale_fill_manual(values = color_settings) +
  theme_classic(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_reps_somaticNum

```



```

replacement_map <- c("NoCall" = 0, "WT" = 0, "Mut" = 1)

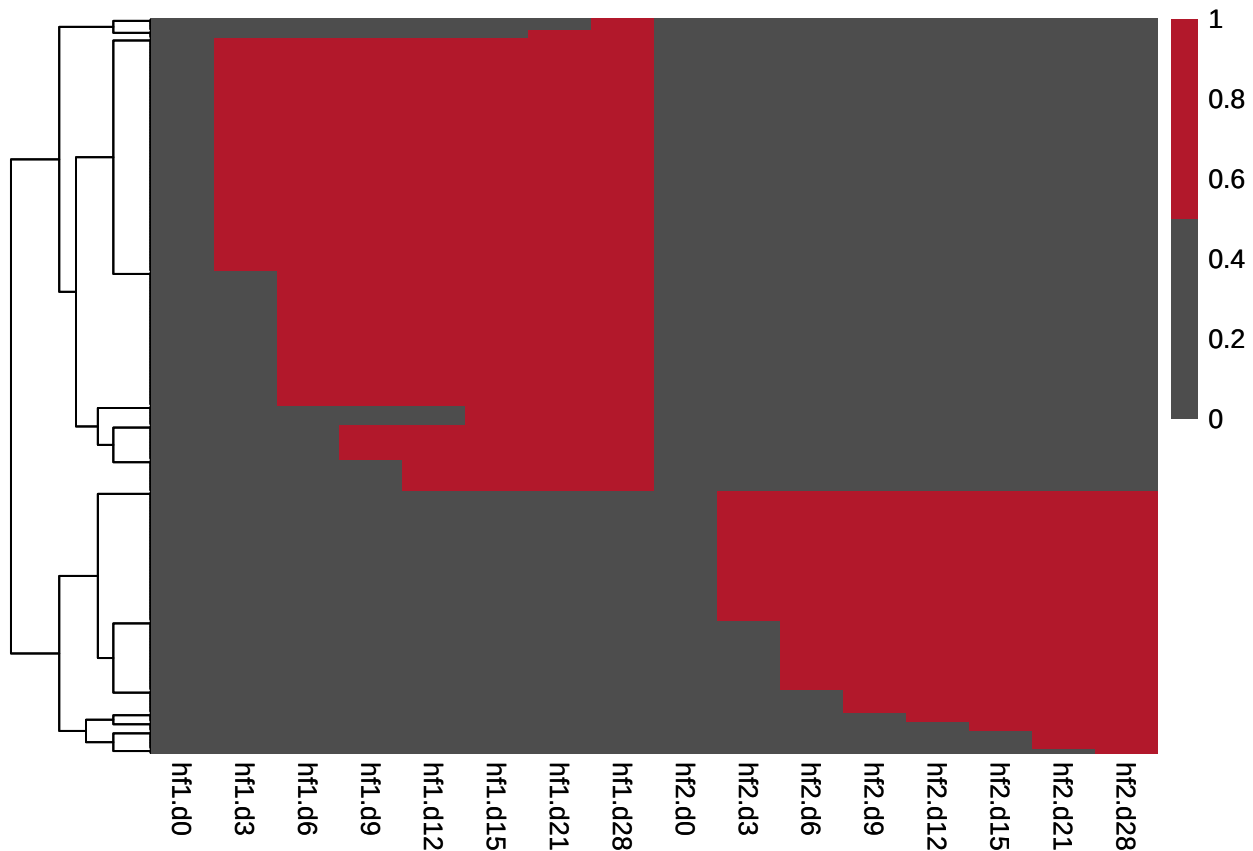
MEF_SSM_hf1_hf2_impute <- merge(MEF_SSM_hf1_impute[,c("mutPos", col_hf1_names)], MEF_SSM_hf2_impute[,c(
MEF_SSM_hf1_hf2_impute[is.na(MEF_SSM_hf1_hf2_impute)] <- "NoCall"

MEF_SSM_hf1_hf2_impute_num <- as.data.frame(lapply(MEF_SSM_hf1_hf2_impute, function(x) {
  ifelse(x %in% names(replacement_map), replacement_map[x], x)
})))

rownames(MEF_SSM_hf1_hf2_impute_num) <- MEF_SSM_hf1_hf2_impute$mutPos

# propagate_mutations
p_accSomaticMut_heatmap_impute <- pheatmap(MEF_SSM_hf1_hf2_impute_num[,c(2:17)], show_rownames = F, col
p_accSomaticMut_heatmap_impute

```



```

# cell number
hf1_cellNum <- as.data.frame(table(MEF_Meta_hf1$Day)) %>%
  mutate(Sample = "hf1")
hf2_cellNum <- as.data.frame(table(MEF_Meta_hf2$Day)) %>%
  mutate(Sample = "hf2")

hf_cellNum <- bind_rows(hf1_cellNum, hf2_cellNum)
colnames(hf_cellNum) <- c("Day", "Count", "Sample")

```

```

hf_cellNum_dayMean <- aggregate(Count ~ Day, data = hf_cellNum, FUN = mean)

days_order_row <- c("Day_0", "Day_3", "Day_6", "Day_9", "Day_12",
                    "Day_15", "Day_21", "Day_28")

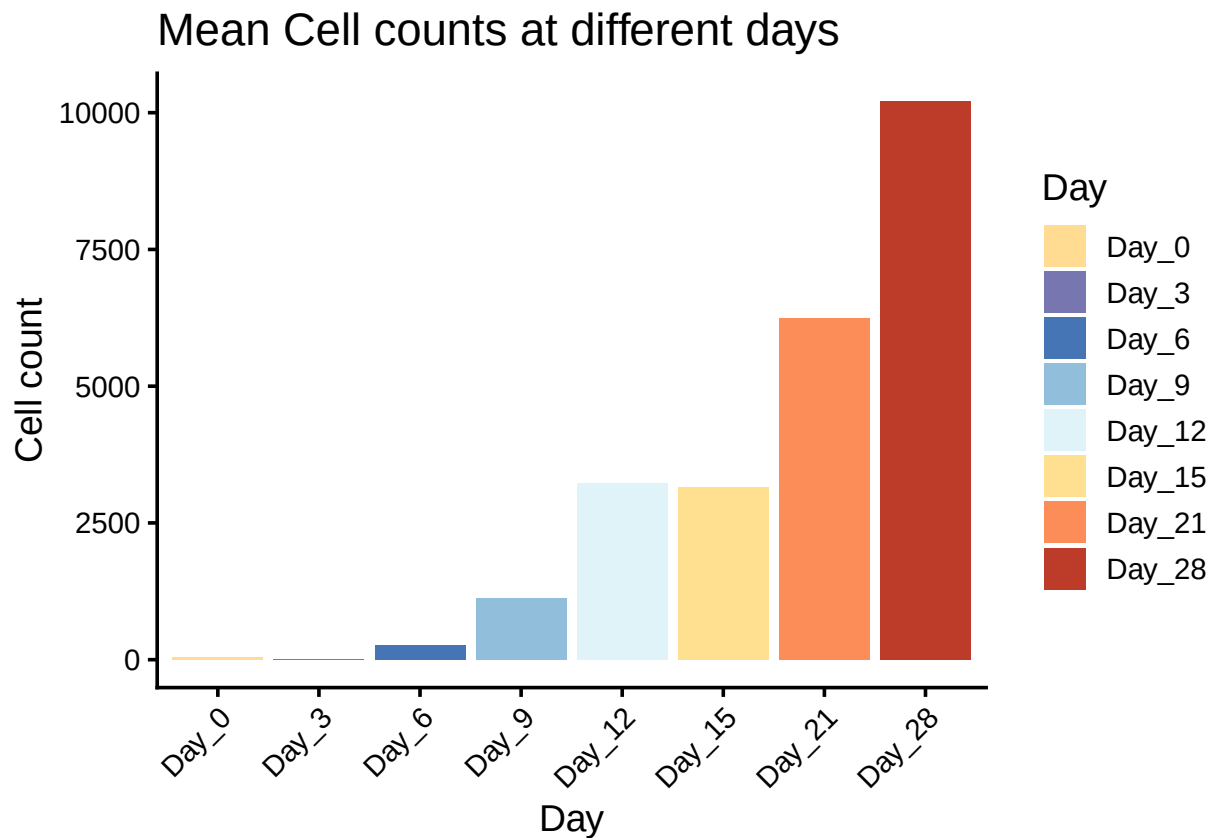
hf_cellNum_dayMean <- hf_cellNum_dayMean %>% filter(Day %in% days_order_row)
hf_cellNum_dayMean$Day <- factor(hf_cellNum_dayMean$Day, levels = days_order_row)

color_settings_2 <- c(Day_0 = "#FFDC91FF", Day_3 = "#7876B1FF", Day_6 = "#4575b4",
                    Day_9 = "#91bfbdb", Day_12 = "#e0f3f8", Day_15 = "#fee090",
                    Day_21 = "#fc8d59", Day_28 = "#BC3C29FF")

p_hf_cellNum <- ggplot(hf_cellNum_dayMean, aes(x = Day, y = Count, fill = Day)) +
  geom_col(position = "dodge") +
  theme_minimal(base_size = 14) +
  labs(title = "Mean Cell counts at different days",
       x = "Day",
       y = "Cell count") +
  scale_fill_manual(values = color_settings_2) +
  theme_classic(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_hf_cellNum

```



7 Accumulated Mutation Num normalized by cell number

```
color_settings_randomCells <- c(hf1.d3 = "#7876B1FF", hf1.d6 = "#4575b4", hf1.d9 = "#91bfdb", hf1.d12 = "#e0f3f8",
                                hf2.d3 = "#7876B1FF", hf2.d6 = "#4575b4", hf2.d9 = "#91bfdb", hf2.d12 = "#e0f3f8",
                                hf2.d15 = "#fee090", hf2.d21 = "#f0f0f0", hf2.d28 = "#f0f0f0")

color_settings_randomCells_day <- c(d6 = "#4575b4", d9 = "#91bfdb", d12 = "#e0f3f8", d15 = "#fee090", d21 = "#f0f0f0", d28 = "#f0f0f0")

totalCells <- c(rownames(MEF_CSM_hf1), rownames(MEF_CSM_hf2))

selectRandomNum <- c(10, 50, 100, 200, 300, 400, 500, 1000, 2000, 3000, 4000, 5000, 6000, 8000)

dataSelect <- c("hf1.d6", "hf1.d9", "hf1.d12", "hf1.d15", "hf1.d21", "hf1.d28",
                "hf2.d6", "hf2.d9", "hf2.d12", "hf2.d15", "hf2.d21", "hf2.d28")

days <- c("d6", "d9", "d12", "d15", "d21", "d28")
reps <- 20
data_random_counts <- as.data.frame(matrix(nrow = (length(selectRandomNum) * length(dataSelect) * reps),
                                           colnames(data_random_counts) <- c("ID", "reps", "selectNum", "MutNum")))

set.seed(2025)
t = 0
for (i in dataSelect){
  dataCells <- totalCells[grepl(i, totalCells)]
  selectNum <- selectRandomNum
  for (j in 1:reps){
    for (k in selectNum){
      dataCells_random <- sample(dataCells, k, replace = TRUE)
      hf1_selected <- dataCells_random[dataCells_random %in% rownames(MEF_CSM_hf1)]
      hf2_selected <- dataCells_random[dataCells_random %in% rownames(MEF_CSM_hf2)]

      data_random_hf1 <- MEF_CSM_hf1[hf1_selected,]
      data_random_hf2 <- MEF_CSM_hf2[hf2_selected,]

      num_hf1 <- sum(colSums(data_random_hf1 == "Mut") > 0)
      num_hf2 <- sum(colSums(data_random_hf2 == "Mut") > 0)
      t = t + 1
      if(t %% 400 == 0){
        print(paste0("processed ", t, " iterations"))
      }
      data_random_counts[t,]$ID <- i
      data_random_counts[t,]$reps <- j
      data_random_counts[t,]$selectNum <- k
      data_random_counts[t,]$MutNum <- num_hf1 + num_hf2
    }
  }
}

## [1] "processed 400 iterations"
## [1] "processed 800 iterations"
## [1] "processed 1200 iterations"
## [1] "processed 1600 iterations"
## [1] "processed 2000 iterations"
## [1] "processed 2400 iterations"
## [1] "processed 2800 iterations"
```

```

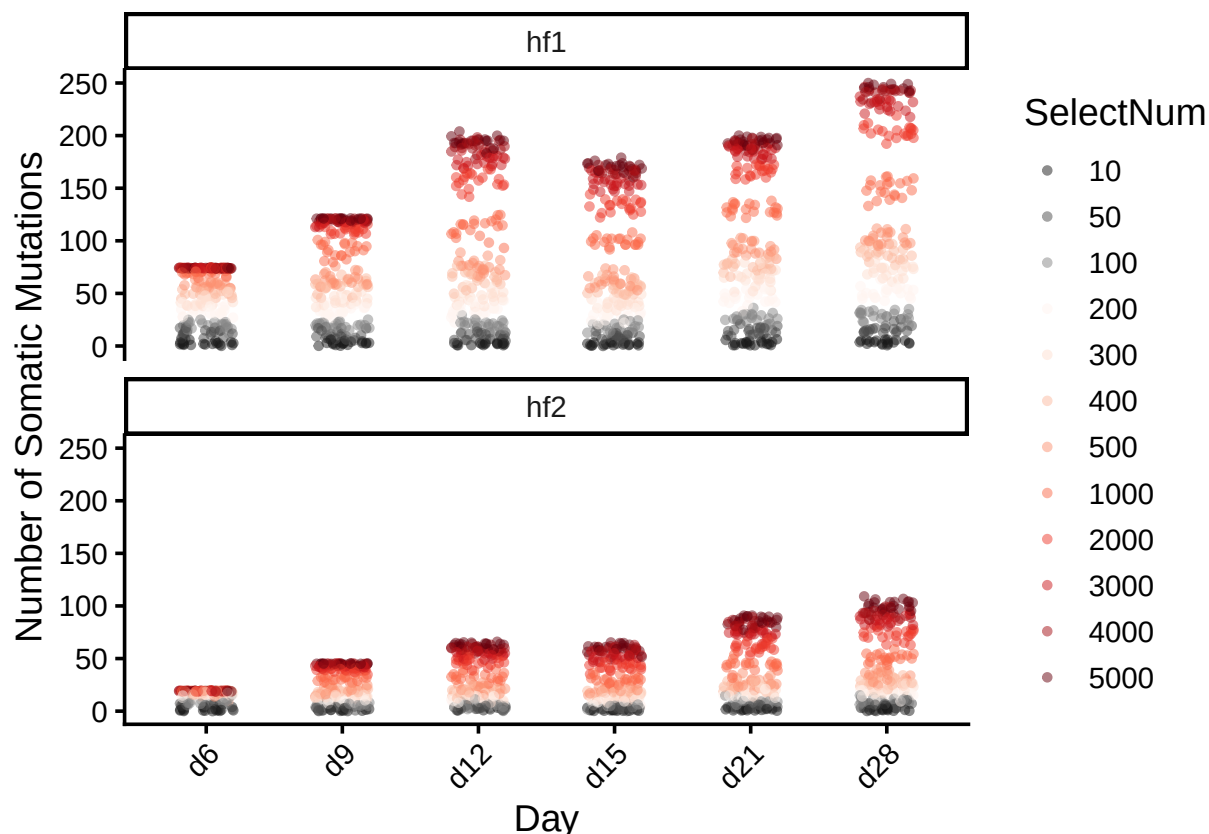
## [1] "processed 3200 iterations"

data_random_counts <- separate(data_random_counts,col = "ID",into = c("Timecourse", "Day"), sep = "\\.").
data_random_counts$ID <- paste(data_random_counts$Timecourse, data_random_counts$Day, sep = ".")
data_random_counts$Day <- factor(data_random_counts$Day, levels = days)
data_random_counts$ID <- factor(data_random_counts$ID, levels = dataSelect)
data_random_counts_plot <- data_random_counts[data_random_counts$selectNum %in% c(10, 50, 100, 200, 300),]

p_random_somaticNum <- ggplot(data_random_counts_plot,
  aes(x = Day, y = MutNum, color = factor(selectNum))) +
  geom_jitter(width = 0.2, alpha = 0.5, size = 1) +
  facet_wrap(~ Timecourse, ncol = 1, scales = "free_y") +
  ylab("Number of Somatic Mutations") +
  xlab("Day") +
  ylim (0, 250) +
  labs(color = "SelectNum") +
  scale_color_manual(values = c(
    "10" = "#1a1a1a",
    "50" = "#4d4d4d",
    "100" = "#878787",
    "200" = "#fff5f0",
    "300" = "#fee0d2",
    "400" = "#fcbba1",
    "500" = "#fc9272",
    "1000" = "#fb6a4a",
    "2000" = "#ef3b2c",
    "3000" = "#cb181d",
    "4000" = "#a50f15",
    "5000" = "#67000d"
  )) +
  theme_classic(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_random_somaticNum
## Warning: Removed 54 rows containing missing values or values outside the scale range
## (`geom_point()`).

```



8 MPGgenotypecheck

```
# check position
MPG_data <- read.csv("../Data/MEF/MPG_combined.csv")
MEF_detected_pos <- unique(union(MEF_SSM_hf1$position_cor, MEF_SSM_hf2$position_cor))
MEF_detected_pos[MEF_detected_pos %in% MPG_data$position_cor]
## [1] "10_28334861" "10_28384990" "11_103070309" "11_59698972" "11_80397502"
## [6] "12_59094027" "12_81947565" "13_44240902" "13_59527224" "15_102984869"
## [11] "15_31472794" "15_96305641" "15_96697854" "16_20124088" "16_37897824"
## [16] "16_38799521" "17_28695383" "1_162857222" "1_178800830" "1_61723999"
## [21] "2_169768737" "2_75700504" "3_19993055" "3_22093358" "3_55288508"
## [26] "4_103363073" "4_6980700" "5_36660914" "5_66596487" "6_128290883"
## [31] "6_90475221" "7_137044144" "7_141638786" "7_16356122" "8_45733398"
## [36] "8_45807946" "8_71690586" "8_94675603" "9_124330134" "9_21429385"
## [41] "9_60385333" "10_32940591" "10_78338121" "13_119610824" "13_49547531"
## [46] "14_103312866" "17_62865029" "18_60642899" "1_137967589" "1_57915729"
## [51] "2_121793360" "5_98880003" "6_120931717" "7_13047148" "7_133890940"
## [56] "8_45786901" "8_45802144" "9_100773522" "9_102813915"
```

```
MPG_data <- MPG_data[,c(3,8:44)]
MPG_data[MPG_data == ""] <- NA
```

```
MEF_SSM_refAlt <- rbind(MEF_SSM_hf1[,c("position_cor", "ref", "alt")], MEF_SSM_hf2[,c("position_cor", "ref", "alt")])
```

```

MPG_data_refAlt <- merge(MEF_SSM_refAlt, MPG_data, by = "position_cor", all.x = T)
MPG_data_refAlt[is.na(MPG_data_refAlt)] <- 0

MPG_data_refAlt[, 4:ncol(MPG_data_refAlt)] <- apply(MPG_data_refAlt[, 4:ncol(MPG_data_refAlt)], 2, fun
  refCheck <- MPG_data_refAlt[, "ref"]
  ifelse(col != 0 & col != refCheck, 1, 0)
})

MPG_data_refAlt <- MPG_data_refAlt[!duplicated(MPG_data_refAlt$position_cor),]
rownames(MPG_data_refAlt) <- MPG_data_refAlt$position_cor

MPG_data_refAlt$MutNum <- rowSums(MPG_data_refAlt[, 4:40])
MPG_data_refAlt_sorted <- MPG_data_refAlt[order(-MPG_data_refAlt$MutNum), ]

colnames_order <- c("C57BL_6J", "C57BL_10J", "C57BL_6NJ", "C57BR_cdJ", "C57L_J",
  "AKR_J", "A_J", "BALB_cJ", "BTBRT_Itpr3_J", "BUB_BnJ",
  "C3H_HeH", "C3H_HeJ", "C58_J", "CAST_EiJ", "CBA_J",
  "DBA_1J", "DBA_2J", "FVB_NJ", "I_LnJ", "KK_H1J",
  "LEWES_EiJ", "LP_J", "MOLF_EiJ",
  "NOD_Sh1LtJ", "NZB_B1NJ", "NZO_H1LtJ", "NZW_LacJ",
  "PWK_PhJ", "RF_J", "SEA_GnJ", "SPRET_EiJ", "ST_bJ", "WSB_EiJ",
  "ZALENDE_EiJ", "X129P2_01aHsd", "X129S1_SvImJ", "X129S5SvEv_Brd")

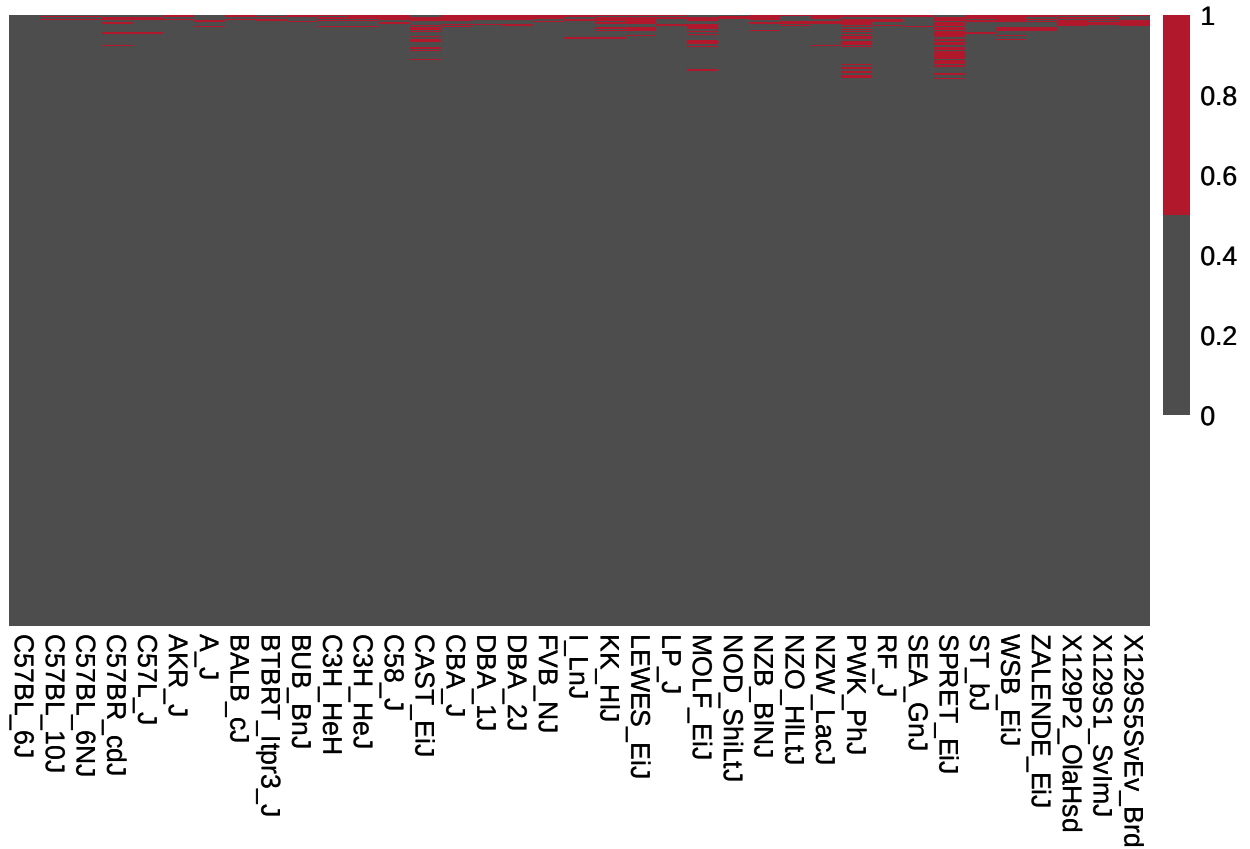
colnames_order[!colnames_order %in% colnames(MPG_data_refAlt_sorted)]
## character(0)

heatmap_data <- MPG_data_refAlt_sorted[, colnames_order]

custom_colors <- c("#4d4d4d", "#b2182b")

p_heatmapMPG <- pheatmap(heatmap_data, show_rownames = FALSE, cluster_rows = FALSE, cluster_cols = FALSE)
p_heatmapMPG

```

9 Calculationg mutations of each cell and cells of each mutation

```
data_MutPerCell_hf1 <- data.frame(
  CellTag = rownames(MEF_CSM_hf1),
  MutNum_somatic = rowSums(MEF_CSM_hf1 == "Mut"),
  WTNum_somatic = rowSums(MEF_CSM_hf1 == "WT")
)

data_CellPerMut_hf1 <- data.frame(
  Location = colnames(MEF_CSM_hf1),
  CellMutNum = colSums(MEF_CSM_hf1 == "Mut"),
  CellWTNum = colSums(MEF_CSM_hf1 == "WT")
)

data_MutPerCell_hf2 <- data.frame(
  CellTag = rownames(MEF_CSM_hf2),
  MutNum_somatic = rowSums(MEF_CSM_hf2 == "Mut"),
  WTNum_somatic = rowSums(MEF_CSM_hf2 == "WT")
)

data_CellPerMut_hf2 <- data.frame(
  Location = colnames(MEF_CSM_hf2),
  CellMutNum = colSums(MEF_CSM_hf2 == "Mut"),
```

```
CellWTNum = colSums(MEF_CSM_hf2 == "WT")
)
```

10 calMulti

```
# [Cell * Time] = [Cell * Mut] * [Mut * Time]
time_types <- c("Day_6", "Day_9", "Day_12", "Day_15", "Day_21", "Day_28")
cell_num = 1
mut_num = 1

# _hf1
cells_candidate_hf1 <- data_MutPerCell_hf1[data_MutPerCell_hf1$MutNum_somatic >= cell_num,]$CellTag
cells_candidate_hf1 <- cells_candidate_hf1[cells_candidate_hf1 %in% MEF_Meta_hf1$Cellbarcode]
mut_candidate_hf1 <- data_CellPerMut_hf1[data_CellPerMut_hf1$CellMutNum >= mut_num,]$Location

# data_Cell_Mut
data_cell_mut_raw_hf1 <- MEF_CSM_hf1[cells_candidate_hf1, mut_candidate_hf1]

# data_Mut_Time
data_mut_time_hf1 <- as.data.frame(matrix(nrow = length(mut_candidate_hf1), ncol = length(time_types)),
rownames(data_mut_time_hf1) <- mut_candidate_hf1
colnames(data_mut_time_hf1) <- time_types

for (i in 1:nrow(data_mut_time_hf1)){
  mut_tmp <- rownames(data_mut_time_hf1)[i]
  cell_tmp <- retrieveCellsByLocation(data_cell_mut_raw_hf1, mut_tmp)
  selected_meta_tags <- MEF_Meta_hf1[MEF_Meta_hf1$Cellbarcode %in% cell_tmp$Mut,]$Day
  data_mut_time_hf1[i,]$Day_6 <- sum(str_count(selected_meta_tags, "Day_6"))
  data_mut_time_hf1[i,]$Day_9 <- sum(str_count(selected_meta_tags, "Day_9"))
  data_mut_time_hf1[i,]$Day_12 <- sum(str_count(selected_meta_tags, "Day_12"))
  data_mut_time_hf1[i,]$Day_15 <- sum(str_count(selected_meta_tags, "Day_15"))
  data_mut_time_hf1[i,]$Day_21 <- sum(str_count(selected_meta_tags, "Day_21"))
  data_mut_time_hf1[i,]$Day_28 <- sum(str_count(selected_meta_tags, "Day_28"))
}

# _hf2
cells_candidate_hf2 <- data_MutPerCell_hf2[data_MutPerCell_hf2$MutNum_somatic >= cell_num,]$CellTag
cells_candidate_hf2 <- cells_candidate_hf2[cells_candidate_hf2 %in% MEF_Meta_hf2$Cellbarcode]
mut_candidate_hf2 <- data_CellPerMut_hf2[data_CellPerMut_hf2$CellMutNum >= mut_num,]$Location

# data_Cell_Mut
data_cell_mut_raw_hf2 <- MEF_CSM_hf2[cells_candidate_hf2, mut_candidate_hf2]

# data_Mut_Time
data_mut_time_hf2 <- as.data.frame(matrix(nrow = length(mut_candidate_hf2), ncol = length(time_types)),
rownames(data_mut_time_hf2) <- mut_candidate_hf2
colnames(data_mut_time_hf2) <- time_types

for (i in 1:nrow(data_mut_time_hf2)){
  mut_tmp <- rownames(data_mut_time_hf2)[i]
  cell_tmp <- retrieveCellsByLocation(data_cell_mut_raw_hf2, mut_tmp)
```

```

selected_meta_tags <- MEF_Meta_hf2[MEF_Meta_hf2$Cellbarcode %in% cell_tmp$Mut,]$Day
data_mut_time_hf2[i,]$Day_6 <- sum(str_count(selected_meta_tags, "Day_6"))
data_mut_time_hf2[i,]$Day_9 <- sum(str_count(selected_meta_tags, "Day_9"))
data_mut_time_hf2[i,]$Day_12 <- sum(str_count(selected_meta_tags, "Day_12"))
data_mut_time_hf2[i,]$Day_15 <- sum(str_count(selected_meta_tags, "Day_15"))
data_mut_time_hf2[i,]$Day_21 <- sum(str_count(selected_meta_tags, "Day_21"))
data_mut_time_hf2[i,]$Day_28 <- sum(str_count(selected_meta_tags, "Day_28"))
}

```

```

res_index_hf1 <- cal_scMutScore(data_cell_mut_raw_hf1, data_mut_time_hf1)
## [1] 4371 297
## [1] 297 6
res_index_hf2 <- cal_scMutScore(data_cell_mut_raw_hf2, data_mut_time_hf2)
## [1] 2627 156
## [1] 156 6
res_index <- rbind(res_index_hf1$score, res_index_hf2$score)

res_index$cb <- rownames(res_index)
res_index <- separate(res_index, col = "cb", into = c("celltype", "cell"), sep = "_")
res_index$Tag <- res_index$celltype
res_index <- separate(res_index, col = "celltype", into = c("rep", "Day"), sep = "\\.")

res_index$Tag <- factor(res_index$Tag, levels = c("hf1.d0", "hf1.d3", "hf1.d6", "hf1.d9", "hf1.d12", "hf1.d15", "hf1.d21", "hf1.d28",
"hf2.d0", "hf2.d3", "hf2.d6", "hf2.d9", "hf2.d12", "hf2.d15", "hf2.d21", "hf2.d28"))

res_index$Day <- factor(res_index$Day, levels = c("d0", "d3", "d6", "d9", "d12", "d15", "d21", "d28"))

res_index <- res_index[res_index$Diversity > 0, ]

```

```

color_settings <- c(d0 = "#FFDC91FF", d3 = "#7876B1FF", d6 = "#4575b4",
d9 = "#91bfdb", d12 = "#e0f3f8", d15 = "#fee090",
d21 = "#fc8d59", d28 = "#BC3C29FF")

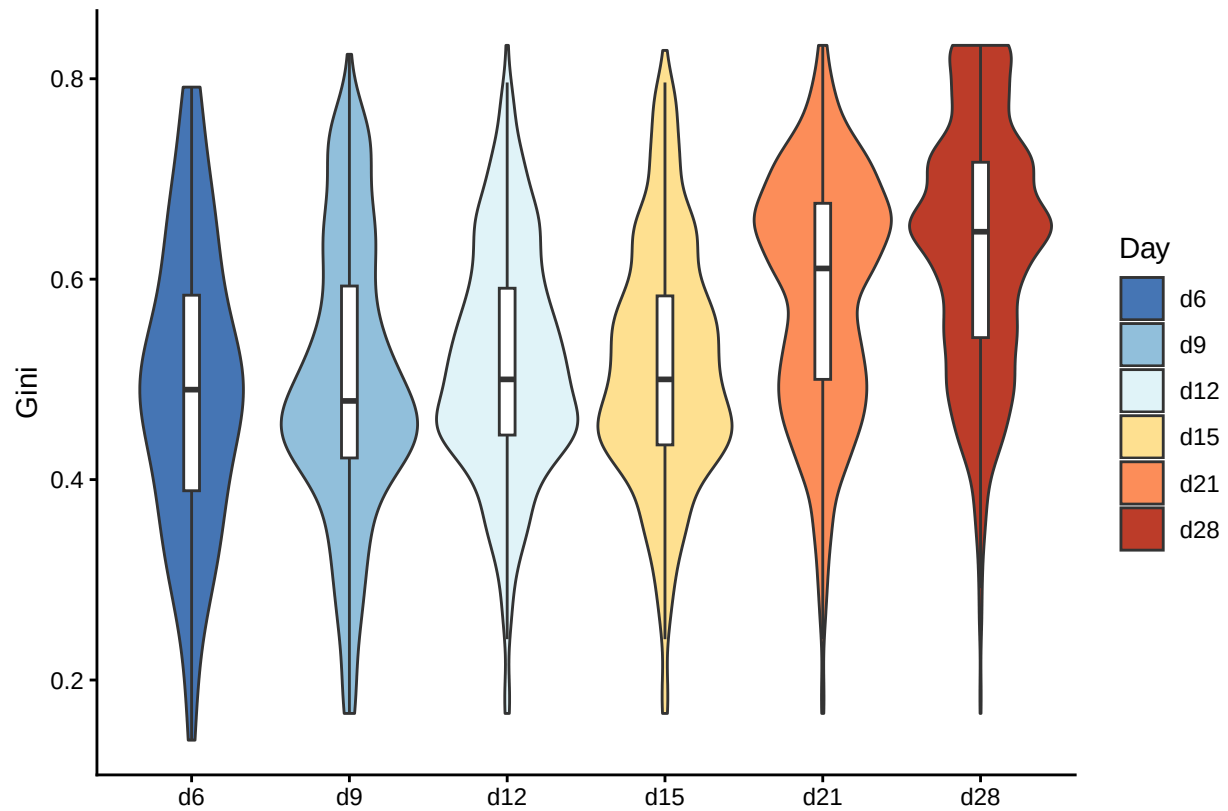
color_settings_2 <- c(Day_0 = "#FFDC91FF", Day_3 = "#7876B1FF", Day_6 = "#4575b4",
Day_9 = "#91bfdb", Day_12 = "#e0f3f8", Day_15 = "#fee090",
Day_21 = "#fc8d59", Day_28 = "#BC3C29FF")

res_index <- res_index[!is.na(res_index$scMutScore),]

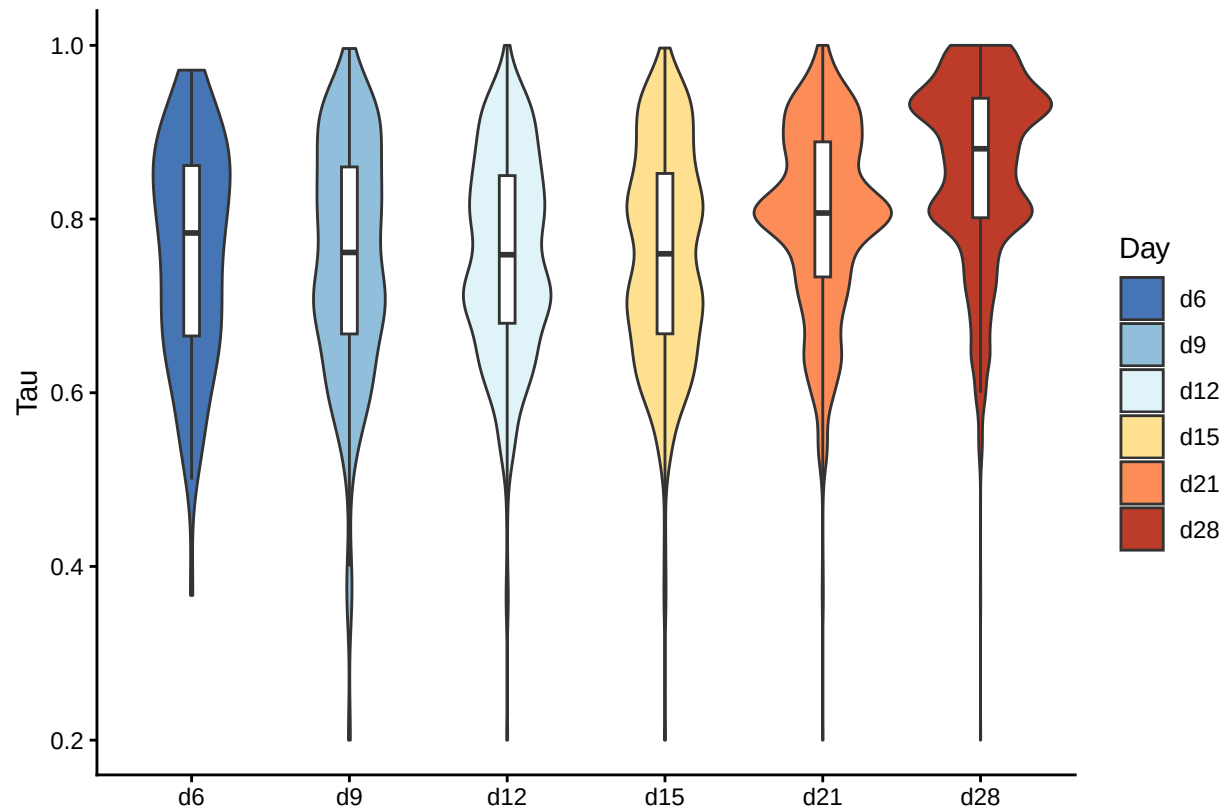
res_index <- res_index[res_index$Day %in% c("d6", "d9", "d12", "d15", "d21", "d28"), ]

ggplot(data = res_index, mapping = aes(x = Day, y = Gini, fill = Day)) +
  geom_violin(trim = TRUE) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  labs(x = "", y = "Gini") +
  scale_fill_manual(values = color_settings) +
  theme_classic()

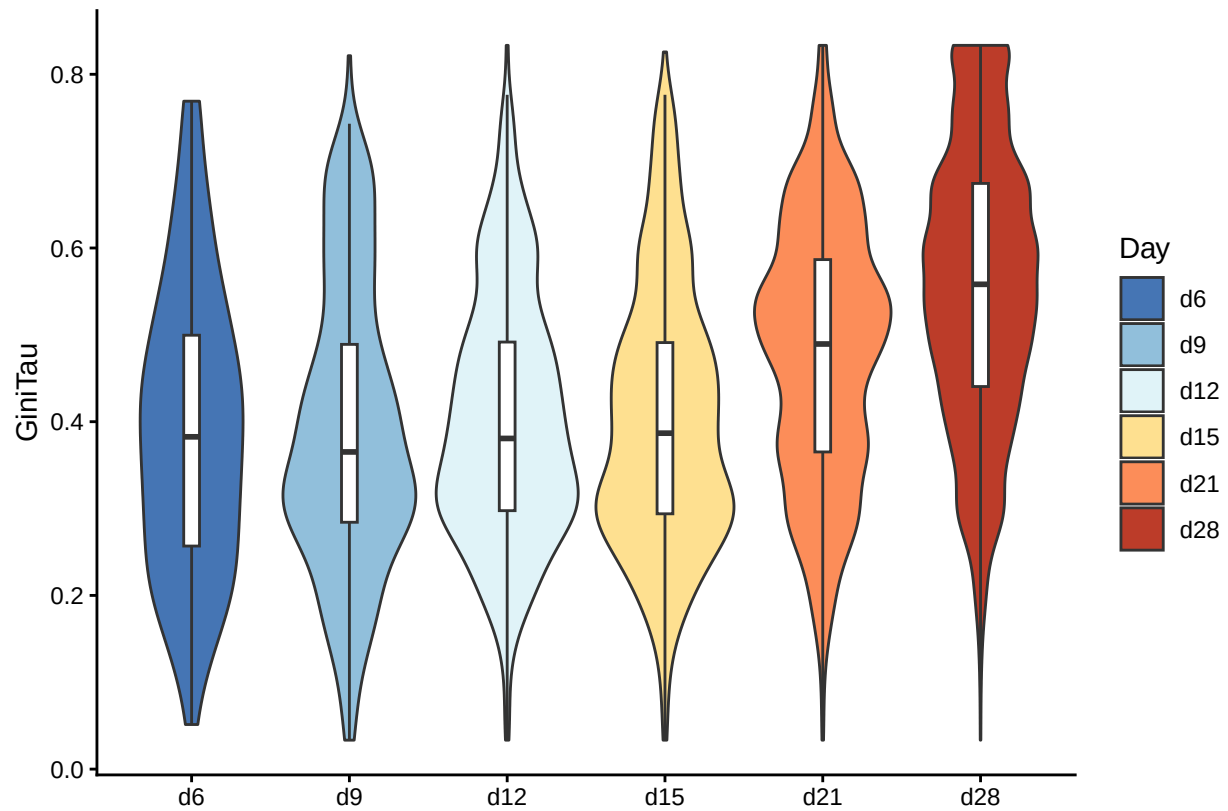
```



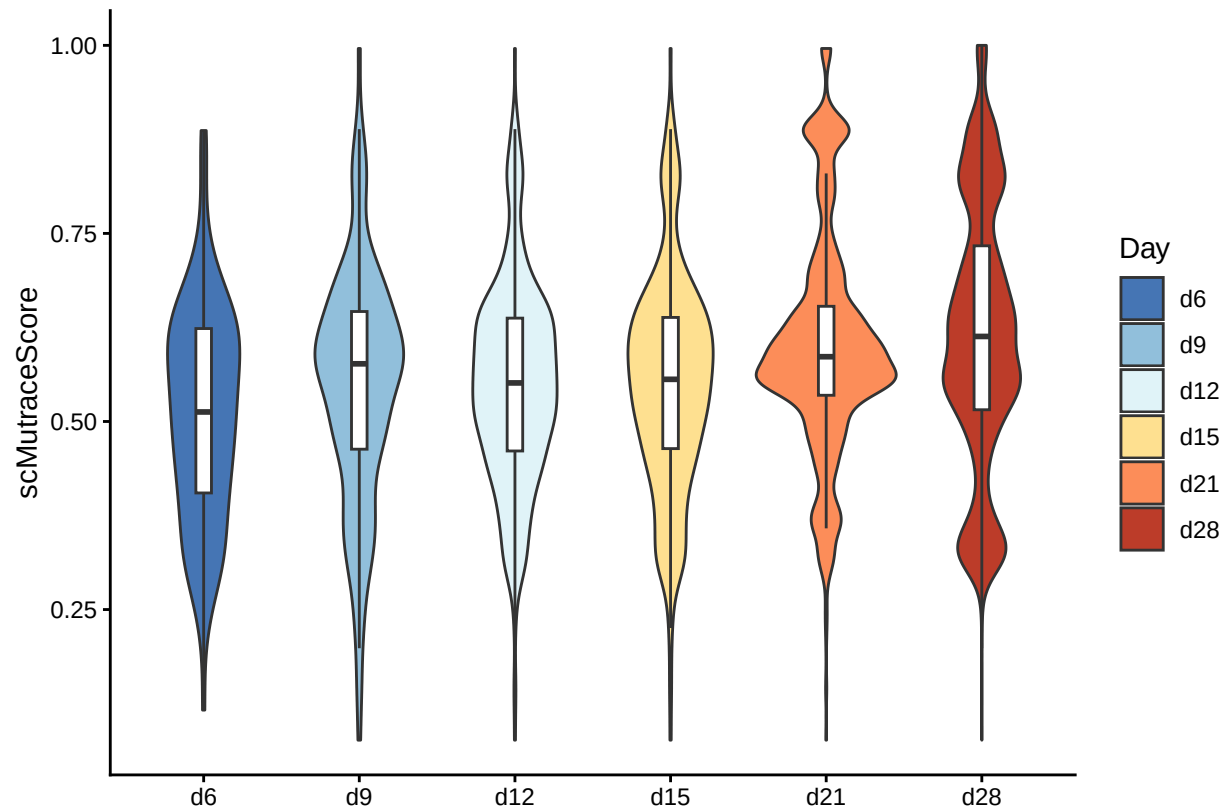
```
ggplot(data = res_index, mapping = aes(x = Day, y = Tau, fill = Day)) +
  geom_violin(trim = TRUE) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  labs(x = "", y = "Tau") +
  scale_fill_manual(values = color_settings) +
  theme_classic()
```



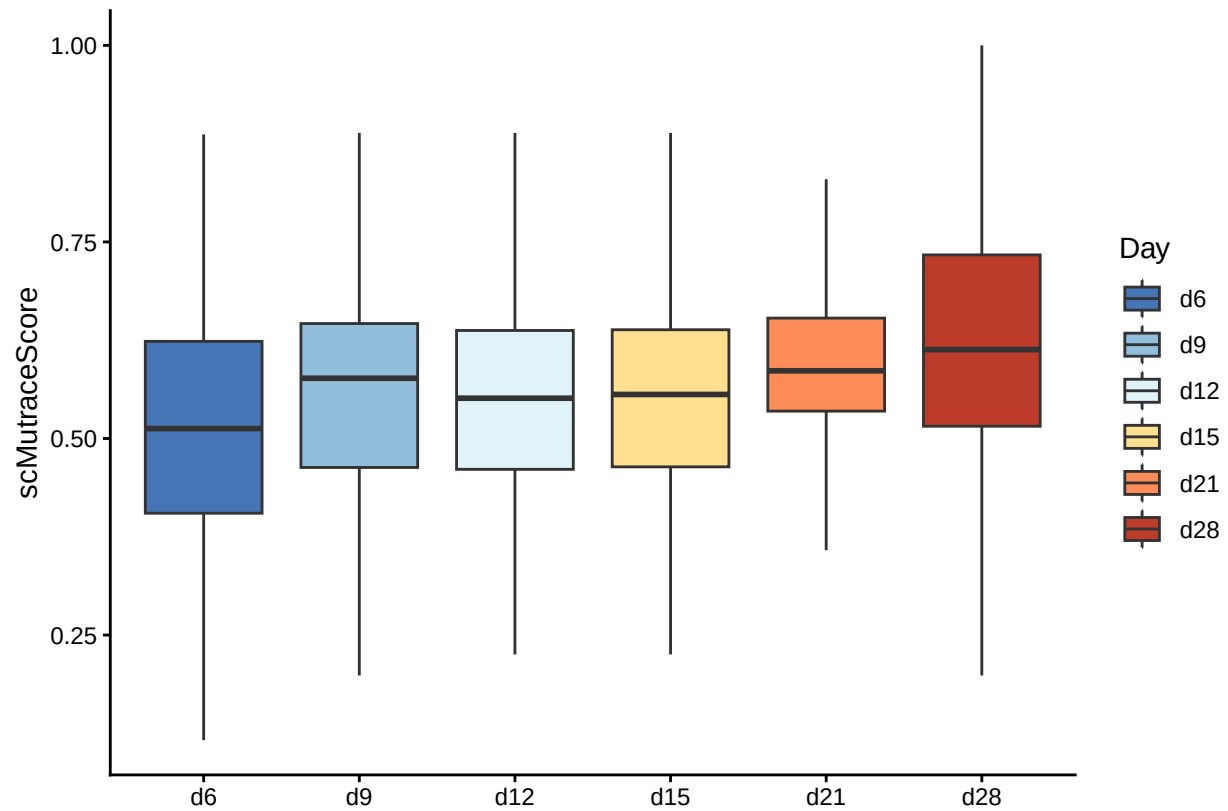
```
ggplot(data = res_index, mapping = aes(x = Day, y = GiniTau, fill = Day)) +
  geom_violin(trim = TRUE) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  labs(x = "", y = "GiniTau") +
  scale_fill_manual(values = color_settings) +
  theme_classic()
```



```
ggplot(data = res_index, mapping = aes(x = Day, y = scMutScore, fill = Day)) +
  geom_violin(trim = TRUE) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  labs(x = "", y = "scMutraceScore") +
  scale_fill_manual(values = color_settings) +
  theme_classic()
```

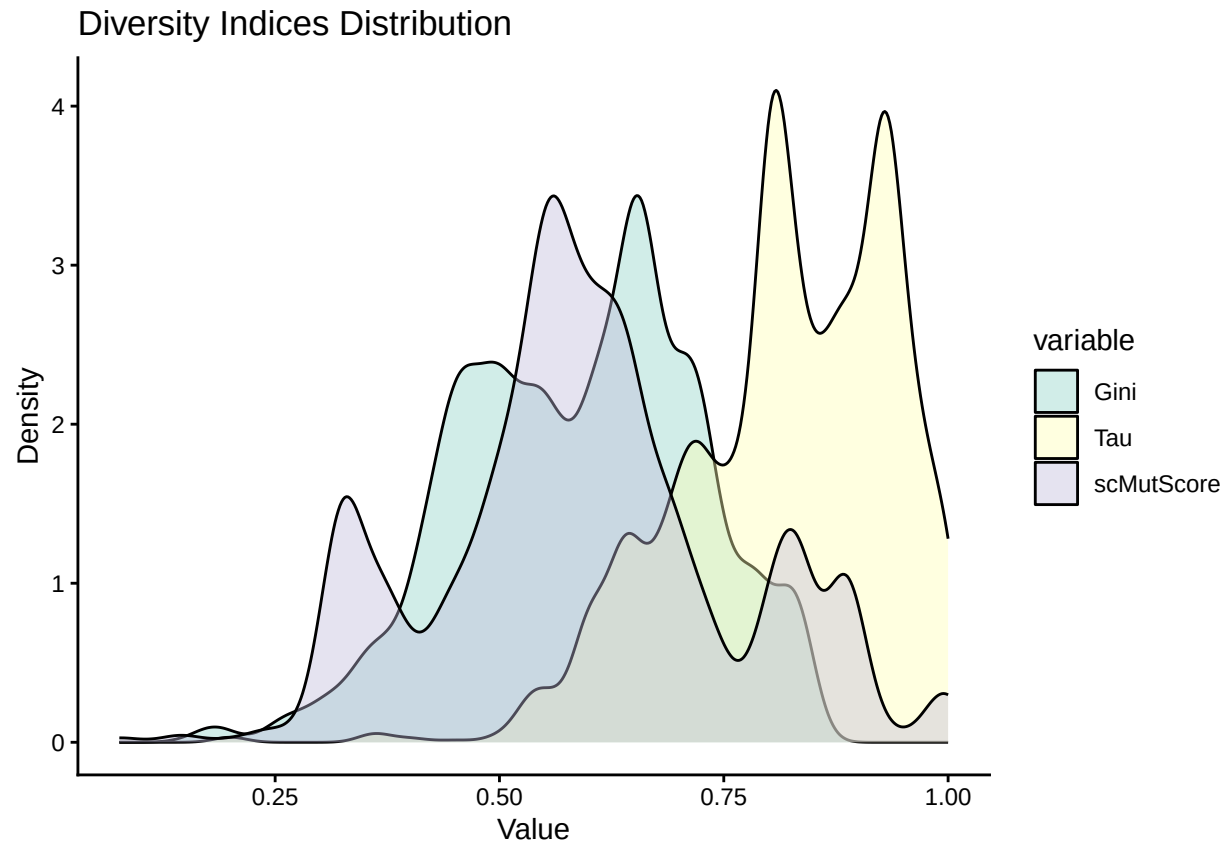


```
ggplot(data = res_index, mapping = aes(x = Day, y = scMutScore, fill = Day)) +
  #geom_violin(trim = TRUE) +
  geom_boxplot(outliers = FALSE) +
  labs(x = "", y = "scMutraceScore") +
  scale_fill_manual(values = color_settings) +
  theme_classic()
```

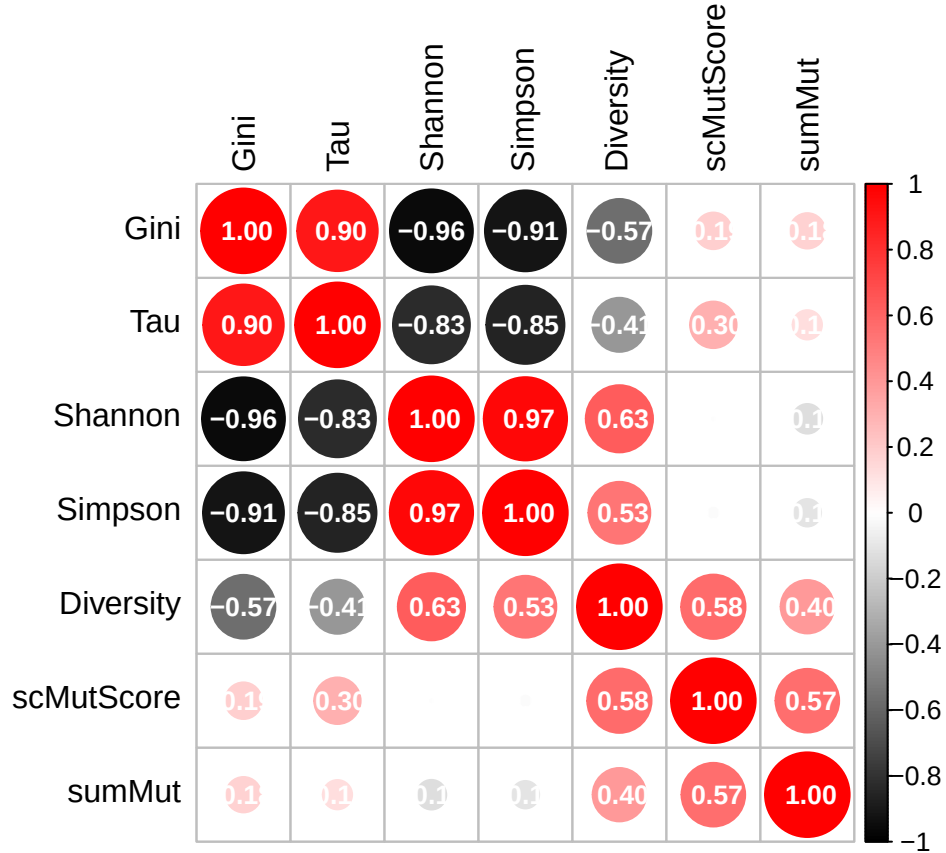


```
res_index_long <- melt(res_index[,c("Gini", "Tau", "scMutScore")])
## No id variables; using all as measure variables

ggplot(res_index_long, aes(x=value, fill=variable)) +
  geom_density(alpha=0.4) +
  theme_classic() +
  labs(x="Value", y="Density", title="Diversity Indices Distribution") +
  scale_fill_brewer(palette="Set3")
```

```
corrplot(cor(res_index[,c("Gini", "Tau", "Shannon", "Simpson", "Diversity", "scMutScore", "sumMut")], us
method="circle", col = colorRampPalette(c("black", "white", "red"))(200),
addCoef.col = "white", tl.col = "black", number.cex = 0.8)
```



11 scMutScore Model Validation

```
set.seed(2025)

# re2 cells number lower than 100
# d12 d15 d21 d28 d3 d6 d9
# 166 190 954 1215 2 18 82

# 1. init
data_check_CM_hf1 <- data_cell_mut_raw_hf1
data_check_Meta_hf1 <- MEF_Meta_hf1[MEF_Meta_hf1$Cellbarcode %in% rownames(data_check_CM_hf1),]
col_check_names_hf1 <- c("hf1.d0", "hf1.d3", "hf1.d6", "hf1.d9", "hf1.d12", "hf1.d15", "hf1.d21", "hf1.d28")
#time_types_hf1 <- c("Day_6", "Day_9", "Day_12", "Day_15", "Day_21", "Day_28")
time_types_hf1 <- c("d6", "d9", "d12", "d15", "d21", "d28")

col_check_names <- c(col_check_names_hf1)
cell_num <- 1
mut_num <- 1
mut_loss_ratio <- 0.5
cell_loss_ratio <- 0.5
iteration_times <- 100

# list for save data
```

```

data_scMutModel_list_hf1 <- vector("list", iteration_times)

num_cells_hf1 <- nrow(data_check_CM_hf1)
num_mutations_hf1 <- ncol(data_check_CM_hf1)
cells_all_hf1 <- rownames(data_check_CM_hf1)
mut_s_all_hf1 <- colnames(data_check_CM_hf1)

rowname_day_hf1 <- data.frame(rowname = rownames(data_check_CM_hf1), day = sub(".*\\.(d\\d+).*", "\\1")
num_randomSelected_forEachTime_hf1 <- ceiling(min(table(rowname_day_hf1$day)) * cell_loss_ratio)

# 2. loop
## -----
## Precompute balanced sampling plan: every cell will be sampled
## before any cell is repeatedly sampled within each day.
## -----
cells_by_day <- split(seq_len(nrow(rowname_day_hf1)), rowname_day_hf1$day)

make_sampling_plan <- function(cells_by_day, n_per_day) {
  lapply(cells_by_day, function(idx) {
    idx_shuffled <- sample(idx)
    split(
      idx_shuffled,
      ceiling(seq_along(idx_shuffled) / n_per_day)
    )
  })
}

sampling_plan <- make_sampling_plan(
  cells_by_day,
  num_randomSelected_forEachTime_hf1
)

max_batches <- max(sapply(sampling_plan, length))
cell_day_map_hf1 <- setNames(rowname_day_hf1$day, rowname_day_hf1$rowname)

for (iter in seq_len(iteration_times)) {

  # 2.1 randomly loss info
  # cell idx
  ## Reshuffle after each full cycle

  if (iter > 1 && (iter - 1) %% max_batches == 0) {
    sampling_plan <- make_sampling_plan(
      cells_by_day,
      num_randomSelected_forEachTime_hf1
    )
    max_batches <- max(sapply(sampling_plan, length))
  }

  ## Balanced cell sampling across days
  loss_cell_indices <- unlist(
    lapply(sampling_plan, function(day_batches) {
      batch_id <- ((iter - 1) %% length(day_batches)) + 1

```

```

    day_batches[[batch_id]]
  }),
  use.names = FALSE
)

loss_mut_indices <- sample(seq_len(num_mutations_hf1), ceiling(mut_loss_ratio * num_mutations_hf1))

C_partial_missing <- data_check_CM_hf1
# new data
if(length(loss_cell_indices) > 0 && length(loss_mut_indices) > 0){
  C_partial_missing[loss_cell_indices, loss_mut_indices] <- "NoCall"
}

# 2.2 count cell/mut number
MutPerCell <- rowSums(C_partial_missing == "Mut")
CellPerMut <- colSums(C_partial_missing == "Mut")

cells_candidate <- cells_all_hf1[MutPerCell >= cell_num & cells_all_hf1 %in% data_check_Meta_hf1$CellID]
mut_candidate <- muts_all_hf1[CellPerMut >= mut_num]

message(paste0("Iteration: ", iter, " loss_cell_indices: ", length(loss_cell_indices), " loss_mut_indices: ", length(loss_mut_indices)))
message(paste0("Iteration: ", iter, " cells_candidate: ", length(cells_candidate), " muts_candidate: ", length(mut_candidate)))

data_cell_mut_raw_check <- C_partial_missing[cells_candidate, mut_candidate, drop = TRUE]

# 2.3 init Mut × Time
M_partial_missing <- matrix(0, nrow = length(mut_candidate), ncol = length(time_types_hf1),
  dimnames = list(mut_candidate, time_types_hf1))

for (mut in mut_candidate) {
  cells_with_mut <- rownames(data_cell_mut_raw_check)[data_cell_mut_raw_check[, mut] == "Mut"]
  cells_with_mut_clean <- sub("\\.[0-9]+$", "", cells_with_mut)
  days <- cell_day_map_hf1[cells_with_mut_clean]
  M_partial_missing[mut, ] <- table(factor(days, levels = time_types_hf1))
}

M_partial_missing_norm <- M_partial_missing

# 2.5 scMutScore
res_index_tmp <- cal_scMutScore(data_cell_mut_raw_check, M_partial_missing_norm)$score
res_index_tmp$iteration <- paste0("iter", iter)
res_index_tmp <- res_index_tmp[res_index_tmp$Diversity > 0, ]
res_index_tmp$cb <- rownames(res_index_tmp)
res_index_tmp$iteration <- paste0("iter", iter)
rownames(res_index_tmp) <- NULL
data_scMutModel_list_hf1[[iter]] <- res_index_tmp
}

## Iteration: 1 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 1 cells_candidate: 4227 muts_candidate: 297
## [1] 4227 297
## [1] 297 6
## Iteration: 2 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 2 cells_candidate: 4230 muts_candidate: 296

```

```

## [1] 4230 296
## [1] 296 6
## Iteration: 3 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 3 cells_candidate: 4266 muts_candidate: 296
## [1] 4266 296
## [1] 296 6
## Iteration: 4 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 4 cells_candidate: 4253 muts_candidate: 297
## [1] 4253 297
## [1] 297 6
## Iteration: 5 loss_cell_indices: 253 loss_mut_indices: 149
## Iteration: 5 cells_candidate: 4250 muts_candidate: 297
## [1] 4250 297
## [1] 297 6
## Iteration: 6 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 6 cells_candidate: 4236 muts_candidate: 296
## [1] 4236 296
## [1] 296 6
## Iteration: 7 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 7 cells_candidate: 4234 muts_candidate: 297
## [1] 4234 297
## [1] 297 6
## Iteration: 8 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 8 cells_candidate: 4225 muts_candidate: 297
## [1] 4225 297
## [1] 297 6
## Iteration: 9 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 9 cells_candidate: 4234 muts_candidate: 296
## [1] 4234 296
## [1] 296 6
## Iteration: 10 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 10 cells_candidate: 4234 muts_candidate: 297
## [1] 4234 297
## [1] 297 6
## Iteration: 11 loss_cell_indices: 229 loss_mut_indices: 149
## Iteration: 11 cells_candidate: 4288 muts_candidate: 297
## [1] 4288 297
## [1] 297 6
## Iteration: 12 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 12 cells_candidate: 4253 muts_candidate: 297
## [1] 4253 297
## [1] 297 6
## Iteration: 13 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 13 cells_candidate: 4243 muts_candidate: 296
## [1] 4243 296
## [1] 296 6
## Iteration: 14 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 14 cells_candidate: 4230 muts_candidate: 297
## [1] 4230 297
## [1] 297 6
## Iteration: 15 loss_cell_indices: 221 loss_mut_indices: 149
## Iteration: 15 cells_candidate: 4267 muts_candidate: 297
## [1] 4267 297

```

```

## [1] 297 6
## Iteration: 16 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 16 cells_candidate: 4246 muts_candidate: 297
## [1] 4246 297
## [1] 297 6
## Iteration: 17 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 17 cells_candidate: 4260 muts_candidate: 295
## [1] 4260 295
## [1] 295 6
## Iteration: 18 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 18 cells_candidate: 4231 muts_candidate: 297
## [1] 4231 297
## [1] 297 6
## Iteration: 19 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 19 cells_candidate: 4254 muts_candidate: 297
## [1] 4254 297
## [1] 297 6
## Iteration: 20 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 20 cells_candidate: 4238 muts_candidate: 297
## [1] 4238 297
## [1] 297 6
## Iteration: 21 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 21 cells_candidate: 4238 muts_candidate: 296
## [1] 4238 296
## [1] 296 6
## Iteration: 22 loss_cell_indices: 228 loss_mut_indices: 149
## Iteration: 22 cells_candidate: 4251 muts_candidate: 296
## [1] 4251 296
## [1] 296 6
## Iteration: 23 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 23 cells_candidate: 4232 muts_candidate: 296
## [1] 4232 296
## [1] 296 6
## Iteration: 24 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 24 cells_candidate: 4255 muts_candidate: 297
## [1] 4255 297
## [1] 297 6
## Iteration: 25 loss_cell_indices: 253 loss_mut_indices: 149
## Iteration: 25 cells_candidate: 4249 muts_candidate: 296
## [1] 4249 296
## [1] 296 6
## Iteration: 26 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 26 cells_candidate: 4265 muts_candidate: 296
## [1] 4265 296
## [1] 296 6
## Iteration: 27 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 27 cells_candidate: 4232 muts_candidate: 296
## [1] 4232 296
## [1] 296 6
## Iteration: 28 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 28 cells_candidate: 4276 muts_candidate: 297
## [1] 4276 297
## [1] 297 6

```

```

## Iteration: 29 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 29 cells_candidate: 4226 muts_candidate: 297
## [1] 4226 297
## [1] 297 6
## Iteration: 30 loss_cell_indices: 220 loss_mut_indices: 149
## Iteration: 30 cells_candidate: 4292 muts_candidate: 297
## [1] 4292 297
## [1] 297 6
## Iteration: 31 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 31 cells_candidate: 4245 muts_candidate: 296
## [1] 4245 296
## [1] 296 6
## Iteration: 32 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 32 cells_candidate: 4255 muts_candidate: 296
## [1] 4255 296
## [1] 296 6
## Iteration: 33 loss_cell_indices: 229 loss_mut_indices: 149
## Iteration: 33 cells_candidate: 4258 muts_candidate: 297
## [1] 4258 297
## [1] 297 6
## Iteration: 34 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 34 cells_candidate: 4245 muts_candidate: 297
## [1] 4245 297
## [1] 297 6
## Iteration: 35 loss_cell_indices: 253 loss_mut_indices: 149
## Iteration: 35 cells_candidate: 4279 muts_candidate: 297
## [1] 4279 297
## [1] 297 6
## Iteration: 36 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 36 cells_candidate: 4259 muts_candidate: 297
## [1] 4259 297
## [1] 297 6
## Iteration: 37 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 37 cells_candidate: 4245 muts_candidate: 296
## [1] 4245 296
## [1] 296 6
## Iteration: 38 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 38 cells_candidate: 4253 muts_candidate: 296
## [1] 4253 296
## [1] 296 6
## Iteration: 39 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 39 cells_candidate: 4228 muts_candidate: 297
## [1] 4228 297
## [1] 297 6
## Iteration: 40 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 40 cells_candidate: 4270 muts_candidate: 297
## [1] 4270 297
## [1] 297 6
## Iteration: 41 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 41 cells_candidate: 4231 muts_candidate: 297
## [1] 4231 297
## [1] 297 6
## Iteration: 42 loss_cell_indices: 263 loss_mut_indices: 149

```

```

## Iteration: 42 cells_candidate: 4240 muts_candidate: 297
## [1] 4240 297
## [1] 297 6
## Iteration: 43 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 43 cells_candidate: 4242 muts_candidate: 297
## [1] 4242 297
## [1] 297 6
## Iteration: 44 loss_cell_indices: 228 loss_mut_indices: 149
## Iteration: 44 cells_candidate: 4268 muts_candidate: 297
## [1] 4268 297
## [1] 297 6
## Iteration: 45 loss_cell_indices: 221 loss_mut_indices: 149
## Iteration: 45 cells_candidate: 4282 muts_candidate: 297
## [1] 4282 297
## [1] 297 6
## Iteration: 46 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 46 cells_candidate: 4249 muts_candidate: 297
## [1] 4249 297
## [1] 297 6
## Iteration: 47 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 47 cells_candidate: 4239 muts_candidate: 295
## [1] 4239 295
## [1] 295 6
## Iteration: 48 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 48 cells_candidate: 4239 muts_candidate: 295
## [1] 4239 295
## [1] 295 6
## Iteration: 49 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 49 cells_candidate: 4245 muts_candidate: 297
## [1] 4245 297
## [1] 297 6
## Iteration: 50 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 50 cells_candidate: 4273 muts_candidate: 297
## [1] 4273 297
## [1] 297 6
## Iteration: 51 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 51 cells_candidate: 4255 muts_candidate: 296
## [1] 4255 296
## [1] 296 6
## Iteration: 52 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 52 cells_candidate: 4245 muts_candidate: 296
## [1] 4245 296
## [1] 296 6
## Iteration: 53 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 53 cells_candidate: 4266 muts_candidate: 297
## [1] 4266 297
## [1] 297 6
## Iteration: 54 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 54 cells_candidate: 4231 muts_candidate: 297
## [1] 4231 297
## [1] 297 6
## Iteration: 55 loss_cell_indices: 180 loss_mut_indices: 149
## Iteration: 55 cells_candidate: 4281 muts_candidate: 297

```



```

## [1] 4281 297
## [1] 297 6
## Iteration: 56 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 56 cells_candidate: 4262 muts_candidate: 296
## [1] 4262 296
## [1] 296 6
## Iteration: 57 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 57 cells_candidate: 4272 muts_candidate: 296
## [1] 4272 296
## [1] 296 6
## Iteration: 58 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 58 cells_candidate: 4230 muts_candidate: 297
## [1] 4230 297
## [1] 297 6
## Iteration: 59 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 59 cells_candidate: 4227 muts_candidate: 297
## [1] 4227 297
## [1] 297 6
## Iteration: 60 loss_cell_indices: 220 loss_mut_indices: 149
## Iteration: 60 cells_candidate: 4277 muts_candidate: 297
## [1] 4277 297
## [1] 297 6
## Iteration: 61 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 61 cells_candidate: 4269 muts_candidate: 296
## [1] 4269 296
## [1] 296 6
## Iteration: 62 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 62 cells_candidate: 4241 muts_candidate: 295
## [1] 4241 295
## [1] 295 6
## Iteration: 63 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 63 cells_candidate: 4250 muts_candidate: 297
## [1] 4250 297
## [1] 297 6
## Iteration: 64 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 64 cells_candidate: 4234 muts_candidate: 297
## [1] 4234 297
## [1] 297 6
## Iteration: 65 loss_cell_indices: 253 loss_mut_indices: 149
## Iteration: 65 cells_candidate: 4227 muts_candidate: 297
## [1] 4227 297
## [1] 297 6
## Iteration: 66 loss_cell_indices: 228 loss_mut_indices: 149
## Iteration: 66 cells_candidate: 4260 muts_candidate: 297
## [1] 4260 297
## [1] 297 6
## Iteration: 67 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 67 cells_candidate: 4252 muts_candidate: 296
## [1] 4252 296
## [1] 296 6
## Iteration: 68 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 68 cells_candidate: 4231 muts_candidate: 296
## [1] 4231 296

```

```

## [1] 296 6
## Iteration: 69 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 69 cells_candidate: 4230 muts_candidate: 296
## [1] 4230 296
## [1] 296 6
## Iteration: 70 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 70 cells_candidate: 4265 muts_candidate: 297
## [1] 4265 297
## [1] 297 6
## Iteration: 71 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 71 cells_candidate: 4233 muts_candidate: 296
## [1] 4233 296
## [1] 296 6
## Iteration: 72 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 72 cells_candidate: 4235 muts_candidate: 296
## [1] 4235 296
## [1] 296 6
## Iteration: 73 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 73 cells_candidate: 4257 muts_candidate: 297
## [1] 4257 297
## [1] 297 6
## Iteration: 74 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 74 cells_candidate: 4241 muts_candidate: 297
## [1] 4241 297
## [1] 297 6
## Iteration: 75 loss_cell_indices: 221 loss_mut_indices: 149
## Iteration: 75 cells_candidate: 4257 muts_candidate: 297
## [1] 4257 297
## [1] 297 6
## Iteration: 76 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 76 cells_candidate: 4239 muts_candidate: 297
## [1] 4239 297
## [1] 297 6
## Iteration: 77 loss_cell_indices: 229 loss_mut_indices: 149
## Iteration: 77 cells_candidate: 4249 muts_candidate: 297
## [1] 4249 297
## [1] 297 6
## Iteration: 78 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 78 cells_candidate: 4234 muts_candidate: 297
## [1] 4234 297
## [1] 297 6
## Iteration: 79 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 79 cells_candidate: 4225 muts_candidate: 296
## [1] 4225 296
## [1] 296 6
## Iteration: 80 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 80 cells_candidate: 4229 muts_candidate: 297
## [1] 4229 297
## [1] 297 6
## Iteration: 81 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 81 cells_candidate: 4265 muts_candidate: 297
## [1] 4265 297
## [1] 297 6

```

```

## Iteration: 82 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 82 cells_candidate: 4260 muts_candidate: 297
## [1] 4260 297
## [1] 297 6
## Iteration: 83 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 83 cells_candidate: 4243 muts_candidate: 296
## [1] 4243 296
## [1] 296 6
## Iteration: 84 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 84 cells_candidate: 4262 muts_candidate: 296
## [1] 4262 296
## [1] 296 6
## Iteration: 85 loss_cell_indices: 253 loss_mut_indices: 149
## Iteration: 85 cells_candidate: 4272 muts_candidate: 297
## [1] 4272 297
## [1] 297 6
## Iteration: 86 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 86 cells_candidate: 4261 muts_candidate: 297
## [1] 4261 297
## [1] 297 6
## Iteration: 87 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 87 cells_candidate: 4268 muts_candidate: 295
## [1] 4268 295
## [1] 295 6
## Iteration: 88 loss_cell_indices: 228 loss_mut_indices: 149
## Iteration: 88 cells_candidate: 4246 muts_candidate: 297
## [1] 4246 297
## [1] 297 6
## Iteration: 89 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 89 cells_candidate: 4253 muts_candidate: 297
## [1] 4253 297
## [1] 297 6
## Iteration: 90 loss_cell_indices: 220 loss_mut_indices: 149
## Iteration: 90 cells_candidate: 4276 muts_candidate: 296
## [1] 4276 296
## [1] 296 6
## Iteration: 91 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 91 cells_candidate: 4262 muts_candidate: 297
## [1] 4262 297
## [1] 297 6
## Iteration: 92 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 92 cells_candidate: 4253 muts_candidate: 296
## [1] 4253 296
## [1] 296 6
## Iteration: 93 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 93 cells_candidate: 4249 muts_candidate: 297
## [1] 4249 297
## [1] 297 6
## Iteration: 94 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 94 cells_candidate: 4245 muts_candidate: 296
## [1] 4245 296
## [1] 296 6
## Iteration: 95 loss_cell_indices: 253 loss_mut_indices: 149

```

```

## Iteration: 95 cells_candidate: 4249 muts_candidate: 297
## [1] 4249 297
## [1] 297 6
## Iteration: 96 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 96 cells_candidate: 4258 muts_candidate: 297
## [1] 4258 297
## [1] 297 6
## Iteration: 97 loss_cell_indices: 264 loss_mut_indices: 149
## Iteration: 97 cells_candidate: 4231 muts_candidate: 296
## [1] 4231 296
## [1] 296 6
## Iteration: 98 loss_cell_indices: 263 loss_mut_indices: 149
## Iteration: 98 cells_candidate: 4243 muts_candidate: 297
## [1] 4243 297
## [1] 297 6
## Iteration: 99 loss_cell_indices: 229 loss_mut_indices: 149
## Iteration: 99 cells_candidate: 4268 muts_candidate: 295
## [1] 4268 295
## [1] 295 6
## Iteration: 100 loss_cell_indices: 252 loss_mut_indices: 149
## Iteration: 100 cells_candidate: 4243 muts_candidate: 297
## [1] 4243 297
## [1] 297 6

# 3. merge res
data_scMutModel_res <- do.call(rbind, data_scMutModel_list_hf1)

# 4. column
data_scMutModel_res <- separate(data_scMutModel_res, col = "cb", into = c("celltype", "cell"), sep = "_")
data_scMutModel_res$Tag <- data_scMutModel_res$celltype
data_scMutModel_res <- separate(data_scMutModel_res, col = "celltype", into = c("rep", "Day"), sep = "\\.")

data_scMutModel_res <- data_scMutModel_res[data_scMutModel_res$Day %in% c("d6", "d9", "d12", "d15", "d21"), ]

data_scMutModel_res$Tag <- factor(data_scMutModel_res$Tag, levels = col_check_names)
data_scMutModel_res$Day <- factor(data_scMutModel_res$Day, levels = c("d6", "d9", "d12", "d15", "d21", "d25"))

# 5. plot
data_scMutModel_res_summary <- data_scMutModel_res %>%
  dplyr::group_by(iteration, Day) %>%
  dplyr::summarise(mean_scMutScore = mean(scMutScore, na.rm = TRUE)) %>%
  dplyr::ungroup()
## `summarise()` has grouped output by 'iteration'. You can override using the
## `.groups` argument.

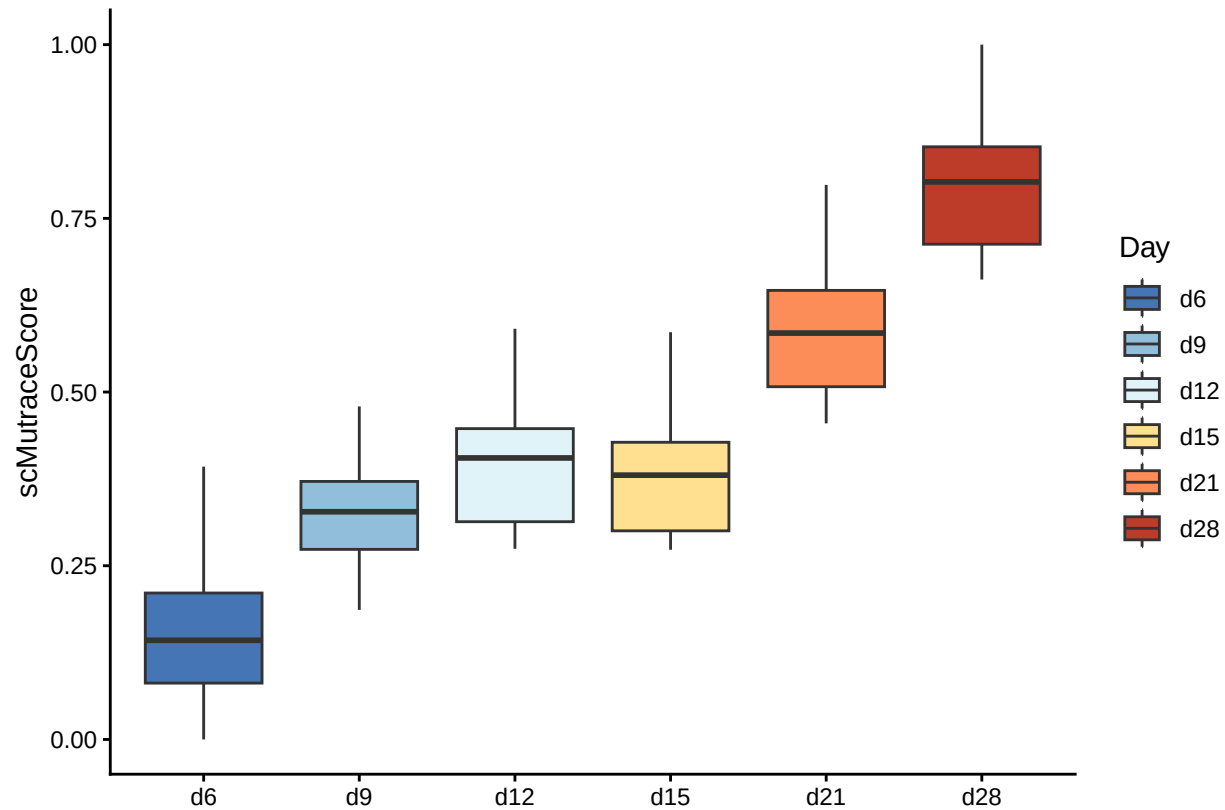
data_scMutModel_res_summary <- data_scMutModel_res_summary %>%
  dplyr::mutate(
    mean_scMutScore_scaled = scales::rescale(mean_scMutScore, to = c(0, 1))
  )

p_iter_scMutraceScore <- ggplot(data = data_scMutModel_res_summary, mapping = aes(x = Day, y = mean_scMutScore_scaled)) +
  geom_boxplot(outliers = FALSE) +

```

```
labs(x = "", y = "scMutraceScore") +
scale_fill_manual(values = color_settings) +
theme_classic()
```

p_iter_scMutraceScore



12 compareClonesScore

```
# generate data for comparision
MEF_meta_hf1_hf2 <- rbind(MEF_Meta_hf1, MEF_Meta_hf2)
res_index_scMut <- data.frame(Cellbarcode = rownames(res_index), scMutScore = res_index$scMutScore, Tag

res_index_scMut_meta <- merge(res_index_scMut, MEF_meta_hf1_hf2, by = "Cellbarcode")
res_index_scMut_meta$Day <- factor(res_index_scMut_meta$Day, levels = c("Day_0", "Day_3", "Day_6", "Day
res_index_scMut_meta <- res_index_scMut_meta[res_index_scMut_meta$Day %in% c("Day_6", "Day_9", "Day_12"

colNames_data_clone_score <- c("D0_tag", "D3_tag", "D13_tag", "Day", "Mean_Day_score", "Mean_Day_pseudo
data_clone_score <- as.data.frame(matrix(ncol = length(colNames_data_clone_score)))
colnames(data_clone_score) <- colNames_data_clone_score
d0_tags <- unique(MEF_meta_hf1_hf2$CellTagD0_48k)

count_line = 0
```

```

for (i in 1:length(d0_tags)){
  d0_tag_tmp <- d0_tags[i]
  d3_tags <- unique(MEF_meta_hf1_hf2[MEF_meta_hf1_hf2$CellTagD0_48k == d0_tag_tmp,]$CellTagD3_48k)
  days_reprogramming <- unique(MEF_meta_hf1_hf2[MEF_meta_hf1_hf2$CellTagD0_48k == d0_tag_tmp,]$Day)
  for (day in 1:length(days_reprogramming)){
    if (length(d3_tags) >= 1){
      for (j in 1:length(d3_tags)){
        d3_tag_tmp <- d3_tags[j]
        d13_tags <- unique(MEF_meta_hf1_hf2[MEF_meta_hf1_hf2$CellTagD0_48k == d0_tag_tmp & MEF_meta_hf1_hf2$CellTagD3_48k == d3_tag_tmp,]$CellTagD13_48k)
        if(length(d13_tags) >= 1){
          for (k in 1:length(d13_tags)){
            d13_tag_tmp <- d13_tags[k]
            count_line = count_line + 1
            data_clone_score[count_line,]$Day = days_reprogramming[day]
            data_clone_score[count_line,]$D0_tag = d0_tag_tmp
            data_clone_score[count_line,]$D3_tag = d3_tag_tmp
            data_clone_score[count_line,]$D13_tag = d13_tag_tmp
          }
        }else{
          count_line = count_line + 1
          data_clone_score[count_line,]$Day = days_reprogramming[day]
          data_clone_score[count_line,]$D0_tag = d0_tag_tmp
          data_clone_score[count_line,]$D3_tag = d3_tag_tmp
          data_clone_score[count_line,]$D13_tag = "NotAva"
        }
      }
    }else{
      count_line = count_line + 1
      data_clone_score[count_line,]$Day = days_reprogramming[day]
      data_clone_score[count_line,]$D0_tag = d0_tag_tmp
      data_clone_score[count_line,]$D3_tag = "NotAva"
      data_clone_score[count_line,]$D13_tag = "NotAva"
    }
  }
}

data_clone_score$ID <- paste(data_clone_score$D0_tag, data_clone_score$D3_tag, data_clone_score$D13_tag, sep = "_")
data_clone_score <- data_clone_score[data_clone_score$ID != "NotAva_NotAva_NotAva",]

for (i in 1:nrow(data_clone_score)){

  d0_tag_tmp <- data_clone_score[i,]$D0_tag
  d3_tag_tmp <- data_clone_score[i,]$D3_tag
  d13_tag_tmp <- data_clone_score[i,]$D13_tag
  day_tmp <- data_clone_score[i,]$Day

  data_clone_score[i,]$Mean_Day_score <- "NotAva"
  data_clone_score[i,]$Mean_Day_pseudotime <- "NotAva"

  id_check <- paste(d0_tag_tmp, d3_tag_tmp, d13_tag_tmp, sep = "_")
  data_clone_score[i,]$ID <- id_check
}

```

```

scMutscore_tmp <- res_index_scMut_meta[res_index_scMut_meta$cloneTimeTag == id_check & res_index_scMut_meta$cloneTimeTag == id_check & res_index_scMut_meta$cloneTimeTag == id_check]
pseudotime_tmp <- res_index_scMut_meta[res_index_scMut_meta$cloneTimeTag == id_check & res_index_scMut_meta$cloneTimeTag == id_check & res_index_scMut_meta$cloneTimeTag == id_check]

scMutscore_tmp <- as.numeric(scMutscore_tmp[scMutscore_tmp != "NotAva"])
pseudotime_tmp <- as.numeric(pseudotime_tmp[pseudotime_tmp != "NotAva"])

data_clone_score[i,]$Mean_Day_score <- mean(scMutscore_tmp)
data_clone_score[i,]$Mean_Day_pseudotime <- mean(pseudotime_tmp)
}

# filter
# clonetagDay
data_clone_score_clonetagDay <- data_clone_score[data_clone_score$Mean_Day_score != "NaN" & data_clone_score$Mean_Day_pseudotime != "NaN"]
ID_counts <- table(data_clone_score_clonetagDay$ID)
valid_IDs <- names(ID_counts[ID_counts >= 2])
data_clone_score_clonetagDay <- subset(data_clone_score_clonetagDay, ID %in% valid_IDs)

data_clone_score_clonetagDay$Mean_Day_score <- as.numeric(data_clone_score_clonetagDay$Mean_Day_score)
data_clone_score_clonetagDay$Mean_Day_pseudotime <- as.numeric(data_clone_score_clonetagDay$Mean_Day_pseudotime)

data_clone_score_clonetagDay$Day <- factor(data_clone_score_clonetagDay$Day, levels = c("Day_0", "Day_3"))

# reprogramming Day
p_clone_scMutraceScore <- ggplot(data_clone_score_clonetagDay, mapping = aes(x = as.factor(Day), y = Mean_Day_score)) +
  geom_boxplot(outliers = TRUE) +
  geom_line(aes(group = ID), color = "gray", alpha = 0.1) +
  scale_fill_manual(values = color_settings_2) +
  ylim(0,1) +
  labs(x = "", y = "scMutraceScore") +
  theme_classic()

p_clone_pseudotime <- ggplot(data_clone_score_clonetagDay, aes(x = as.factor(Day), y = Mean_Day_pseudotime)) +
  geom_boxplot(outliers = TRUE) +
  geom_line(aes(group = ID), color = "gray", alpha = 0.1) +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = "Pseudotime") +
  theme_classic()

p_cell_scMutraceScore <- ggplot(data = res_index_scMut_meta, mapping = aes(x = Day, y = scMutScore, fill = cloneTimeTag)) +
  geom_boxplot(outliers = FALSE) +
  ylim(0,1) +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = "scMutraceScore") +
  theme_classic()

p_cell_pseudotime <- ggplot(data = res_index_scMut_meta[res_index_scMut_meta$pseudotime != "NotAva",], mapping = aes(x = Day, y = pseudotime, fill = cloneTimeTag)) +
  geom_boxplot(outliers = FALSE) +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = "pseudotime") +
  theme_classic()

res_index_scMut_meta_success <- res_index_scMut_meta[res_index_scMut_meta$Outcome == "Successful",][,c("Day", "scMutScore", "pseudotime", "cloneTimeTag")]

```

```

res_index_scMut_meta_deadend <- res_index_scMut_meta[res_index_scMut_meta$Outcome == "DeadEnd",][,c("scMutScore", "Pseudotime")]

res_index_scMut_meta_success_deadend <- rbind(res_index_scMut_meta_success, res_index_scMut_meta_deadend)

# Outcome

p_reprogramming_scMutTraceScore <- ggplot(res_index_scMut_meta[res_index_scMut_meta$reprogramming %in% c("F_reprogramming", "T_reprogramming"),],
  labs(x = "", y = "scMutScore") +
  geom_boxplot(outliers = FALSE) +
  stat_compare_means(method = "wilcox.test", method.args = list(alternative = "two.sided")) +
  scale_fill_manual(values = c("#7876B1FF", "#FFDC91FF")) +
  theme_classic()

p_reprogramming_pseudotime <- ggplot(res_index_scMut_meta[res_index_scMut_meta$reprogramming %in% c("F_reprogramming", "T_reprogramming"),],
  labs(x = "", y = "pseudotime") +
  geom_boxplot(outliers = FALSE) +
  stat_compare_means(method = "wilcox.test", method.args = list(alternative = "two.sided")) +
  scale_fill_manual(values = c("#7876B1FF", "#FFDC91FF")) +
  theme_classic()

timepoint_color <- c(Day_6 = '#7D5188', Day_9 = '#7D71A5', Day_12 = '#738DAF', Day_15 = '#6EA5B0', Day_18 = '#6A8CB8', Day_21 = '#6A8CB8')
cluster_color <- c(c0 = '#D5393E', c1 = '#5A6B93', c2 = '#519376', c3 = '#6E8371', c4 = '#A55F7F', c5 = '#A55F7F')
outcom_color <- c(BAB = "lightgray", DeadEnd = "#7876B1FF", Succesful = "#FFDC91FF")
reprogramming_colors <- c(F_reprogramming = '#7876B1FF', T_reprogramming = '#FFDC91FF', NotAva = "#d9d9d9")

p_umap_reprogramming <- ggplot(data = res_index_scMut_meta[res_index_scMut_meta$reprogramming %in% c("F_reprogramming", "T_reprogramming"),],
  scale_color_manual(values = reprogramming_colors) +
  xlab("") + ylab("") +
  theme_classic()

p_umap_outcome <- ggplot(data = res_index_scMut_meta[res_index_scMut_meta$Outcome %in% c("DeadEnd", "Succesful"),],
  scale_color_manual(values = outcom_color) +
  xlab("") + ylab("") +
  theme_classic()

p_umap_cell_day <- ggplot(data = res_index_scMut_meta) + geom_point(mapping = aes(x = TSNE_1, y = TSNE_2)) +
  scale_color_manual(values = color_settings_2) +
  xlab("") + ylab("") +
  theme_classic()

res_index_scMut_meta$cluster <- paste0("c", res_index_scMut_meta$cluster)
p_umap_cell_cluster <- ggplot(data = res_index_scMut_meta) + geom_point(mapping = aes(x = TSNE_1, y = TSNE_2)) +
  scale_color_manual(values = cluster_color) +
  xlab("") + ylab("") +
  theme_classic()

p_umap_cell_Pseudotime <- ggplot(res_index_scMut_meta[res_index_scMut_meta$pseudotime != "NotAva",], aes(x = TSNE_1, y = TSNE_2)) +
  geom_point(shape = 21, size = 2, alpha = 0.3) +
  scale_fill_gradientn(colours = c("#313695", "#4575b4", "#74add1", "#abd9e9", "#e0f3f8", "#ffffbf", "#fdae61", "#ff7f00", "#d95f02", "#993366", "#800000")) +
  theme_classic() +
  labs(fill = "Pseudotime")

p_umap_cell_scMutScore <- ggplot(res_index_scMut_meta[res_index_scMut_meta$scMutScore != "NotAva",], aes(x = TSNE_1, y = TSNE_2)) +
  geom_point(shape = 21, size = 2, alpha = 0.3) +
  scale_fill_gradientn(colours = c("#313695", "#4575b4", "#74add1", "#abd9e9", "#e0f3f8", "#ffffbf", "#fdae61", "#ff7f00", "#d95f02", "#993366", "#800000")) +
  theme_classic() +
  labs(fill = "scMutScore")

```



```

geom_point(shape = 21, size = 2, alpha = 0.3) +
scale_fill_gradientn(colours = c("#313695", "#4575b4", "#74add1", "#abd9e9", "#e0f3f8", "#ffffbf", "#f4a460", "#d73027", "#a65628", "#81c784", "#4daf4a", "#1b9e77", "#8c564b", "#e377c2", "#7f7e7e", "#bcbd22", "#17becf")) +
theme_classic() +
labs(fill = "scMutScore")

data_scMutScore_pseudotime <- res_index_scMut_meta[,c("scMutScore", "pseudotime")]
data_scMutScore_pseudotime <- data_scMutScore_pseudotime[data_scMutScore_pseudotime$scMutScore != "NotA"]
data_scMutScore_pseudotime$scMutScore <- as.numeric(data_scMutScore_pseudotime$scMutScore)
data_scMutScore_pseudotime$pseudotime <- as.numeric(data_scMutScore_pseudotime$pseudotime)
data_scMutScore_pseudotime$pseudotime_scale <- (data_scMutScore_pseudotime$pseudotime - min(data_scMutScore_pseudotime$pseudotime)) / (max(data_scMutScore_pseudotime$pseudotime) - min(data_scMutScore_pseudotime$pseudotime))

p_cor_scMutScore_pseudotime <- ggplot(data_scMutScore_pseudotime, aes(x = scMutScore, y = pseudotime_scale)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE, color = "blue") +
  stat_cor(method = "pearson", label.x = 0.5, label.y = 1) +
  labs(x = "scMutScore", y = "Pseudotime") +
  theme_classic()

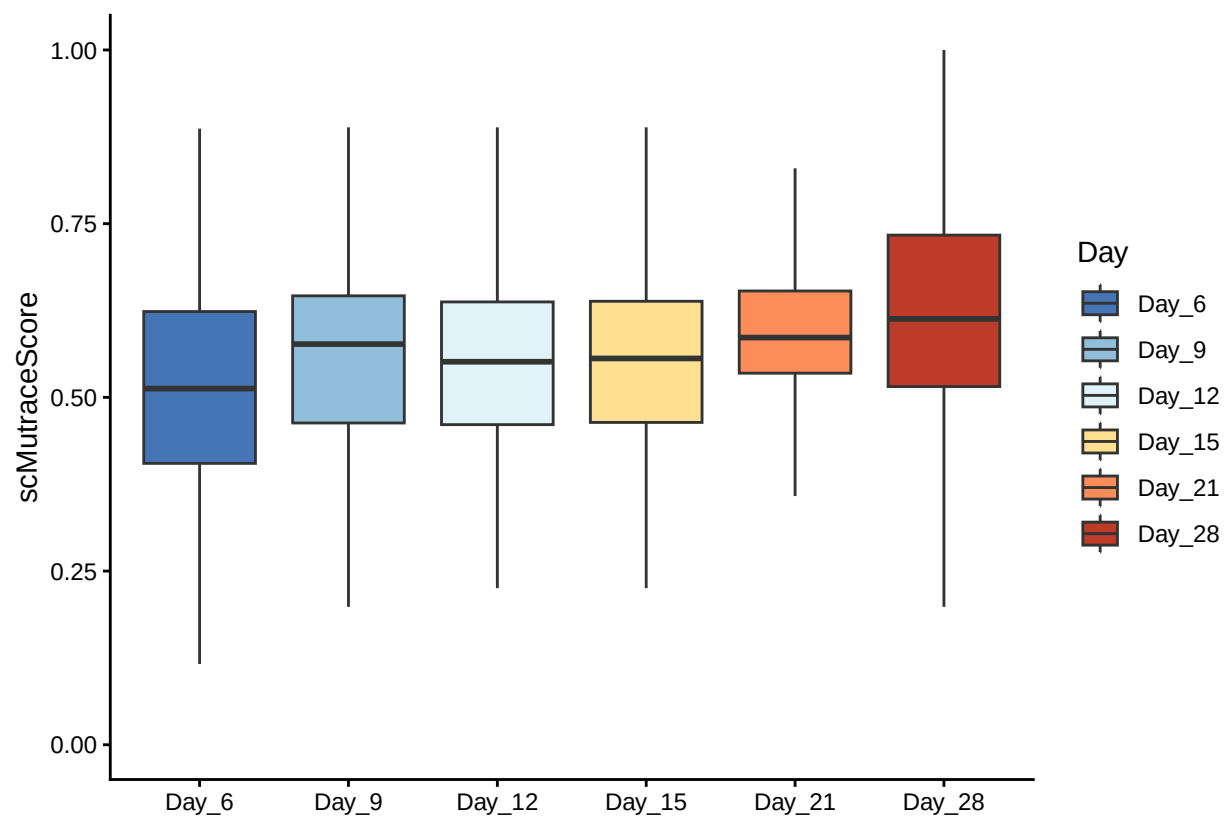
p_cor_scMutScore_sumMut <- ggplot(res_index, aes(x = scMutScore, y = log2(sumMut))) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE, color = "blue") +
  stat_cor(method = "pearson", label.x = 0.5, label.y = 1) +
  labs(x = "scMutScore", y = "sum of matrix") +
  theme_classic()

cor.test(data_scMutScore_pseudotime$scMutScore, data_scMutScore_pseudotime$pseudotime_scale)
##
## Pearson's product-moment correlation
##
## data: data_scMutScore_pseudotime$scMutScore and data_scMutScore_pseudotime$pseudotime_scale
## t = 11.598, df = 5007, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.1346625 0.1886032
## sample estimates:
## cor
## 0.1617537
cor.test(res_index$scMutScore, res_index$sumMut)
##
## Pearson's product-moment correlation
##
## data: res_index$scMutScore and res_index$sumMut
## t = 57.076, df = 6915, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.5496565 0.5817021
## sample estimates:
## cor
## 0.565893

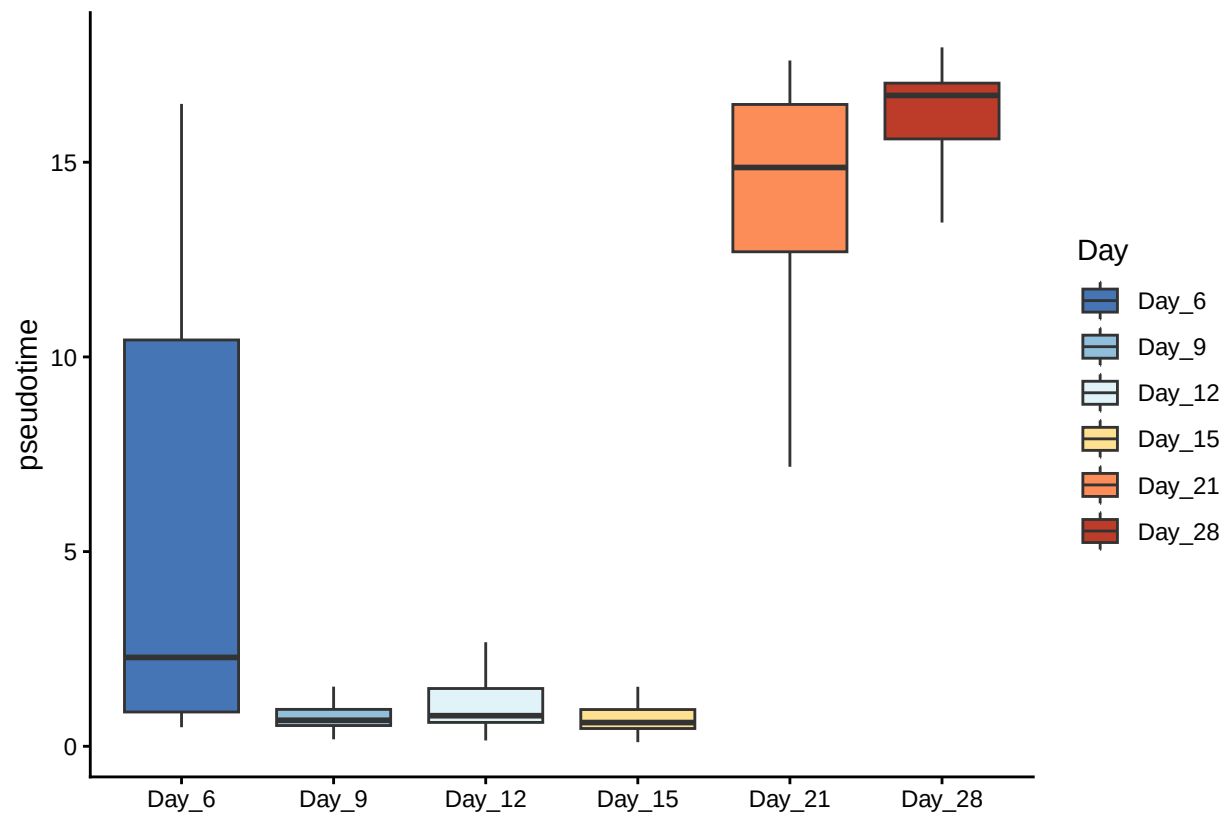
```

13 compareClonesScore figures

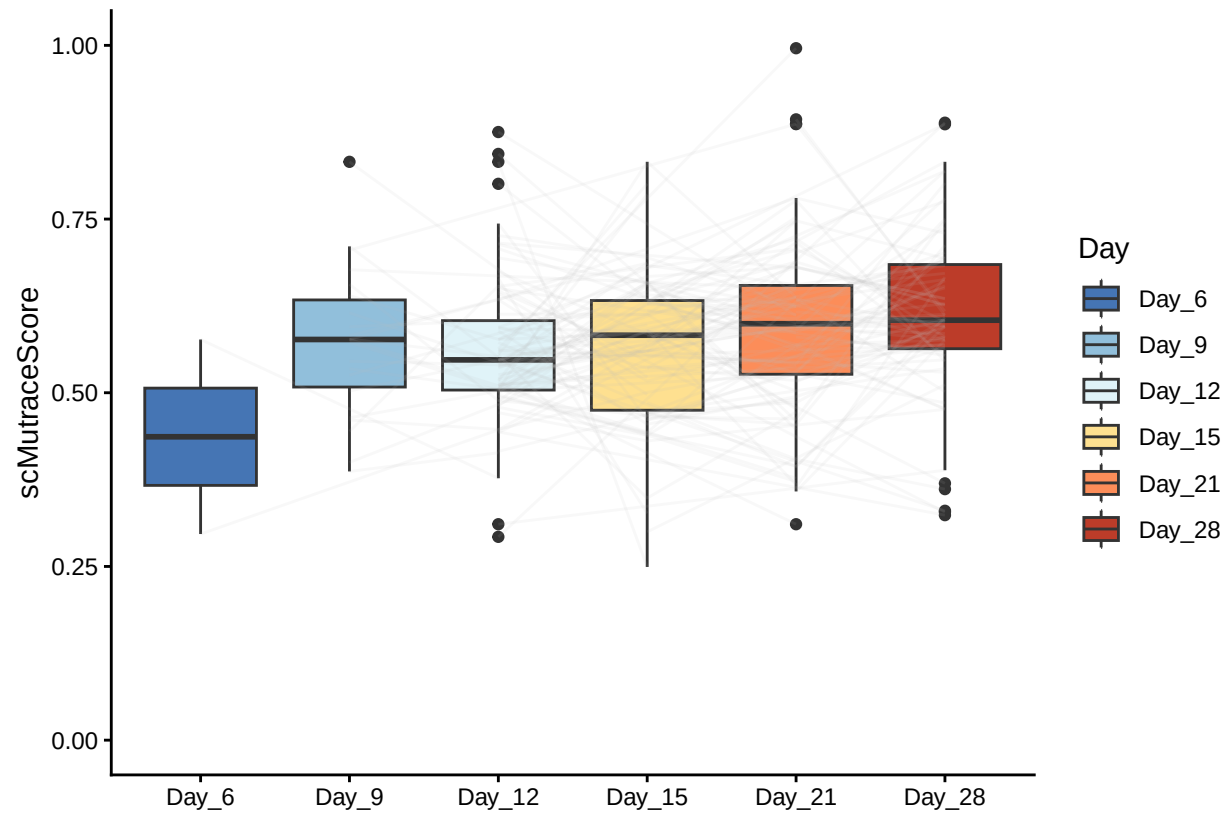
p_cell_scMutraceScore



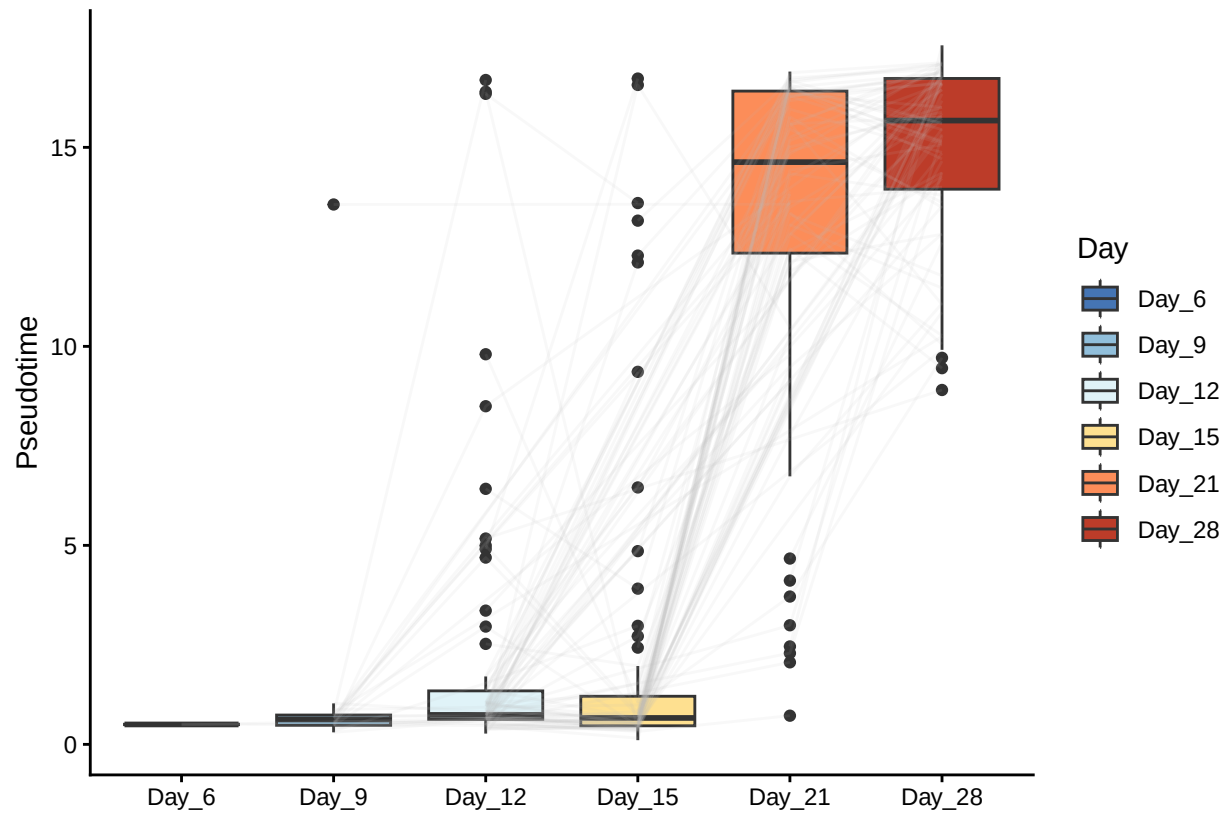
p_cell_pseudotime



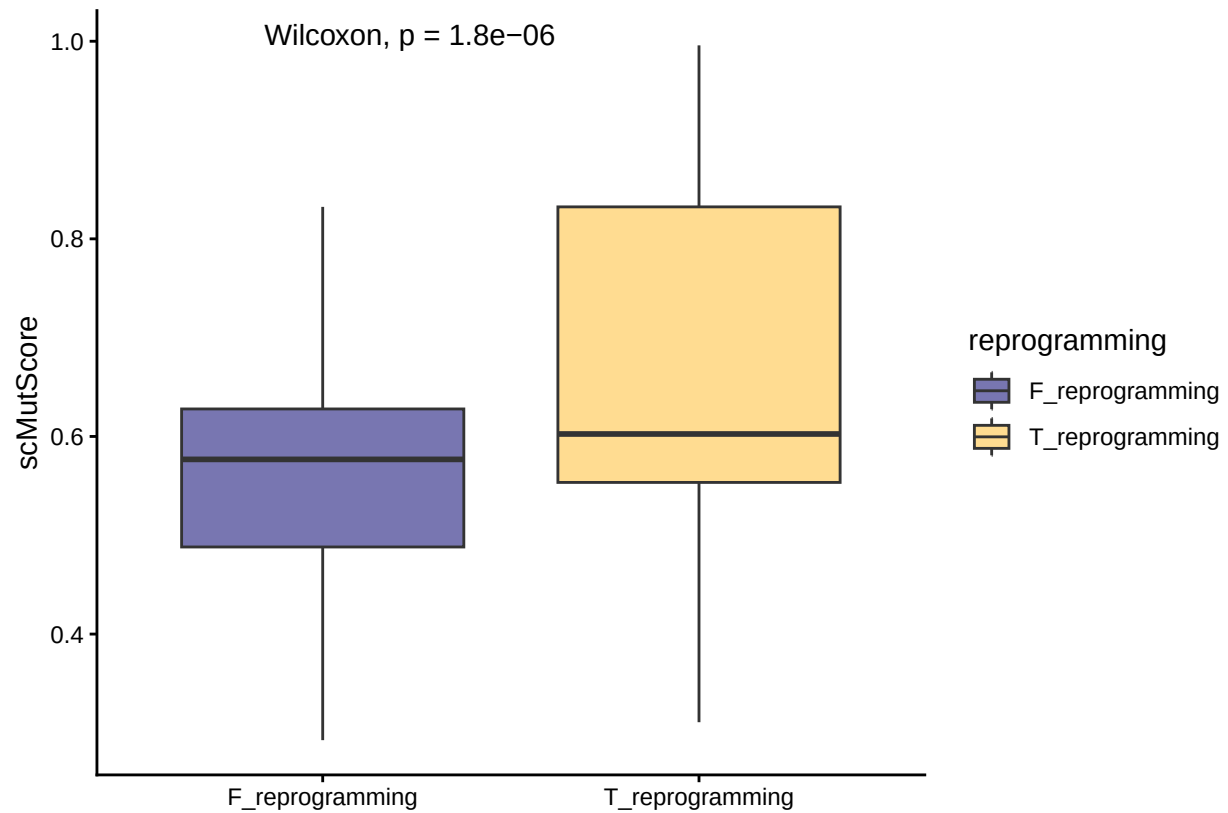
p_clone_scMutraceScore



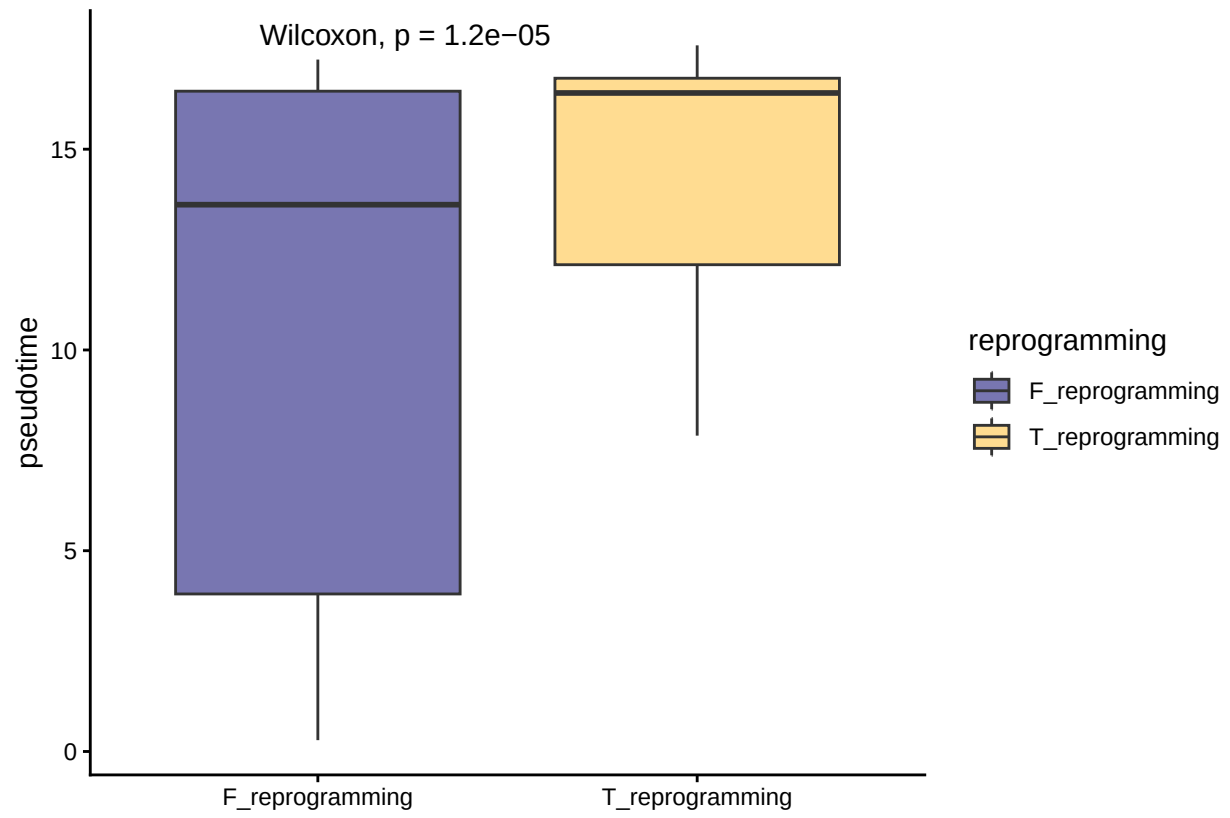
p_clone_pseudotime



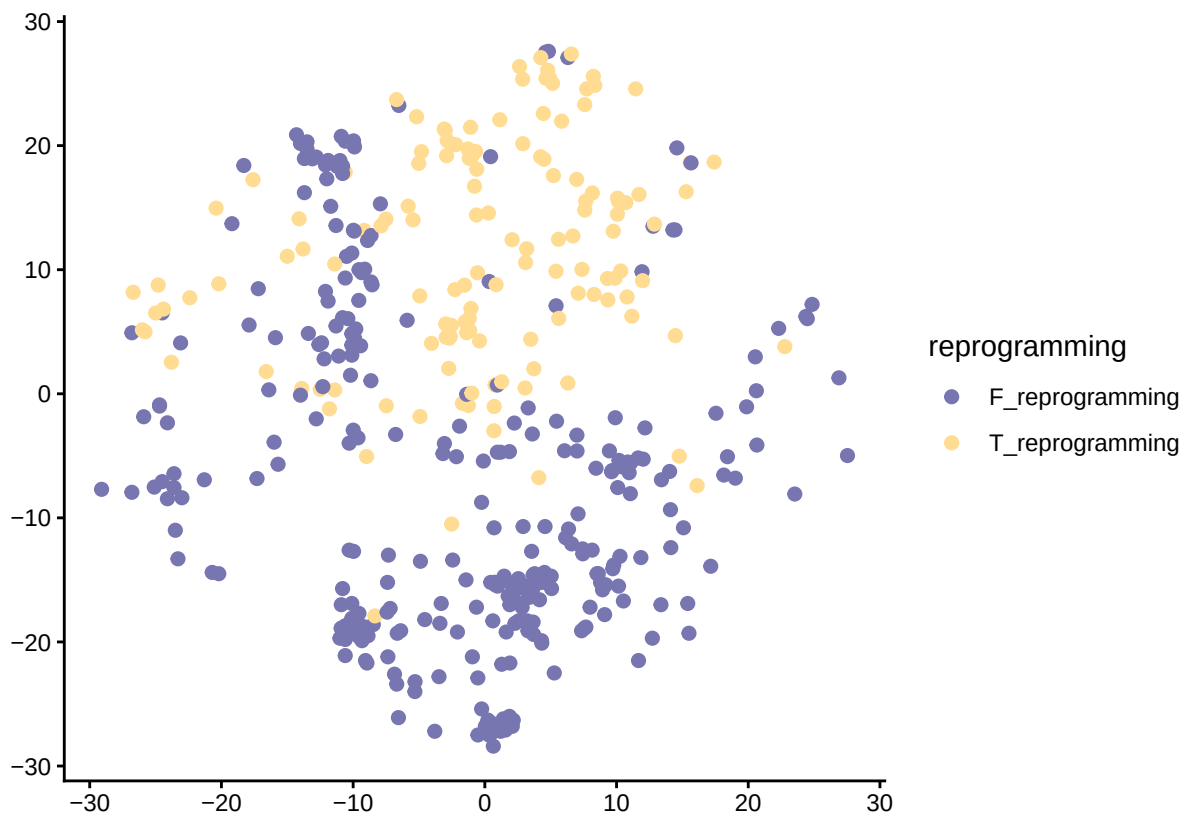
p_reprogramming_scMutraceScore



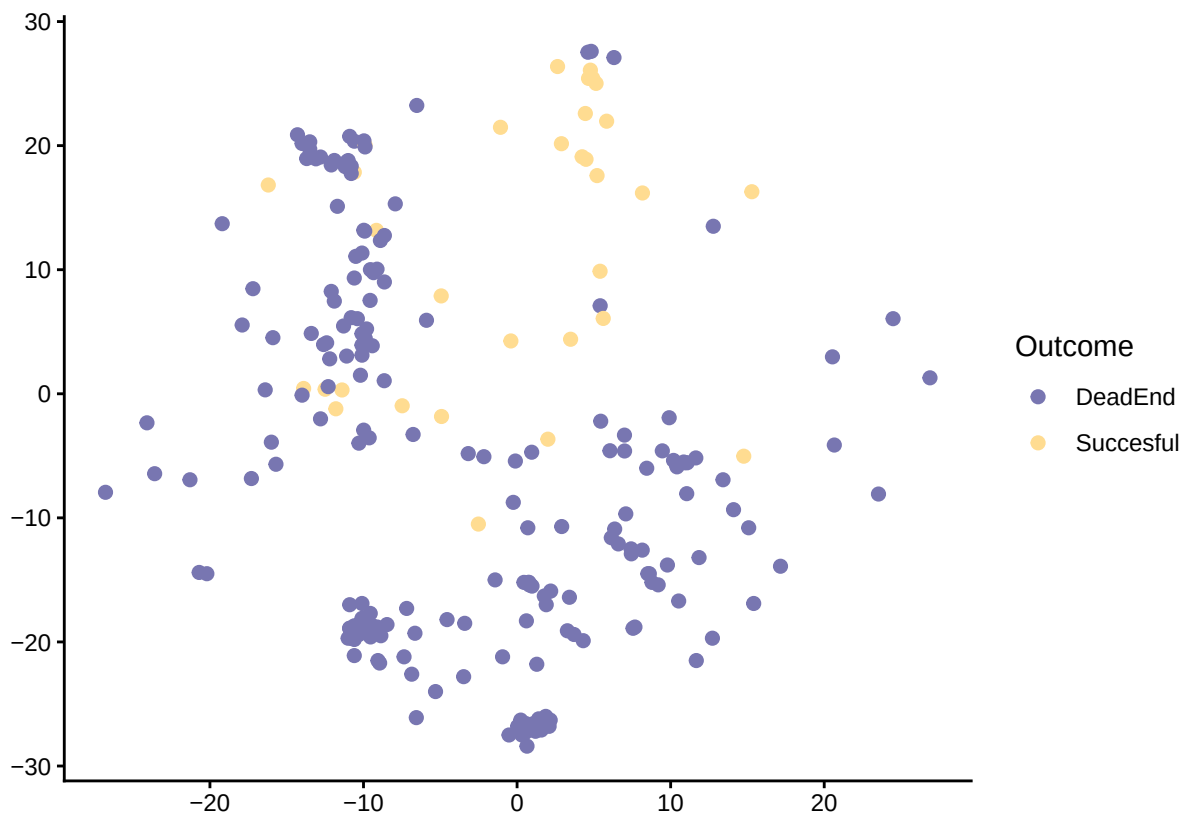
```
p_reprogramming_pseudotime
## Warning in FUN(X[[i]], ...): NAs introduced by coercion
## Warning in FUN(X[[i]], ...): NAs introduced by coercion
## Warning: Removed 135 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
## Warning: Removed 135 rows containing non-finite outside the scale range
## (`stat_compare_means()`).
```



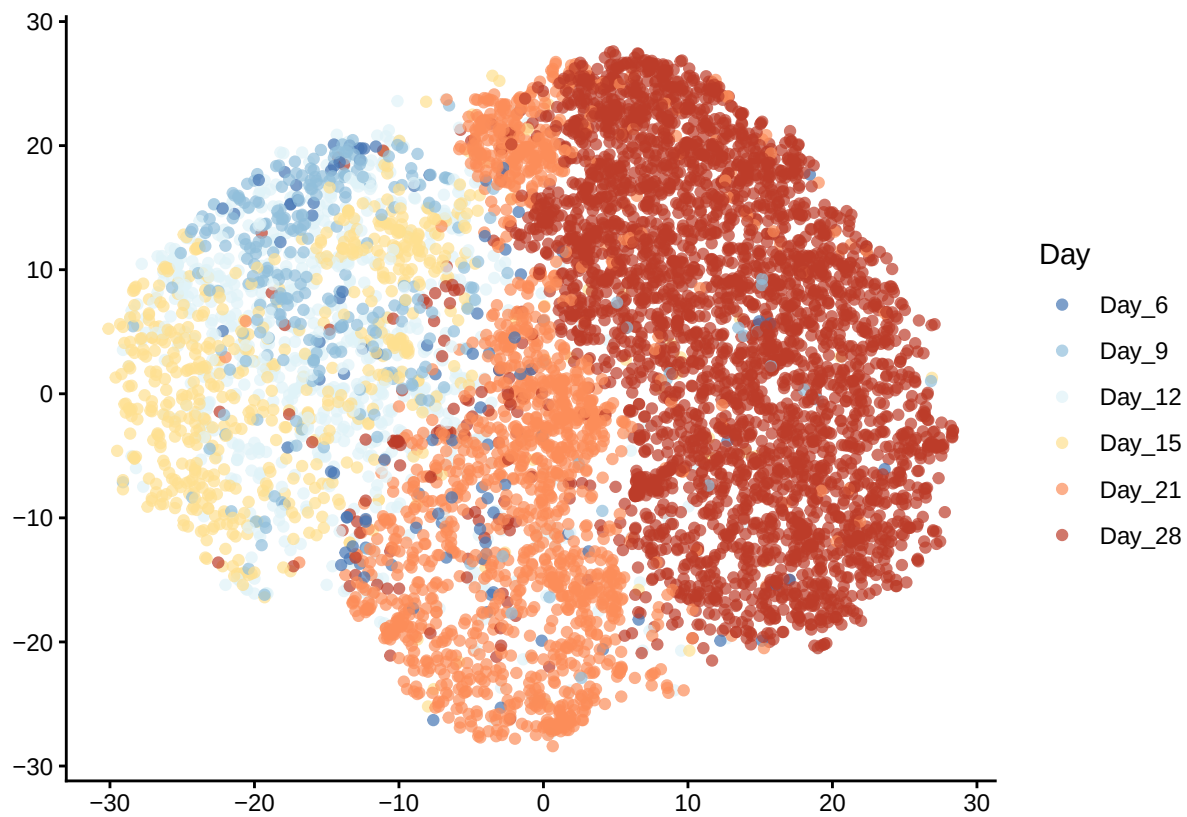
p_umap_reprogramming



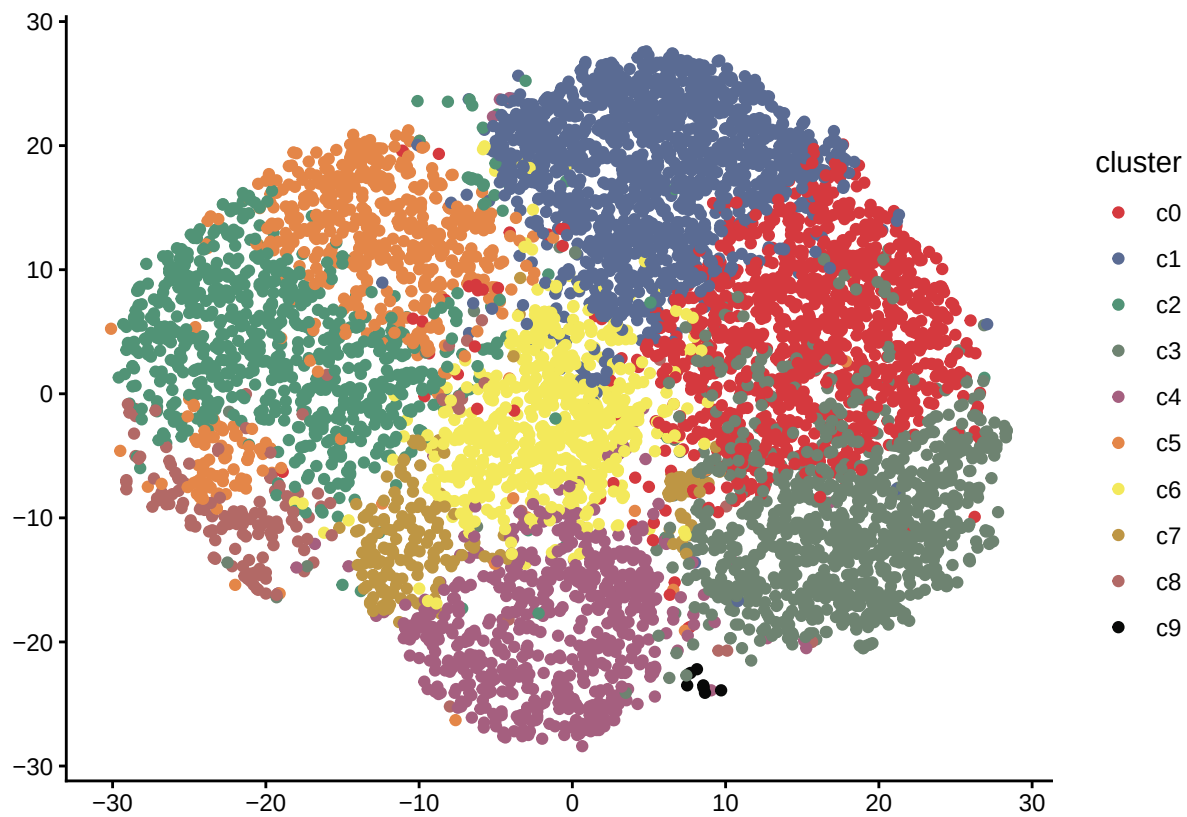
`p_umap_outcome`



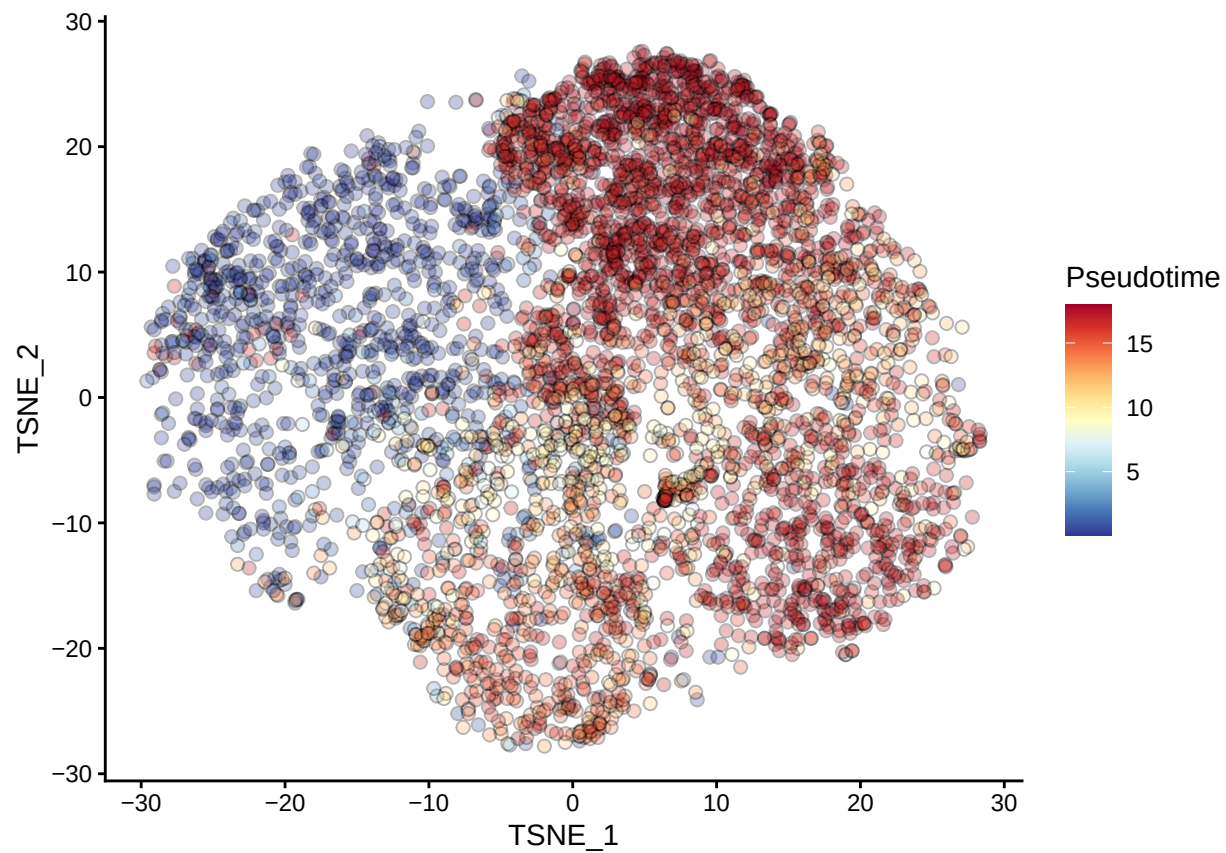
p_umap_cell_day



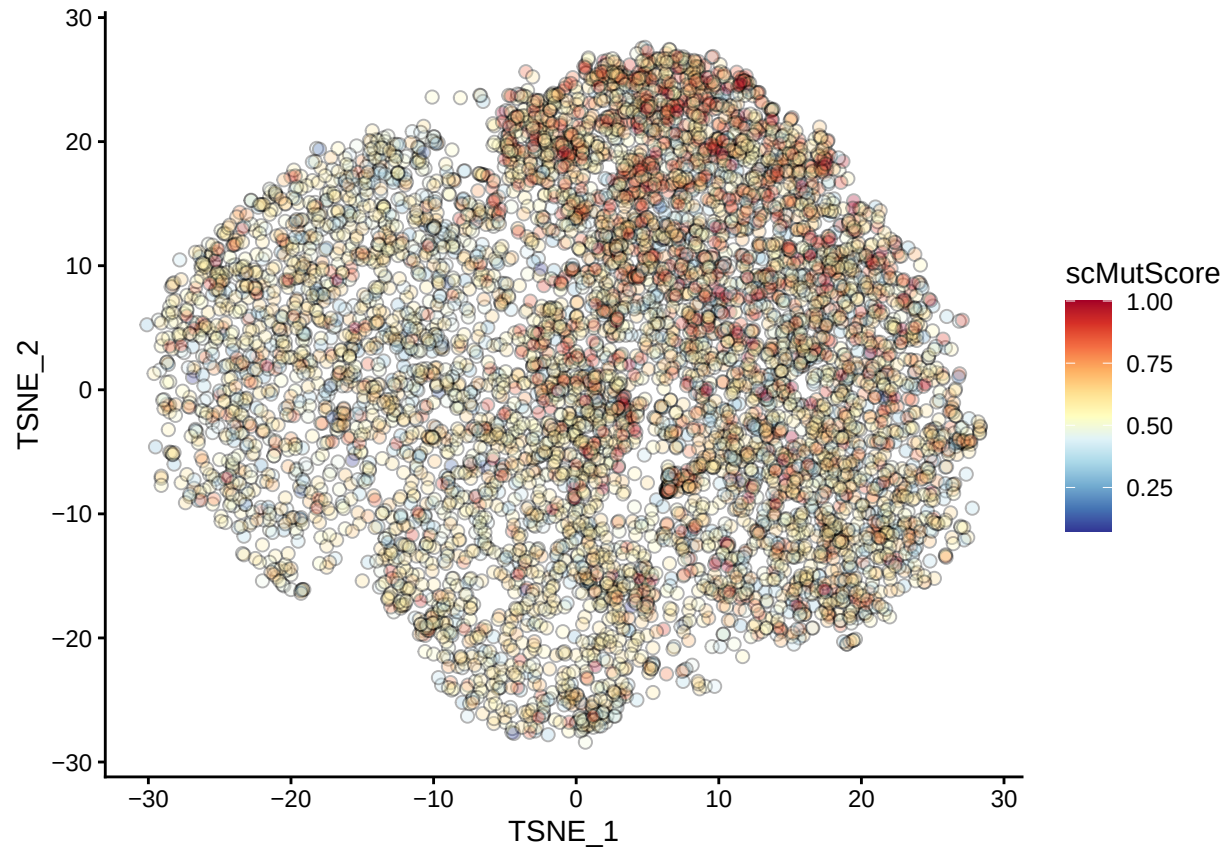
p_umap_cell_cluster



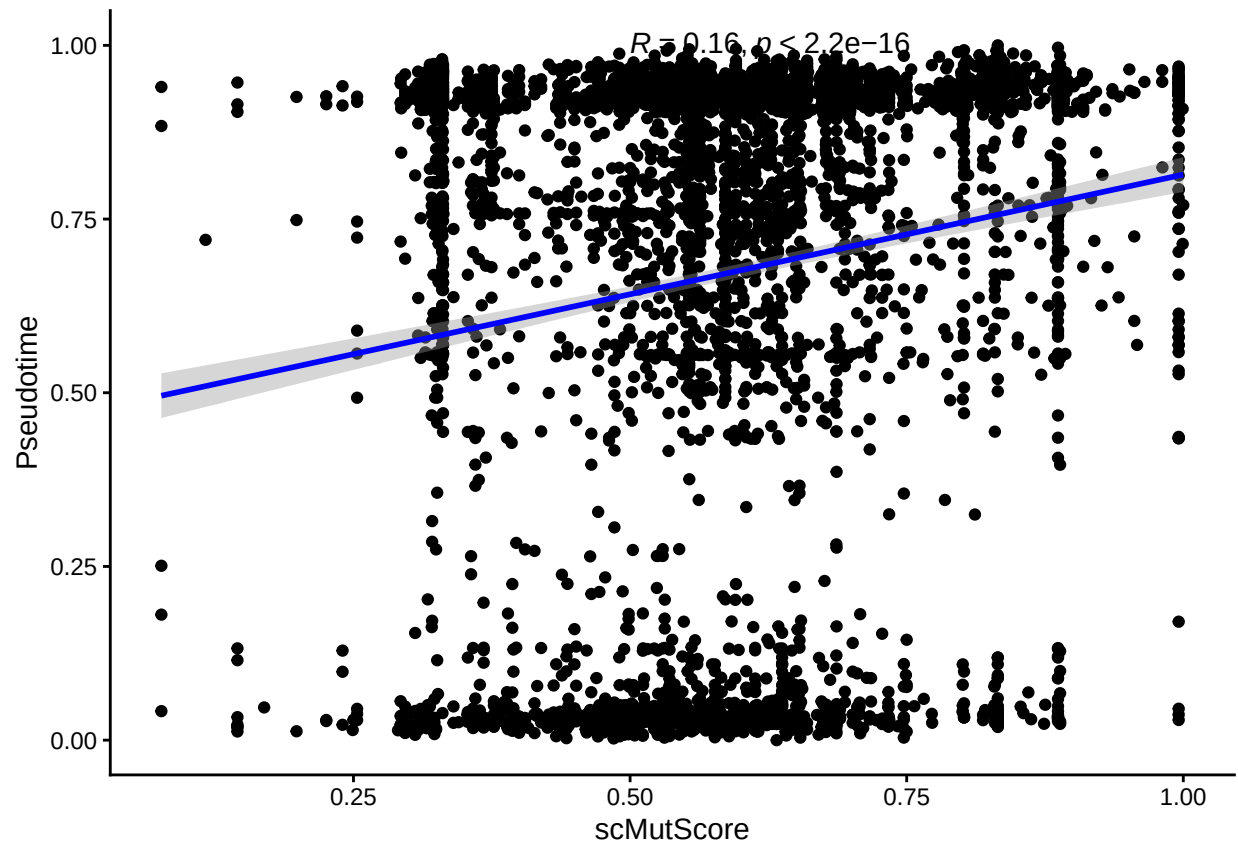
p_umap_cell_Pseudotime



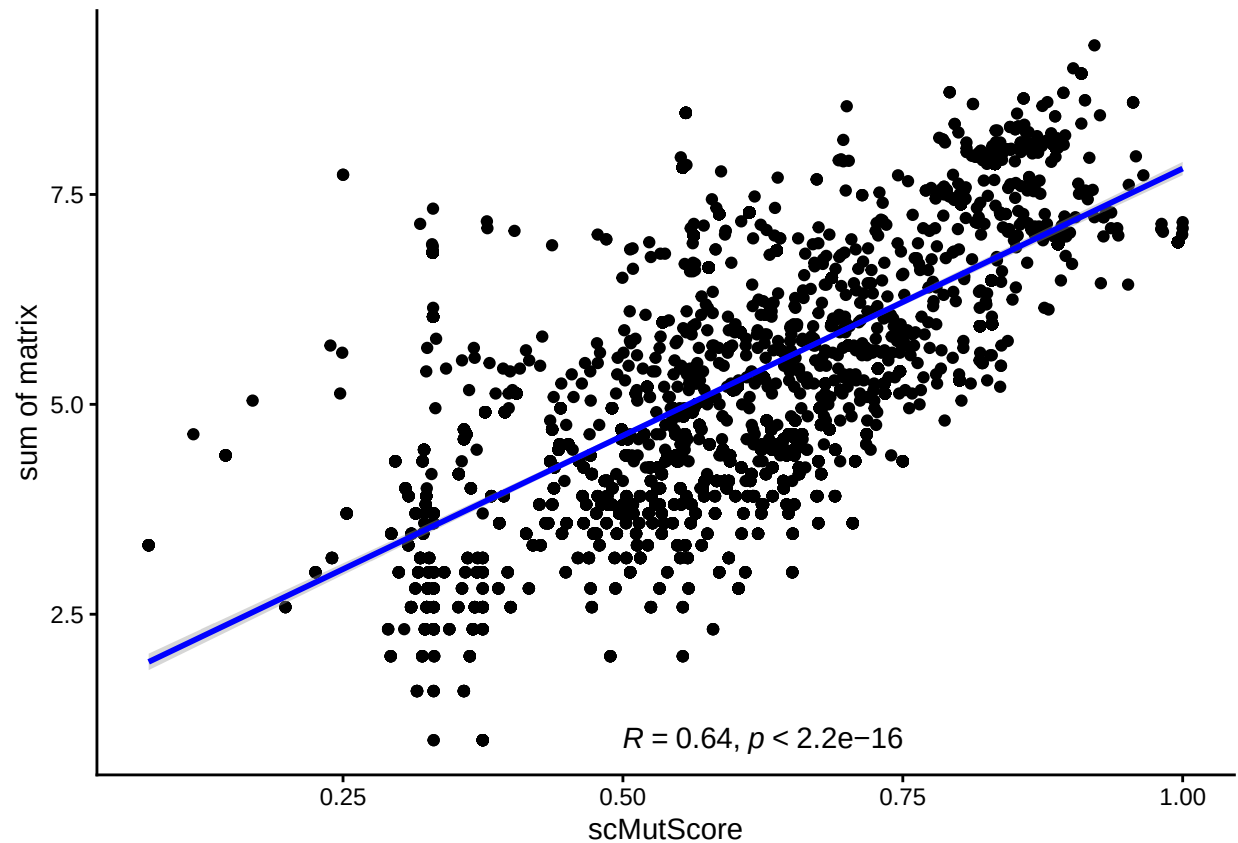
p_umap_cell_scMutScore



```
p_cor_scMutScore_pseudotime  
## `geom_smooth()` using formula = 'y ~ x'
```



```
p_cor_scMutScore_sumMut  
## `geom_smooth()` using formula = 'y ~ x'
```



14 save figures

```
# save data
pdf("../Figure/MEF/p_cell_scMutraceScore.pdf", width = 5.36, height = 3.42)
p_cell_scMutraceScore
dev.off()

pdf("../Figure/MEF/p_cell_pseudotime.pdf", width = 5.36, height = 3.42)
p_cell_pseudotime
dev.off()

pdf("../Figure/MEF/p_clone_scMutraceScore.pdf", width = 5.36, height = 3.42)
p_clone_scMutraceScore
dev.off()

pdf("../Figure/MEF/p_clone_pseudotime.pdf", width = 5.36, height = 3.42)
p_clone_pseudotime
dev.off()

pdf("../Figure/MEF/p_umap_cell_day.pdf", width = 5.36, height = 3.42)
p_umap_cell_day
dev.off()
```

```
pdf("../Figure/MEF/p_umap_cell_scMutScore.pdf", width = 5.36, height = 3.42)
p_umap_cell_scMutScore
dev.off()

pdf("../Figure/MEF/p_umap_cell_Pseudotime.pdf", width = 5.36, height = 3.42)
p_umap_cell_Pseudotime
dev.off()

pdf("../Figure/MEF/p_success_faile_scMutraceScore.pdf", width = 5.36, height = 3.42)
p_reprogramming_scMutraceScore
dev.off()
```

15 check randomly clone scMutScore

```
selected_Days <- c("Day_6", "Day_9", "Day_12", "Day_15", "Day_21", "Day_28")
set.seed(2025)
data_random_cells <- res_index_scMut_meta[sample(nrow(res_index_scMut_meta), 1000, replace = TRUE), c("scMutScore", "Day")]
data_random_cells$Day <- "Day_6"

for (i in 2:length(selected_Days)){
  random_rows_1 <- res_index_scMut_meta[sample(nrow(res_index_scMut_meta), 50), c("scMutScore", "Day")]
  random_rows_1$Day <- selected_Days[i]
  data_random_cells <- rbind(data_random_cells, random_rows_1)
}

data_random_cells$Day <- factor(data_random_cells$Day, levels = c("Day_6", "Day_9", "Day_12", "Day_15", "Day_21", "Day_28"))

p_random_cell_days <- ggplot(data = data_random_cells, mapping = aes(x = Day, y = scMutScore, fill = Day)) +
  geom_boxplot(outliers = FALSE) +
  scale_fill_manual(values = color_settings_2) +
  ylim(0,1) +
  labs(x = "", y = "scMutScore") +
  theme_classic()

data_random_clones <- data_clone_score_clonetagDay[sample(nrow(data_clone_score_clonetagDay), 400, replace = TRUE), c("Mean_Day_score", "Day")]
data_random_clones$Day <- "Day_6"

for (i in 2:length(selected_Days)){
  random_rows_1 <- data_clone_score_clonetagDay[sample(nrow(data_clone_score_clonetagDay), 50), c("Mean_Day_score", "Day")]
  random_rows_1$Day <- selected_Days[i]
  data_random_clones <- rbind(data_random_clones, random_rows_1)
}

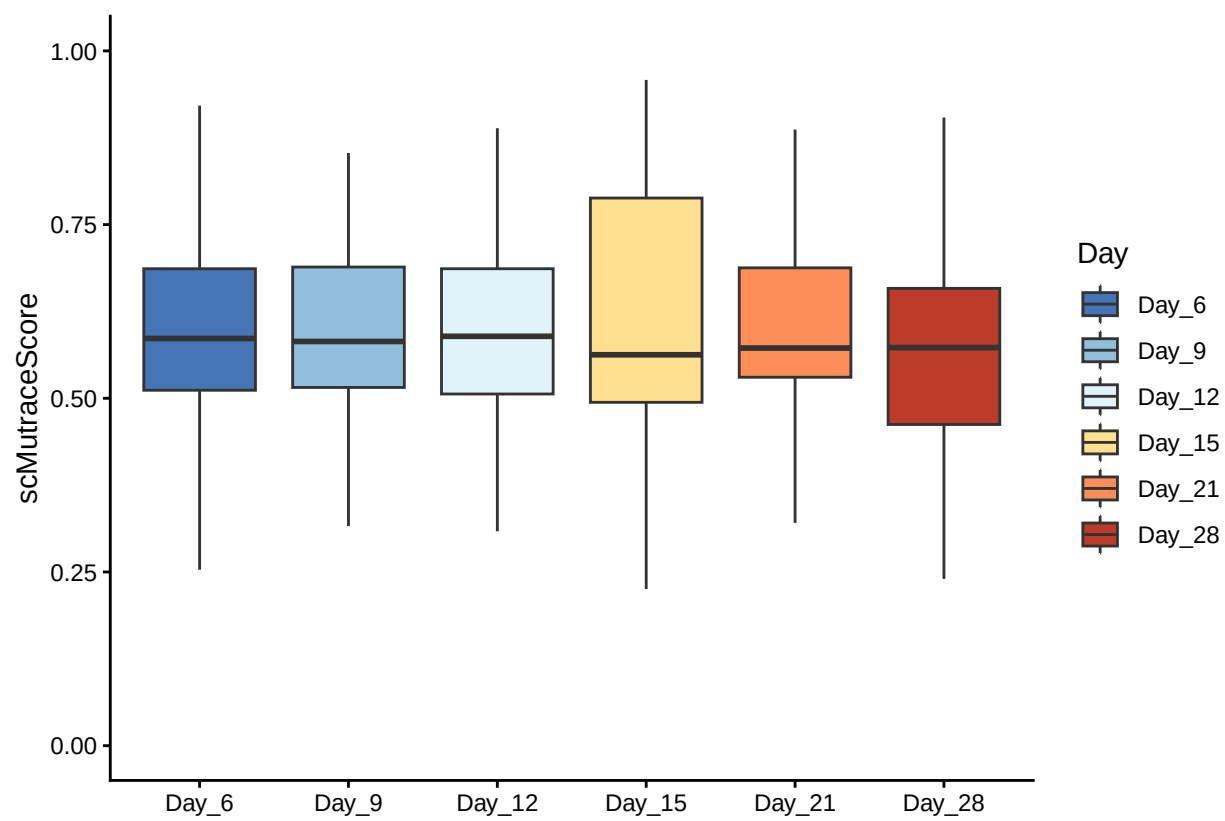
data_random_clones$Day <- factor(data_random_clones$Day, levels = c("Day_6", "Day_9", "Day_12", "Day_15", "Day_21", "Day_28"))

p_random_clone_days <- ggplot(data_random_clones, mapping = aes(x = as.factor(Day), y = Mean_Day_score, group = ID)) +
  geom_boxplot(outliers = TRUE) +
  geom_line(aes(group = ID), color = "gray", alpha = 0.1) +
  scale_fill_manual(values = color_settings_2) +
  ylim(0,1) +
  labs(x = "", y = "scMutScore") +
  theme_classic()
```

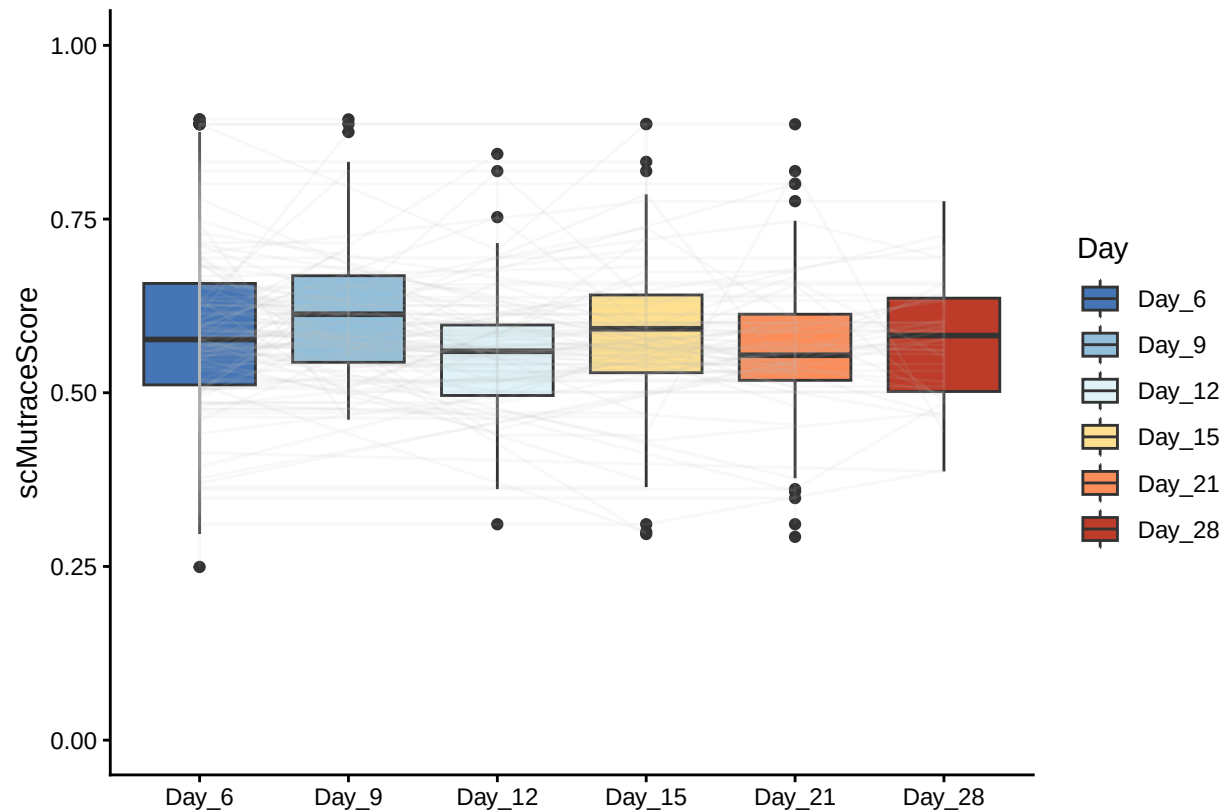


```
theme_classic()
```

```
p_random_cell_days
```



```
p_random_clone_days
```



```
library(clinfun)
# cells scMutscore
res_index_selected <- res_index_scMut_meta[res_index_scMut_meta$Day %in% c('Day_6', 'Day_9', 'Day_12',
res_index_selected$Day <- factor(res_index_selected$Day , levels = c('Day_6', 'Day_9', 'Day_12', 'Day_15',
jonckheere.test(x = res_index_selected$scMutScore, g = res_index_selected$Day, alternative = "increasing",
## Warning in jonckheere.test(x = res_index_selected$scMutScore, g = res_index_selected$Day, : Sample size > 100 or data
## p-value based on normal approximation. Specify nperm for permutation p-value
##
## Jonckheere-Terpstra test
##
## data:
## JT = 9075317, p-value < 2.2e-16
## alternative hypothesis: increasing
# JT = 9075317, p-value < 2.2e-16
# alternative hypothesis: increasing

res_index_selected <- res_index_scMut_meta[res_index_scMut_meta$Day %in% c('Day_6', 'Day_9', 'Day_12',
res_index_selected <- res_index_scMut_meta[res_index_scMut_meta$pseudotime != "NotAva",]
res_index_selected$Day <- factor(res_index_selected$Day , levels = c('Day_6', 'Day_9', 'Day_12', 'Day_15',
jonckheere.test(x = as.numeric(res_index_selected$pseudotime), g = res_index_selected$Day, alternative = "increasing",
## Warning in jonckheere.test(x = as.numeric(res_index_selected$pseudotime), : Sample size > 100 or data
## p-value based on normal approximation. Specify nperm for permutation p-value
##
## Jonckheere-Terpstra test
##
```

```

## data:
## JT = 6857494, p-value < 2.2e-16
## alternative hypothesis: increasing
#JT = 6857494, p-value < 2.2e-16
#alternative hypothesis: increasing

# clone scMutscore
data_clone_score_clonetagDay_selected <- data_clone_score_clonetagDay[data_clone_score_clonetagDay$Day %in% "Day_6", ]
data_clone_score_clonetagDay_selected$Day <- factor(data_clone_score_clonetagDay_selected$Day , levels = c("Day_6", "Day_9", "Day_12", "Day_15"))
jonckheere.test(x = data_clone_score_clonetagDay_selected$Mean_Day_score, g = data_clone_score_clonetagDay_selected$Day, alternative = "increasing")
## Warning in jonckheere.test(x = data_clone_score_clonetagDay_selected$Mean_Day_score, : Sample size > 1000
## p-value based on normal approximation. Specify nperm for permutation p-value
##
## Jonckheere-Terpstra test
##
## data:
## JT = 26956, p-value = 0.000107
## alternative hypothesis: increasing
#JT = 26956, p-value = 0.000107
#alternative hypothesis: increasing
jonckheere.test(x = as.numeric(data_clone_score_clonetagDay_selected$Mean_Day_pseudotime), g = data_clone_score_clonetagDay_selected$Day, alternative = "increasing")
## Warning in jonckheere.test(x = as.numeric(data_clone_score_clonetagDay_selected$Mean_Day_pseudotime), : Sample size > 1000
## p-value based on normal approximation. Specify nperm for permutation p-value
##
## Jonckheere-Terpstra test
##
## data:
## JT = 37857, p-value < 2.2e-16
## alternative hypothesis: increasing
#JT = 37857, p-value < 2.2e-16
#alternative hypothesis: increasing

# data_random_cells
data_random_cells$Day <- factor(data_random_cells$Day , levels = c("Day_6", "Day_9", "Day_12", "Day_15"))
jonckheere.test(x = data_random_cells$scMutScore, g = data_random_cells$Day, alternative = "increasing")
## Warning in jonckheere.test(x = data_random_cells$scMutScore, g = data_random_cells$Day, : Sample size > 1000
## p-value based on normal approximation. Specify nperm for permutation p-value
##
## Jonckheere-Terpstra test
##
## data:
## JT = 135485, p-value = 0.6523
## alternative hypothesis: increasing
#JT = 135420, p-value = 0.657
#alternative hypothesis: increasing

#data_random_clones
data_random_clones$Day <- factor(data_random_clones$Day , levels = c("Day_6", "Day_9", "Day_12", "Day_15"))
jonckheere.test(x = data_random_clones$Mean_Day_score, g = data_random_clones$Day, alternative = "increasing")
## Warning in jonckheere.test(x = data_random_clones$Mean_Day_score, g = data_random_clones$Day, : Sample size > 1000
## p-value based on normal approximation. Specify nperm for permutation p-value
##
## Jonckheere-Terpstra test

```

```
##
## data:
## JT = 60830, p-value = 0.7551
## alternative hypothesis: increasing
#JT = 60830, p-value = 0.7551
#alternative hypothesis: increasing
```

16 tracing allClones

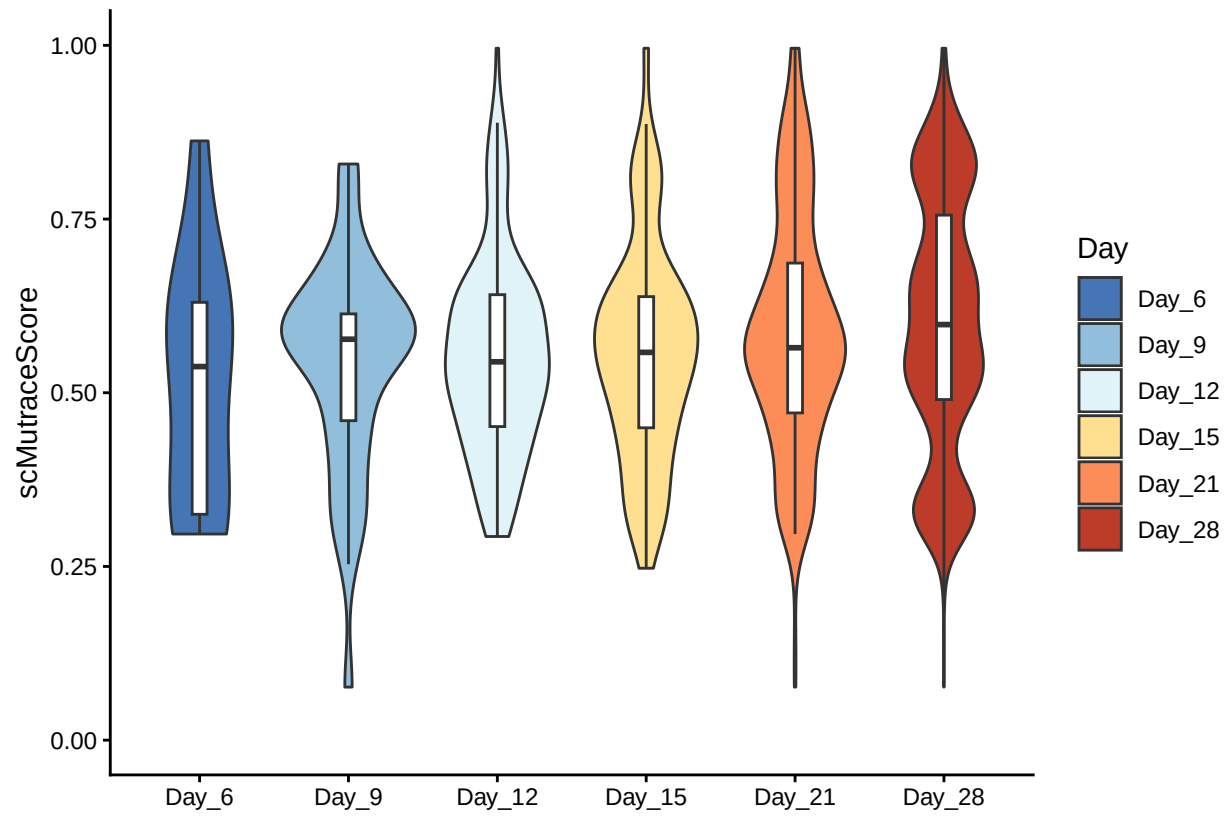
```
res_index_scMut_meta_MEF <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k != "NotAva" & res_i
res_index_scMut_meta_D3 <- res_index_scMut_meta[res_index_scMut_meta$CellTagD3_48k != "NotAva" & res_i
res_index_scMut_meta_D13 <- res_index_scMut_meta[res_index_scMut_meta$CellTagD13_48k != "NotAva",]

p_cell_MEF <- ggplot(data = res_index_scMut_meta_MEF, mapping = aes(x = Day, y = scMutScore, fill = Day))
  geom_violin(trim = TRUE) + coord_cartesian(ylim = c(0, 1)) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = "scMuttraceScore") +
  theme_classic()

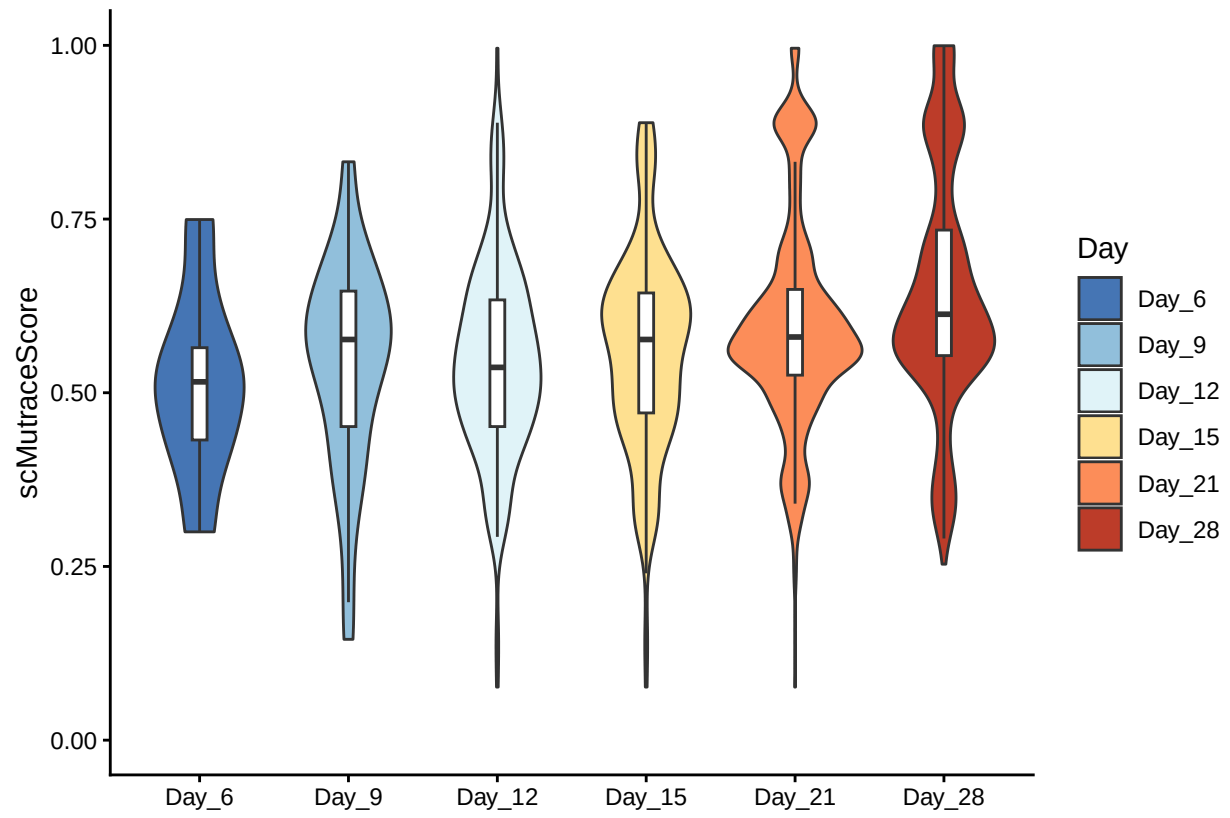
p_cell_D3 <- ggplot(data = res_index_scMut_meta_D3, mapping = aes(x = Day, y = scMutScore, fill = Day))
  geom_violin(trim = TRUE) + coord_cartesian(ylim = c(0, 1)) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = "scMuttraceScore") +
  theme_classic()

p_cell_D13 <- ggplot(data = res_index_scMut_meta_D13, mapping = aes(x = Day, y = scMutScore, fill = Day))
  geom_violin(trim = TRUE) + coord_cartesian(ylim = c(0, 1)) +
  geom_boxplot(width = 0.1, fill = "white", outliers = FALSE) +
  #stat_summary(fun.data="mean_sdl", fun.args = list(mult=1),geom="pointrange", color = "black") +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = "scMuttraceScore") +
  theme_classic()

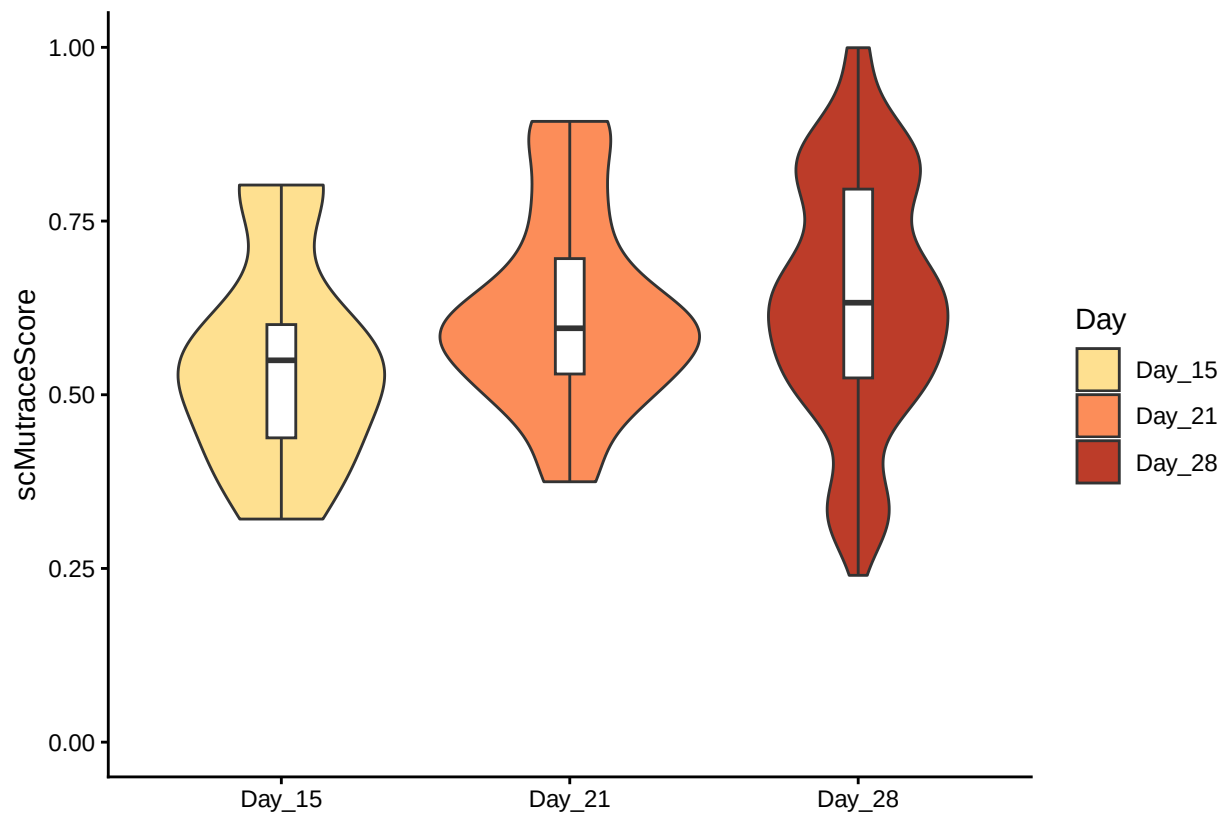
p_cell_MEF
```



p_cell1_D3



p_cell_D13



17 randomlySelectMut

```
color_settings <- c(d0 = "#FFDC91FF", d3 = "#7876B1FF", d6 = "#4575b4",
  d9 = "#91bfdb", d12 = "#e0f3f8", d15 = "#fee090",
  d21 = "#fc8d59", d28 = "#BC3C29FF")

color_settings_2 <- c(Day_0 = "#FFDC91FF", Day_3 = "#7876B1FF", Day_6 = "#4575b4",
  Day_9 = "#91bfdb", Day_12 = "#e0f3f8", Day_15 = "#fee090",
  Day_21 = "#fc8d59", Day_28 = "#BC3C29FF")

col_names_days <- c("Day_6", "Day_9", "Day_12", "Day_15", "Day_21", "Day_28", "sampleSize")
reps_random <- 100
row_names_reps <- paste("iter", c(1:reps_random), sep = "_")

sample_num <- c(10, 30, 50, 100, 150)

data_random_muts2cell_hf1 <- as.data.frame(matrix(nrow = reps_random * length(sample_num), ncol = length(sample_num)))
data_random_muts2cell_hf2 <- as.data.frame(matrix(nrow = reps_random * length(sample_num), ncol = length(sample_num)))

colnames(data_random_muts2cell_hf1) <- col_names_days
colnames(data_random_muts2cell_hf2) <- col_names_days

#rownames(data_random_muts2cell_hf1) <- row_names_reps
```

```

#rownames(data_random_muts2cell_hf2) <- row_names_reps

hf1_muts <- data_CellPerMut_hf1$Location
hf2_muts <- data_CellPerMut_hf2$Location

set.seed(2025)
line_count <- 0
for (i in 1:reps_random){

  for (j in sample_num){

    line_count <- line_count + 1
    hf1_muts_tmp <- sample(hf1_muts, j, replace = TRUE)
    hf2_muts_tmp <- sample(hf2_muts, j, replace = TRUE)

    hf1_cells <- retrieveCellsByLocation(MEF_CSM_hf1, hf1_muts_tmp)$Mut
    hf2_cells <- retrieveCellsByLocation(MEF_CSM_hf2, hf2_muts_tmp)$Mut

    data_random_muts2cell_hf1[line_count, "Day_6"] <- sum(str_count(hf1_cells, "hf1.d6"))
    data_random_muts2cell_hf1[line_count, "Day_9"] <- sum(str_count(hf1_cells, "hf1.d9"))
    data_random_muts2cell_hf1[line_count, "Day_12"] <- sum(str_count(hf1_cells, "hf1.d12"))
    data_random_muts2cell_hf1[line_count, "Day_15"] <- sum(str_count(hf1_cells, "hf1.d15"))
    data_random_muts2cell_hf1[line_count, "Day_21"] <- sum(str_count(hf1_cells, "hf1.d21"))
    data_random_muts2cell_hf1[line_count, "Day_28"] <- sum(str_count(hf1_cells, "hf1.d28"))
    data_random_muts2cell_hf1[line_count, "sampleSize"] <- j

    data_random_muts2cell_hf2[line_count, "Day_6"] <- sum(str_count(hf2_cells, "hf2.d6"))
    data_random_muts2cell_hf2[line_count, "Day_9"] <- sum(str_count(hf2_cells, "hf2.d9"))
    data_random_muts2cell_hf2[line_count, "Day_12"] <- sum(str_count(hf2_cells, "hf2.d12"))
    data_random_muts2cell_hf2[line_count, "Day_15"] <- sum(str_count(hf2_cells, "hf2.d15"))
    data_random_muts2cell_hf2[line_count, "Day_21"] <- sum(str_count(hf2_cells, "hf2.d21"))
    data_random_muts2cell_hf2[line_count, "Day_28"] <- sum(str_count(hf2_cells, "hf2.d28"))
    data_random_muts2cell_hf2[line_count, "sampleSize"] <- j
  }
}

data_random_muts2cell_hf1$iteration <- rownames(data_random_muts2cell_hf1)
data_random_muts2cell_hf2$iteration <- rownames(data_random_muts2cell_hf2)

data_random_muts2cell_hf1_long <- data_random_muts2cell_hf1 %>%
  pivot_longer(
    cols = starts_with("Day"),
    names_to = "Day",
    values_to = "CellNum"
  )

data_random_muts2cell_hf2_long <- data_random_muts2cell_hf2 %>%
  pivot_longer(
    cols = starts_with("Day"),
    names_to = "Day",
    values_to = "CellNum"
  )

```

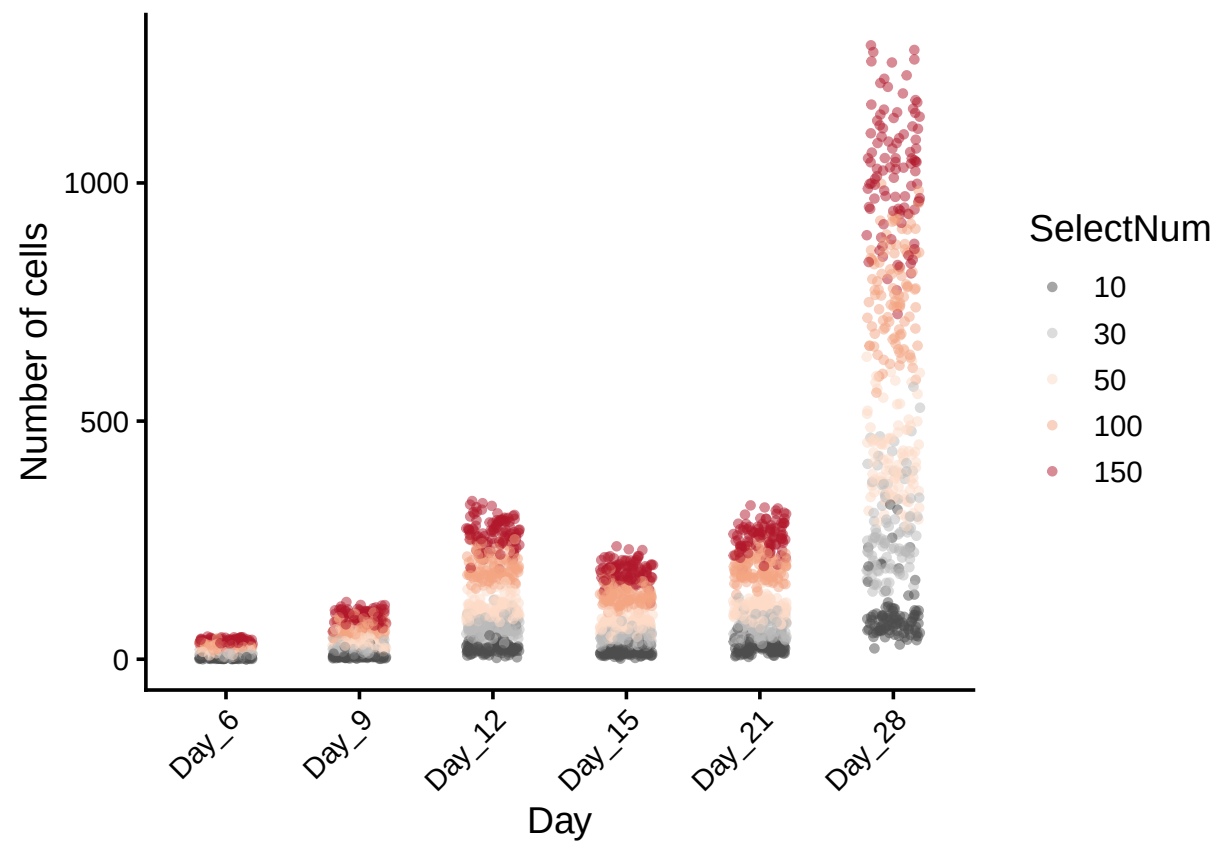


```
data_random_muts2cell_hf1_long$Day <- factor(data_random_muts2cell_hf1_long$Day, levels = c("Day_6", "D
data_random_muts2cell_hf2_long$Day <- factor(data_random_muts2cell_hf2_long$Day, levels = c("Day_6", "D
```

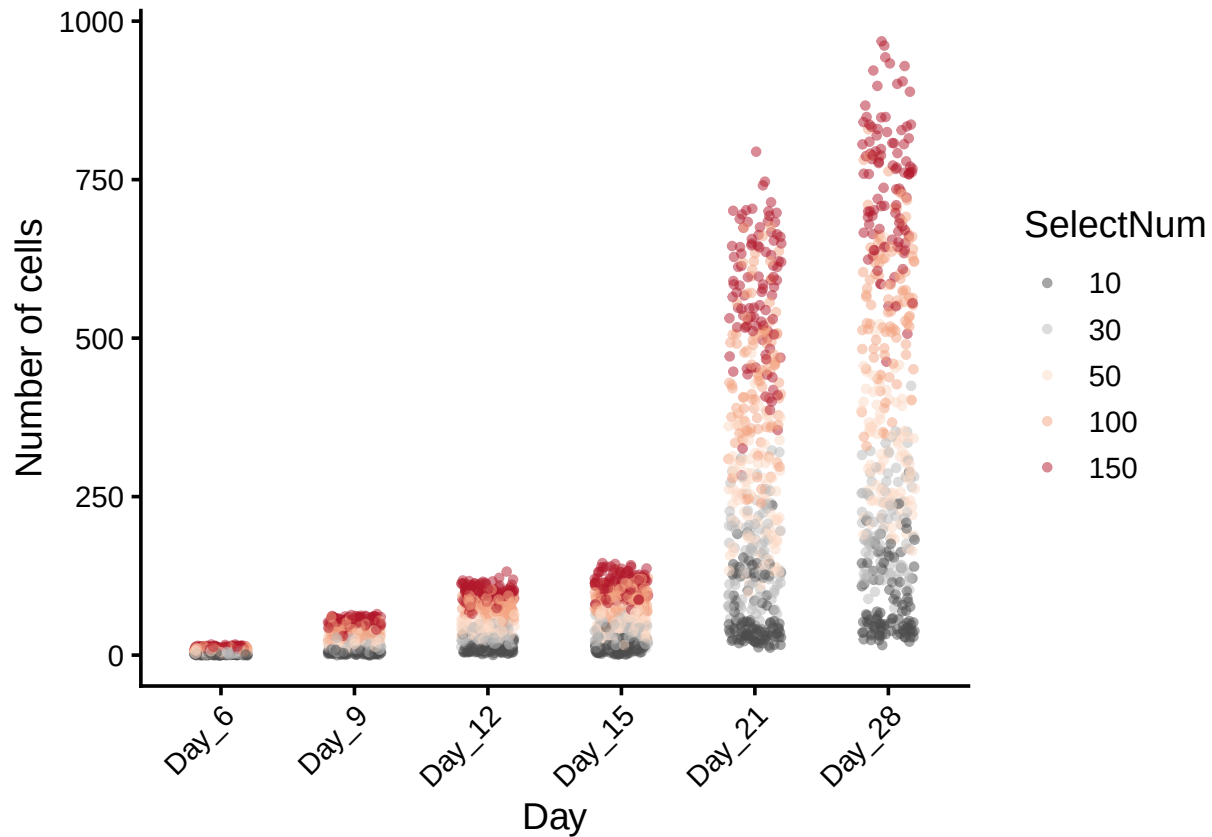
```
p_Mut2Cell_hf1 <- ggplot(data_random_muts2cell_hf1_long,
  aes(x = Day, y = CellNum, color = factor(sampleSize))) +
  geom_jitter(width = 0.2, alpha = 0.5, size = 1) +
  ylab("Number of cells") +
  xlab("Day") +
  labs(color = "SelectNum") +
  scale_color_manual(values = c(
    "10" = "#4d4d4d",
    "30" = "#bababa",
    "50" = "#fddbc7",
    "100" = "#f4a582",
    "150" = "#b2182b"
  )) +
  theme_classic(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
p_Mut2Cell_hf2 <- ggplot(data_random_muts2cell_hf2_long,
  aes(x = Day, y = CellNum, color = factor(sampleSize))) +
  geom_jitter(width = 0.2, alpha = 0.5, size = 1) +
  ylab("Number of cells") +
  xlab("Day") +
  labs(color = "SelectNum") +
  scale_color_manual(values = c(
    "10" = "#4d4d4d",
    "30" = "#bababa",
    "50" = "#fddbc7",
    "100" = "#f4a582",
    "150" = "#b2182b"
  )) +
  theme_classic(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
p_Mut2Cell_hf1
```



p_Mut2Cell1_hf2



18 lineageTracingExample

```
selected_start_cloneID <- 2352
data_clone_score_selected <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k == selected_start_
colnames(data_clone_score_selected) <- c("Cellbarcode", "scMutScore", "D0_tag", "D3_tag", "D13_tag")

# Categorization of all situations
data_clone_score_selected_long <- bind_rows(
  # C1: D3 == NotAva & D13 != NotAva → D0 → D13
  data_clone_score_selected %>%
    dplyr::filter(D3_tag == "NotAva" & D13_tag != "NotAva") %>%
    dplyr::select(source = D0_tag, target = D13_tag),
  # C2: D3 == NotAva & D13 == NotAva → D0 → D0_self
  data_clone_score_selected %>%
    dplyr::filter(D3_tag == "NotAva" & D13_tag == "NotAva") %>%
    dplyr::mutate(target = paste0(D0_tag, "_self")) %>%
    dplyr::select(source = D0_tag, target),
  # C3: D3 != NotAva & D13 != NotAva → D0→D3
  data_clone_score_selected %>%
    dplyr::filter(D3_tag != "NotAva" & D13_tag != "NotAva") %>%
```

```

    dplyr::select(source = D0_tag, target = D3_tag),

# C4: D3 != NotAva & D13 != NotAva → D3→D13
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag != "NotAva") %>%
  dplyr::select(source = D3_tag, target = D13_tag),

# C5: D3 != NotAva & D13 == NotAva → D0→D3
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag == "NotAva") %>%
  dplyr::select(source = D0_tag, target = D3_tag),

# C6: D3 != NotAva & D13 == NotAva → D3→D3_self
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag == "NotAva") %>%
  dplyr::mutate(target = paste0(D3_tag, "_self")) %>%
  dplyr::select(source = D3_tag, target)
)

# Counting the number of pairs
clone_links <- data_clone_score_selected_long %>%
  dplyr::group_by(source, target) %>%
  dplyr::summarise(value = n()) %>%
  dplyr::arrange(desc(value))
## `summarise()` has grouped output by 'source'. You can override using the
## `.groups` argument.

clone_links <- as.data.frame(clone_links)

clone_nodes <- data.frame(
  name=c(as.character(clone_links$source),
    as.character(clone_links$target)) %>% unique()
)

clone_nodes$group <- ""

for (i in 1:nrow(clone_nodes)){
  checkNode <- as.character(clone_nodes[i,"name"])
  if (checkNode %in% as.character(data_clone_score_selected$D0_tag)){
    clone_nodes[i,"group"] <- "MEF"
  }

  if (checkNode %in% as.character(data_clone_score_selected$D3_tag)){
    clone_nodes[i,"group"] <- "D3"
  }

  if (checkNode %in% as.character(data_clone_score_selected$D13_tag)){
    clone_nodes[i,"group"] <- "D13"
  }
}

clone_links$IDsource <- match(clone_links$source, clone_nodes$name)-1
clone_links$IDtarget <- match(clone_links$target, clone_nodes$name)-1

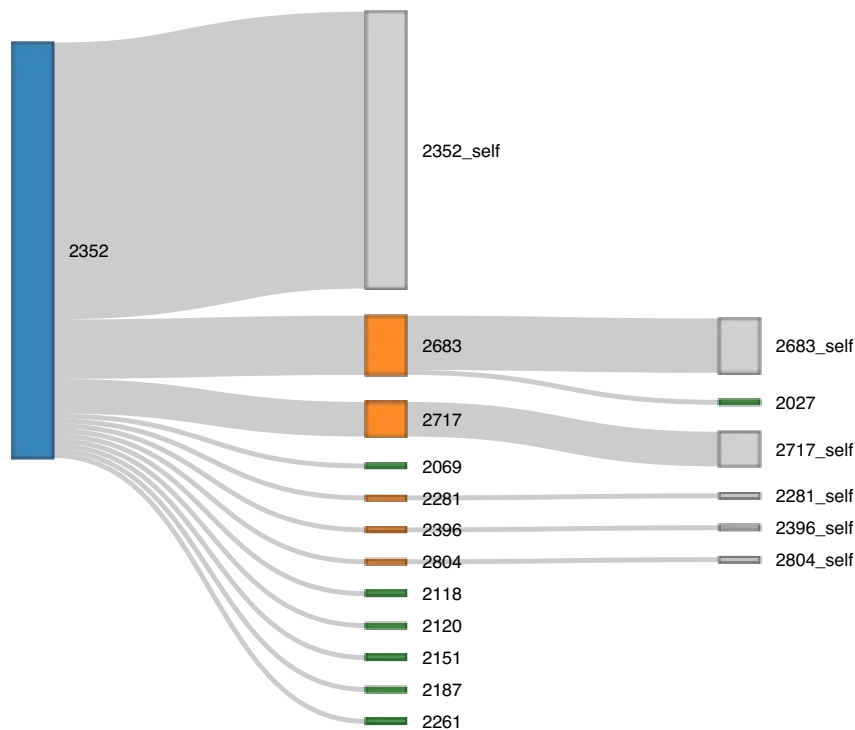
```

```

p_sankeyNetwork <- sankeyNetwork(Links = clone_links, Nodes = clone_nodes,
  Source = "IDsource", Target = "IDtarget",
  Value = "value", NodeID = "name",
  sinksRight=FALSE,
  colourScale = JS("d3.scaleOrdinal()
    .domain(['MEF', 'D3', 'D13'])
    .range(['#1f77b4', '#ff7f0e', '#2ca02c'])"),
  NodeGroup = "group")

p_sankeyNetwork

```



19 lineageTracingExample with scMutScore

```

selected_start_cloneID <- c(124, 487, 2352)

data_clone_score_selected <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k %in% selected_start_cloneID]
colnames(data_clone_score_selected) <- c("Cellbarcode", "scMutScore", "D0_tag", "D3_tag", "D13_tag")

# Categorization of all situations
data_clone_score_selected_long <- bind_rows(
  # C1: D3 == NotAva & D13 != NotAva → D0 → D13

```

```

data_clone_score_selected %>%
  dplyr::filter(D3_tag == "NotAva" & D13_tag != "NotAva") %>%
  dplyr::select(source = D0_tag, target = D13_tag),

# C2: D3 == NotAva & D13 == NotAva → D0 → D0_self
data_clone_score_selected %>%
  dplyr::filter(D3_tag == "NotAva" & D13_tag == "NotAva") %>%
  dplyr::mutate(target = paste0(D0_tag, "_self")) %>%
  dplyr::select(source = D0_tag, target),

# C3: D3 != NotAva & D13 != NotAva → D0→D3
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag != "NotAva") %>%
  dplyr::select(source = D0_tag, target = D3_tag),

# C4: D3 != NotAva & D13 != NotAva → D3→D13
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag != "NotAva") %>%
  dplyr::select(source = D3_tag, target = D13_tag),

# C5: D3 != NotAva & D13 == NotAva → D0→D3
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag == "NotAva") %>%
  dplyr::select(source = D0_tag, target = D3_tag),

# C6: D3 != NotAva & D13 == NotAva → D3→D3_self
data_clone_score_selected %>%
  dplyr::filter(D3_tag != "NotAva" & D13_tag == "NotAva") %>%
  dplyr::mutate(target = paste0(D3_tag, "_self")) %>%
  dplyr::select(source = D3_tag, target)
)

# Counting the number of pairs
clone_links <- data_clone_score_selected_long %>%
  dplyr::group_by(source, target) %>%
  dplyr::summarise(value = n()) %>%
  dplyr::arrange(desc(value))
## `summarise()` has grouped output by 'source'. You can override using the
## `.groups` argument.

clone_links <- as.data.frame(clone_links)

clone_links <- clone_links[!grepl("_self", clone_links$target), ]

#clone_links <- clone_links[clone_links$value > 1,]

clone_nodes <- data.frame(
  name=c(as.character(clone_links$source),
  as.character(clone_links$target)) %>% unique()
)

clone_nodes$group <- ""

```

```

for (i in 1:nrow(clone_nodes)){
  checkNode <- as.character(clone_nodes[i,"name"])
  if (checkNode %in% as.character(data_clone_score_selected$D0_tag)){
    clone_nodes[i,"group"] <- "MEF"
  }

  if (checkNode %in% as.character(data_clone_score_selected$D3_tag)){
    clone_nodes[i,"group"] <- "D3"
  }

  if (checkNode %in% as.character(data_clone_score_selected$D13_tag)){
    clone_nodes[i,"group"] <- "D13"
  }
}

# cal node scMutScore
clone_nodes$scMutScore <- 0

data_MEF_D3_D13 <- data.frame()

for (i in 1:nrow(clone_nodes)){
  group_tmp <- clone_nodes[i, "group"]
  node_tmp <- clone_nodes[i, "name"]
  print(node_tmp)
  print(group_tmp)
  if (group_tmp == "MEF"){
    data_mef_tmp <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k %in% selected_start_cloneID &
    if(nrow(data_mef_tmp) == 0){
      data_mef_tmp_tmp <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k == selected_start_cloneID &
      data_mef_tmp_tmp$Tag <- "MEF"
      clone_nodes[i, "scMutScore"] <- min(data_mef_tmp_tmp$scMutScore)
      data_MEF_D3_D13 <- rbind(data_MEF_D3_D13, data_mef_tmp_tmp)
    }else{
      data_mef_tmp$Tag <- "MEF"
      clone_nodes[i, "scMutScore"] <- min(data_mef_tmp$scMutScore)
      data_MEF_D3_D13 <- rbind(data_MEF_D3_D13, data_mef_tmp)
    }
  }
  if (group_tmp == "D3"){
    data_d3_tmp <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k %in% selected_start_cloneID &
    if(nrow(data_d3_tmp) == 0){
      data_d3_tmp_tmp <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k == selected_start_cloneID &
      data_d3_tmp_tmp$Tag <- "D3"
      clone_nodes[i, "scMutScore"] <- min(data_d3_tmp_tmp$scMutScore)
      data_MEF_D3_D13 <- rbind(data_MEF_D3_D13, data_d3_tmp_tmp)
    }else{
      data_d3_tmp$Tag <- "D3"
      clone_nodes[i, "scMutScore"] <- min(data_d3_tmp$scMutScore)
      data_MEF_D3_D13 <- rbind(data_MEF_D3_D13, data_d3_tmp)
    }
  }
  if (group_tmp == "D13"){
    data_d13_tmp <- res_index_scMut_meta[res_index_scMut_meta$CellTagD0_48k %in% selected_start_cloneID &

```

```

data_d13_tmp$Tag <- "D13"
clone_nodes[i, "scMutScore"] <- min(data_d13_tmp$scMutScore)
data_MEF_D3_D13 <- rbind(data_MEF_D3_D13, data_d13_tmp)
}
}
## [1] "487"
## [1] "MEF"
## [1] "124"
## [1] "MEF"
## [1] "2352"
## [1] "MEF"
## [1] "204"
## [1] "D3"
## [1] "240"
## [1] "D3"
## [1] "212"
## [1] "D3"
## [1] "2683"
## [1] "D3"
## [1] "767"
## [1] "D3"
## [1] "488"
## [1] "D3"
## [1] "2717"
## [1] "D3"
## [1] "543"
## [1] "D3"
## [1] "779"
## [1] "D3"
## [1] "83"
## [1] "D13"
## [1] "93"
## [1] "D13"
## [1] "770"
## [1] "D3"
## [1] "784"
## [1] "D3"
## [1] "104"
## [1] "D13"
## [1] "43"
## [1] "D13"
## [1] "47"
## [1] "D13"
## [1] "99"
## [1] "D13"
## [1] "2069"
## [1] "D13"
## [1] "2118"
## [1] "D13"
## [1] "2120"
## [1] "D13"
## [1] "2151"
## [1] "D13"

```



```

## [1] "2187"
## [1] "D13"
## [1] "2261"
## [1] "D13"
## [1] "2281"
## [1] "D3"
## [1] "2396"
## [1] "D3"
## [1] "2804"
## [1] "D3"
## [1] "178"
## [1] "D13"
## [1] "2027"
## [1] "D13"
## [1] "110"
## [1] "D13"
## [1] "522"
## [1] "D3"
## [1] "128"
## [1] "D13"

data_MEF_D3_D13$Tag <- factor(data_MEF_D3_D13$Tag, levels = c("MEF", "D3", "D13"))
clone_nodes$group <- factor(clone_nodes$group, levels = c("MEF", "D3", "D13"))
data_MEF_D3_D13$Day <- factor(data_MEF_D3_D13$Day, levels = c("Day_0", "Day_3", "Day_6", "Day_9", "Day_12"))

network <- graph_from_data_frame(d=clone_links, vertices=clone_nodes, directed=T)

color_settings_network_1 <- c( D13 = "#e41a1c", D3 = "#377eb8", MEF = "#4daf4a")

color_settings_network_2 <- colorRampPalette(c("#fee8c8", "#fdbb84", "#e34a33"))

colors <- color_settings_network_2(5)

breaks <- unique(quantile(V(network)$scMutScore, probs = seq(0, 1, length.out = 6)))

value_group <- cut(V(network)$scMutScore, breaks = breaks, include.lowest = TRUE, labels = FALSE)

my_color_2 <- colors[value_group]

edge_width <- log2(E(network)$value + 0.1)

vertex_frame_color <- color_settings_network_1[V(network)$group]

p_selected_clone_MEF <- ggplot(data = data_MEF_D3_D13[data_MEF_D3_D13$Tag == "MEF",], mapping = aes(x = clone_ID, y = scMutScore)) +
  geom_violin(trim = FALSE) + coord_cartesian(ylim = c(0, 1)) +
  geom_boxplot(width = 0.05, fill = "white", outliers = FALSE) +
  #stat_summary(fun.data="mean_sdl", fun.args = list(mult=1),geom="pointrange", color = "black") +
  scale_fill_manual(values = color_settings_2) +
  labs(x = "", y = paste0("scMutraceScore_", selected_start_cloneID)) +
  theme_classic()

p_selected_clone_D3 <- ggplot(data = data_MEF_D3_D13[data_MEF_D3_D13$Tag == "D3",], mapping = aes(x = clone_ID, y = scMutScore)) +
  geom_violin(trim = FALSE) + coord_cartesian(ylim = c(0, 1)) +

```

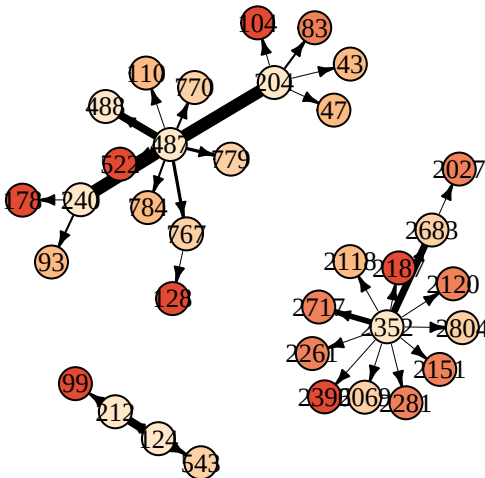
```

geom_boxplot(width = 0.05, fill = "white", outliers = FALSE) +
#stat_summary(fun.data="mean_sdl", fun.args = list(mult=1),geom="pointrange", color = "black") +
scale_fill_manual(values = color_settings_2) +
labs(x = "", y = paste0("scMutraceScore_", selected_start_cloneID)) +
theme_classic()

p_selected_clone_D13 <- ggplot(data = data_MEF_D3_D13[data_MEF_D3_D13$Tag == "D13",], mapping = aes(x =
geom_violin(trim = FALSE) + coord_cartesian(ylim = c(0, 1)) +
geom_boxplot(width = 0.05, fill = "white", outliers = FALSE) +
#stat_summary(fun.data="mean_sdl", fun.args = list(mult=1),geom="pointrange", color = "black") +
scale_fill_manual(values = color_settings_2) +
labs(x = "", y = paste0("scMutraceScore_", selected_start_cloneID)) +
theme_classic()

plot(network,
      vertex.color=my_color_2,
      vertex.label.color = "black",
      vertex.label.cex = 0.8,
      vertex.size = 15,
      #vertex.frame.width = 3,
      #vertex.frame.color = vertex_frame_color,
      edge.color = "black",
      edge.arrow.size = 0.4,
      edge.width = edge_width)

```



```
col_fun <- colorRampPalette(c("#fee8c8", "#fdbb84", "#e34a33"))
col_seq <- col_fun(length(my_color_2))
image(z = matrix(1:length(my_color_2), ncol = 1), col = col_seq,
      axes = FALSE, main = "Gradient Legend")
axis(4, at = seq(0, 1, length.out = 5), labels = round(seq(0, 1, length.out = 5), 2))
```

Gradient Legend



20 ModelFigure

```
m1 <- c(1, 1, 1, 1, 1, 1, 1, 1)
m2 <- c(0, 1, 1, 1, 1, 1, 1, 1)
m3 <- c(0, 0, 1, 0, 0, 0, 0, 0)
m4 <- c(0, 0, 0, 1, 1, 1, 1, 1)
m5 <- c(0, 0, 0, 0, 1, 1, 1, 1)
m6 <- c(0, 0, 0, 0, 0, 1, 0, 0)
m7 <- c(0, 0, 0, 0, 0, 0, 1, 0)
m8 <- c(0, 0, 0, 0, 0, 0, 0, 1)

t1 <- c(1, 0, 0, 0, 0, 0, 0, 0)
t2 <- c(1, 1, 0, 0, 0, 0, 0, 0)
t3 <- c(1, 1, 1, 1, 0, 0, 0, 0)
t4 <- c(1, 1, 1, 1, 0, 0, 0, 0)
```

```

t5 <- c(1, 1, 0, 1, 1, 0, 0, 0)
t6 <- c(1, 1, 0, 1, 1, 1, 1, 1)

test_Matrix_cm <- data.frame(m1 = m1, m2 = m2, m3 = m3, m4 = m4, m5 = m5, m6 = m6, m7 = m7, m8 = m8)
test_Matrix_mt <- data.frame(t1 = t1, t2 = t2, t3 = t3, t4 = t4, t5 = t5, t6 = t6)
rownames(test_Matrix_cm) <- c("c1", "c2", "c3", "c4", "c5", "c6", "c7", "c8")

test_Matrix_ct <- as.matrix(test_Matrix_cm) %*% as.matrix(test_Matrix_mt)
rownames(test_Matrix_ct) <- c("c1", "c2", "c3", "c4", "c5", "c6", "c7", "c8")

res <- cal_MultiSpe(test_Matrix_ct)

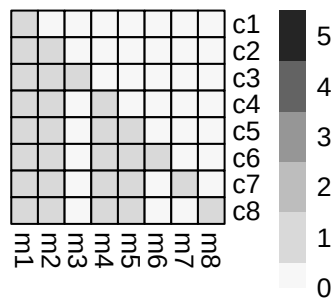
cal_scMutScore(test_Matrix_cm, test_Matrix_mt)
## [1] 8 8
## [1] 8 6
## $diversity
##      t1      t2      t3      t4      t5      t6 Diversity
## c1  0 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## c2  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
## c3  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
## c4  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
## c5  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
## c6  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
## c7  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
## c8  0 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.8333333
##
## $score
##      Shannon  Simpson      Gini sumMut      Tau      GiniTau scMutScore
## c1 1.791759 0.8333333 0.0000000      6 0.0000000 0.000000000 0.00000000
## c2 1.767761 0.8264463 0.07575716     11 0.1000000 0.007575716 0.07402306
## c3 1.737983 0.8165680 0.16666590     13 0.3333333 0.055555299 0.51324369
## c4 1.736741 0.8177778 0.14444387     15 0.2000000 0.028888773 0.27516374
## c5 1.711547 0.8096886 0.20588163     17 0.3500000 0.072058569 0.65356041
## c6 1.692020 0.8024691 0.24073994     18 0.4800000 0.115555170 1.00000000
## c7 1.692020 0.8024691 0.24073994     18 0.4800000 0.115555170 1.00000000
## c8 1.692020 0.8024691 0.24073994     18 0.4800000 0.115555170 1.00000000
##      Diversity
## c1 0.0000000
## c2 0.8333333
## c3 0.8333333
## c4 0.8333333
## c5 0.8333333
## c6 0.8333333
## c7 0.8333333
## c8 0.8333333
##
## $matrixD
##      t1 t2 t3 t4 t5 t6
## c1  1  1  1  1  1  1
## c2  1  2  2  2  2  2
## c3  1  2  3  3  2  2
## c4  1  2  3  3  3  3
## c5  1  2  3  3  4  4

```

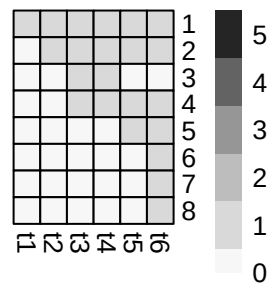
```
## c6 1 2 3 3 4 5
## c7 1 2 3 3 4 5
## c8 1 2 3 3 4 5
```

```
my_colors <- c("#f7f7f7", "#d9d9d9", "#bdbdbd", "#969696", "#636363", "#252525")
breaks <- c(0, 0.5, 1.5, 2.5, 3.5, 4.5, 5.5)
```

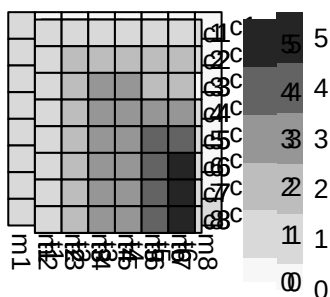
```
p_Matrix_cm <- pheatmap(test_Matrix_cm, cluster_rows = F, cluster_cols = F, color = my_colors, breaks =
```



```
p_Matrix_mt <- pheatmap(test_Matrix_mt, cluster_rows = F, cluster_cols = F, color = my_colors, breaks =
```



```
p_Matrix_ct <- pheatmap(test_Matrix_ct, cluster_rows = F, cluster_cols = F, color = my_colors, breaks =
p_Matrix_cm
p_Matrix_mt
p_Matrix_ct
```



21 sessionInfo

```
sessionInfo()
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.4.1
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Stockholm
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] clinfun_1.1.5      patchwork_1.3.2    lubridate_1.9.4    forcats_1.0.0
## [5] purrr_1.1.0        readr_2.1.5        tibble_3.3.0       tidyverse_2.0.0
```

```

## [9] networkD3_0.4.1      tidygraph_1.3.1      reshape2_1.4.4      pheatmap_1.0.13
## [13] corrplot_0.95        stringr_1.5.2        ggpmisc_0.6.2        ggpp_0.5.9
## [17] webshot_0.5.5        ggraph_2.2.2         igraph_2.1.4         Seurat_5.3.0
## [21] SeuratObject_5.2.0   sp_2.2-0             ggpubr_0.6.1         ggplot2_4.0.0
## [25] tidyr_1.3.1          dplyr_1.1.4          vegan_2.7-1          permute_0.9-8
## [29] ineq_0.2-13          FSA_0.10.0
##
## loaded via a namespace (and not attached):
## [1] RcppAnnoy_0.0.22      splines_4.5.1         later_1.4.4
## [4] polyclip_1.10-7       fastDummies_1.7.5     lifecycle_1.0.4
## [7] rstatix_0.7.2         processx_3.8.6        globals_0.18.0
## [10] lattice_0.22-7        MASS_7.3-65           backports_1.5.0
## [13] magrittr_2.0.4        plotly_4.11.0         rmarkdown_2.30
## [16] yaml_2.3.10           httpuv_1.6.16         sctransform_0.4.2
## [19] spam_2.11-1           spatstat.sparse_3.1-0 reticulate_1.43.0
## [22] cowplot_1.2.0         pbapply_1.7-4         RColorBrewer_1.1-3
## [25] abind_1.4-8           Rtsne_0.17            tweenr_2.0.3
## [28] data.tree_1.2.0       ggrepel_0.9.6         irlba_2.3.5.1
## [31] listenv_0.9.1         spatstat.utils_3.2-0  MatrixModels_0.5-4
## [34] goftest_1.2-3         RSpectra_0.16-2       spatstat.random_3.4-2
## [37] fitdistrplus_1.2-4    parallelly_1.45.1     codetools_0.2-20
## [40] ggforce_0.5.0         tidyselect_1.2.1      farver_2.1.2
## [43] viridis_0.6.5         matrixStats_1.5.0     spatstat.explore_3.5-3
## [46] jsonlite_2.0.0        progressr_0.16.0      Formula_1.2-5
## [49] ggribes_0.5.7         survival_3.8-3        tools_4.5.1
## [52] ica_1.0-3             Rcpp_1.1.0            glue_1.8.0
## [55] gridExtra_2.3         xfun_0.53             mgcv_1.9-3
## [58] withr_3.0.2           fastmap_1.2.0         SparseM_1.84-2
## [61] callr_3.7.6           digest_0.6.37         timechange_0.3.0
## [64] R6_2.6.1             mime_0.13             colorspace_2.1-2
## [67] scattermore_1.2       tensor_1.5.1          dichromat_2.0-0.1
## [70] spatstat.data_3.1-8   generics_0.1.4        data.table_1.17.8
## [73] graphlayouts_1.2.2    httr_1.4.7            htmlwidgets_1.6.4
## [76] uwot_0.2.3           pkgconfig_2.0.3       gtable_0.3.6
## [79] lmttest_0.9-40        S7_0.2.0              htmltools_0.5.8.1
## [82] carData_3.0-5         dotCall64_1.2          scales_1.4.0
## [85] png_0.1-8            spatstat.univar_3.1-4 knitr_1.50
## [88] rstudioapi_0.17.1     tzdb_0.5.0            nlme_3.1-168
## [91] zoo_1.8-14           cachem_1.1.0          KernSmooth_2.23-26
## [94] parallel_4.5.1        miniUI_0.1.2          pillar_1.11.1
## [97] grid_4.5.1           vctrs_0.6.5           RANN_2.6.2
## [100] promises_1.3.3        car_3.1-3             xtable_1.8-4
## [103] cluster_2.1.8.1       evaluate_1.0.5        mvtnorm_1.3-3
## [106] cli_3.6.5            compiler_4.5.1        rlang_1.1.6
## [109] future.apply_1.20.0    ggsignif_0.6.4        labeling_0.4.3
## [112] ps_1.9.1             plyr_1.8.9            stringi_1.8.7
## [115] viridisLite_0.4.2     deldir_2.0-4          lazyeval_0.2.2
## [118] spatstat.geom_3.6-0    quantreg_6.1          Matrix_1.7-4
## [121] RcppHNSW_0.6.0        hms_1.1.3             future_1.67.0
## [124] shiny_1.11.1          ROCR_1.0-11           broom_1.0.10
## [127] memoise_2.0.1         polynom_1.4-1

```